

班 级 2116029
学 号 21009100863

西安电子科技大学

本科毕业设计论文



题 目 RISCV 开源 IP 平台对于移植
开源飞控的资源匹配性研究

学 院 空间科学与技术学院

专 业 空间科学与技术

学 生 姓 名 何成楷

导 师 姓 名 韩高

西安电子科技大学

毕业设计（论文）诚信声明书

本人声明：本人所提交的毕业论文《RISC-V 开源 IP 平台对于移植开源飞控的资源匹配性研究》是本人在指导教师指导下独立研究、写作的成果，论文中所引用他人的无论以何种方式发布的文字、研究成果，均在论文中加以说明；有关教师、同学和其他人员对本文的写作、修订提出过并为我论文中加以采纳的意见、建议，均已在我的致谢辞中加以说明并深致谢意。

本论文和资料若有不实之处，本人承担一切相关责任。

论文作者：何成楷（签字） 时间：2025 年 5 月 20 日

指导教师已阅：韩勇（签字） 时间：2025 年 5 月 20 日

西安电子科技大学

毕业设计（论文）任务书

学生姓名 何成楷 学号 21009100863 指导教师 韩高 职称 讲师

学院 空间科学与技术学院 专业 空间科学与技术

题目名称 RISC-V 开源 IP 平台对于移植开源飞控的资源匹配性研究

任务与要求

本项目研究任务包括：

1. 选定 2-3 家国产化 RISC-V 平台（平头哥，芯来，沁恒）对应的商业化芯片作为研究对象
2. 从主频，计算性能，外设资源，工具链，周边传感器配套，电机驱动控制等维度，对标开源飞控 PX4，对比应用现有 STM32F4/H7 的优劣势
3. 研究和评估国产化 RISC-V 平台在微型行业应用无人机的应用可行性和待解决的问题
4. 研究 RISC-V 国产化芯片实现开源飞控平台全面替代的可行性，研究全自主芯片+全自主飞控的发展设计发展路径

开始日期 2025 年 11 月 25 日 完成日期 2025 年 6 月 5 号

院长（签字）周峰 2024 年 11 月 25 日

注：本任务书一式两份，一份交学院，一份学生自己保存。

西安电子科技大学
毕业设计（论文）工作计划

学生姓名 何成楷 学 号 21009100863

指导教师 韩高 职 称 讲师

学 院 空间科学与技术学院 专 业 空间科学与技术

题目名称 RISCv 开源 IP 平台对于移植开源飞控的资源匹配性研究

一、毕业设计（论文）进度

起止时间	工作内容
2024.11.25—2024.12.10	选定 2-3 家国产化 RISCv 平台（平头哥，芯来，沁恒）对应的商业化芯片作为研究对象
2024.12.10—2024.12.31	从主频，计算性能，外设资源，工具链，周边传感器配套，电机驱动控制等维度，对标开源飞控 PX4，对比应用现有 STM32F4/H7 的优劣势
2025.1.1—2025.2.10	研究和评估国产化 RISCv 平台在小微型行业应用无人机的应用可行性和

	待解决的问题
2025.2.10—2025.2.28	
2025.3.1—2025.3.22	进行实验和测试，验证无人机上核心功能模块的功能和性能。
2025.3.22—2025.3.31	研究 RISC-V 国产化芯片实现开源飞控平台全面替代的可行性，研究全自主芯片+全自主飞控的发展设计发展路径
2025.4.1—2025.4.25	整理材料，撰写毕业设计论文初稿。
2025.4.25—2025.5.30	修改定稿，进行论文评审，毕设答辩等相关工作。

二、主要参考书目（资料）

[1] 李若曦,陈杰,刘威.基于芯来蜂鸟 E203 处理器的架构优化[J].电子设计工程,2025,33(08):6-11+16. DOI:10.14022/j.issn1674-6236.2025.08.002.

[2] Valence, L., Nadalini, A., Veeran, A. H. C., Sinigaglia, M., Sá, B., & Wistoff, N. (2024). A heterogeneous RISC-V based SoC for secure nano-UAV navigation. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 71(5), 2266-2279.

[3] 邓雅静.AI 时代下兆易创新 MCU 的跃升之路[J].电器,2025,(03):10.

[4] 李国斌.基于 STM32 和 HAL 库的嵌入式四轴无人机项目设计[J].信息与电脑,2024,36(24):20-23.

[5] Kühne, J., Magno, M., & Benini, L. (2022). Parallelizing optical flow estimation on an ultra-low power RISC-V cluster for nano-UAV navigation. In 2022 IEEE International Symposium on Circuits and Systems (ISCAS) (pp. 1-5). IEEE. <https://doi.org/10.1109/ISCAS48785.2022.9937215>.

[6] 刘东,王瑞锦,赵彦钧,等.基于 ARM 架构的边缘计算服务器关键平台研究[J].计算机科学,2024,51(S1):746-753.

- [7] 周志广,许孟孟,石梅林,等.基于 STM32F4 与 FreeRTOS 的无人机飞行控制系统[J].航空工程进展,2024,15(04):93-99.DOI:10.16615/j.cnki.1674-8190.2024.04.11.
- [8] 杨龙飞.基于 RISC-V 的 SoC 设计与实现[D].湖北大学,2024.DOI:10.27130/d.cnki.ghubu.2024.001891.
- [9] Nikiema P R, Palumbo A, Aasma A, et al. Towards Dependable RISC-V Cores for Edge Computing Devices[C]. In: Procee.
- [10] dings of 2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS), Crete, Greece, 2023: 03-05 July.
- [11] 马建业,许致火,汪月霞.基于四轴无人机的快递投递系统仿真[J].电讯技术,2024,64(08):1257-1265. DOI:10.20079/j.issn.1001-893x.230509004.
- [12] 周成,孙健,赵天骐,等.基于 STM32 的小型机器人控制系统电路设计[J].机器人产业,2023,(04):72-77.DOI:10.19609/j.cnki.cn10-1324/tp.2023.04.006.
- [13] 刘鑫.面向 RISC-V 架构处理器的集成开发环境设计与实现[D].苏州大学,2023.DOI:10.27351/d.cnki.gsyzhu.2023.001253.
- [14] 贾思敏.RISC-V 处理器核设计优化与扩展指令集实现研究[D].山东大学,2023.DOI:10.27272/d.cnki.gshdu.2023.002851.
- [15] Bian S, Schulthess L, Rutishauser G, et al. ColibriUAV: An Ultra-Fast, Energy-Efficient Neuro-morphic Edge Processing UAV-Platform with Event-Based and Frame-Based Cameras[C]. In: Proceedings of 2023 9th International Workshop on Advances in Sensors and Interfaces (IWAS I), Monopoli (Bari), Italy, 2023: 08-09 June.
- [16] 李少童,何明兴,李大彬,等.基于 STM32F4 的四轴无人机设计[J].电子制作,2023,31(07):29-33.DOI:10.16589/j.cnki.cn11-3571/tn.2023.07.014.
- [17] 李娇,向洲,陈华,等.基于 MPU6050 六轴陀螺仪的高精度动态水平姿态监测系统的开发与应用[J].河南科技,2025,52(05):38-43.DOI:10.19968/j.cnki.hnkj.1003-5168.2025.05.007.
- [18] 熊赢超.四轴无人机的设计与优化[J].机械工程与自动化,2022,(05):124-125+128.
- [19] 张荣辉,严国峰.基于 STM32 四轴飞行器的设计与实现[J].数字技术与应用,2022,40(06):210-212. DOI:10.19695/j.cnki.cn12-1369.2022.06.64.
- [20] 梁日杰.小型四轴无人机目标识别与跟踪技术研究[D].长春理工大学,2022.DOI:10.26977/d.cnki.gcgc.2022.000581.
- [21] 唐陈辉.一种基于 RISC-V 架构的嵌入式 SoC 系统设计[D].西安电子科技大学,2022.DOI:10.27389/d.cnki.gxadu.2022.000521.
- [22] 余宇宙.基于 PX4 的小型无人机导航技术开发[D].南京航空航天大学,2022.DOI:10.27239/d.cnki.gnhhu.2022.000105.
- [23] 杨超琼,韦发清.STM32 的四轴飞行器的设计[J].电子世界,2022,(01):187-188.DOI:10.19353/j.cnki.dzsj.2022.01.087.
- [24] 刘畅,武延军,吴敬征,等.RISC-V 指令集架构研究综述[J].软件学报,2021,32(12):3992-4024.DOI:10.13328/j.cnki.jos.006490.

- [25] 顾兆军,陈辉,王家亮,等.一种小型四轴飞行器目标跟踪控制算法[J].西安电子科技大学学报,2021,48(05):117-127+177.DOI:10.19665/j.issn1001-2400.2021.05.015.
- [26] 丁多斌,冯向萍,丁燕,等.基于 STM32 四轴无人机自主巡线设计[J].电子设计工程,2021,29(02):142-145.DOI:10.14022/j.issn1674-6236.2021.02.031.
- [27] 陈韩悦,刘晓航,王品智,等.基于 STM32 的小型四轴飞行器编队控制系统设计[J].现代计算机,2021,(01):115-120.
- [28] 李想,李阳.四轴无人机在林业管理中的应用[J].广西林业科学,2020,49(02):296-299.DOI:10.19692/j.cnki.gfs.2020.02.028.
- [29] 宋权霖,张伟,刘业钊,等.四轴飞行器系统设计与制作[J].电子制作,2020,(07):20-22.DOI:10.16589/j.cnki.cn11-3571/tn.2020.07.007.
- [30] 张治,谢峰,董敏周,等.基于 RaspberryPi 的飞控系统快速原型设计[J].兵器装备工程学报,2020,41(09):182-186.
- [31] 王筱淇,刘震磊,李颖,等.四轴无人机总体结构模块化设计[J].中国科技信息,2020,(02):21-23.
- [32] 李成诚,乔东海.四轴无人机姿态解算的 FPGA 实现[C]//中国声学学会微声学分会,四川省声学学会,上海市声学学会,山东省声学学会,浙江省声学学会.2019 中国西部声学学术交流会论文集.苏州大学电子信息学院;中国科学院声学研究所,;2019:362-364.DOI:10.26914/c.cnkihy.2019.062463.
- [33] 肖猷坤,梁勇东,曹金梅,等.四轴无人机群协同控制系统的开发[J].机电信息,2019,(06):48-49.DOI:10.19514/j.cnki.cn32-1628/tm.2019.06.028.
- [34] 张宏伟,钱叶册,许卫兵.基于 STM32 的微型四轴救援机的设计与实现[J].现代信息科技,2019,3(01):158-159.
- [35] 李晓英,刘敏.基于 RISC-V-32I 的微处理器的设计与实现[J].内江科技,2018,39(11):49-50+35.
- [36] 王亭岭,芦杜洋,马跃涛.PIXHAWK 开源飞控的多旋翼无人机避障技术研究[J].单片机与嵌入式系统应用,2017,17(10):30-32.
- [37] 姜俊俊,李明,张宇.基于视觉的四旋翼无人机目标跟踪系统设计[J].中国科技信息,2016,(14):65-66.
- [38] 贺欢.世界微小型无人机最新发展应用概览[J].中国安防,2015,(15):82-95.
- [39] 冀亮,钱正洪,白茹.基于四元数的四轴无人机姿态的估计和控制[J].现代电子技术,2015,38(11):112-116.DOI:10.16652/j.issn.1004-373x.2015.11.004.
- [40] 王芳良,张伟功,于立新,等.基准集在嵌入式系统性能分析中的应用[J].计算机工程与设计,2015,36(01):115-119+126.DOI:10.16208/j.issn1000-7024.2015.01.021.

三、主要仪器设备及材料

计算机（虚拟机 Linux，VSCode 或 Arduino，APM 或 PX4）

无人机

微控制器（Tangnano9k,）

四、教师的指导安排情况（场地安排、指导方式等）

答疑时间：每周日晚上安排一次固定时间为学生指导答疑，并根据实际情况及时调整答疑时间。

答疑场地：西安电子科技大学南校区 G216 办公室

实验场地：西安电子科技大学南校区 G230 实验室

指导方式：一方面根据计划安排给学生定时安排任务并进行检查，另一方面根据学生做的实际情况进行指导，以解答学生提出的问题为主，并适当给学生做出建议和指引。

五、对计划的说明

注：1.本计划一式两份，一份交学院，一份学生自己保存（计划双面打印）。

2.各学院可根据本学院专业认证要求及细则对表格进行修改，修改后的表格需向本科生院备案。

西安电子科技大学 空间科学与技术学院

本科生毕业论文（设计）开题报告
(2025 届)

学生姓名 何成楷

专 业 空间科学与技术

学 号 21009100863

指导教师 韩高

2024 年 12 月 30 日

（本表一式三份，学生、指导教师、学院各一份）

一、论文名称及项目来源

论文名称：**RISCV 开源 IP 平台对于移植开源飞控的资源匹配性研究**

项目来源：随着全球供应链的变化和国内技术自主化的推进，国产化 RISCV 平台的发展对于保障国家安全和产业安全具有重要意义。国产 RISCV 平台能够提供更加灵活的定制选项，满足特定应用场景的需求。无人机飞控系统是无人机的“大脑”，负责处理飞行数据、控制飞行姿态和路径。目前，市场上主流的飞控系统多采用 STM32 系列芯片，但随着技术的发展，RISCV 平台因其高性能和低功耗的特点，逐渐成为无人机飞控系统的新选择。该项目旨在深入研究和评估国产 RISCV 平台在无人机飞控系统中的应用潜力和资源匹配性，以期推动国内无人机技术的发展和自主化进程。

二、研究目的和意义

研究目的：选定 2-3 家国产化 RISCV 平台（平头哥，芯来，沁恒）对应的商业化芯片作为研究对象。研究和评估国产化 RISCV 平台在小微型行业应用无人机的应用可行性和待解决的问题。研究 RISCV 国产化芯片实现开源飞控平台全面替代的可行性，研究全自主芯片+全自主飞控的发展设计发展路径。

三、国内外研究现状和发展趋势

RISC-V 的国内外研究现状：

指令集（Instruction Set Architecture,ISA）是计算机体系结构中用于定义计算机的硬件可以执行哪些操作的一套规则和规范。它包括一系列的指令，这些指令定义了处理器能够理解和执行的操作。指令集架构是计算机硬件和软件之间的接口，它决定了软件如何控制硬件执行任务。

指令集架构可以分为 CISC 和 RISC 两大类，CISC 是复杂指令集，RISC 是精简指令集。CISC 在处理器诞生的早期，曾经是主流 ISA，然而，随着复杂指令集 CISC 的不断发展，越来越多不常用的特殊指令被添加进来，而这些不常用的特殊指令让处理器设计起来变得十分复杂，需要占用更多的设计面积以及更多的开

发时间，为了避免这种情况，开发人员开始对指令集进行简化，并在设计处理器方面尽可能多的使用常用的简单指令，减少使用那些不常用的特殊指令，固定指令长度和使用大量寄存器等，以此来减少处理器设计的复杂度和提高处理器的频率，于是 RISC 便应运而生了，例如 ARM 和 MIPS 架构都属于 RISC 架构。

RISC-V 指令集架构的官方处理器实现项目名为 Rocket Chip，一个完全开源的 RISC-V 处理器生成器，由加州大学伯克利分校的 RISC-V 研发团队推出，也是最符合规范的硬件实现，源代码托管在 GitHub 平台上。Rocket Chip 是一个开源的项目，使用开源许可证 BSD（Berkeley Software Distribution，伯克利软件套件），用 Chisel（Constructing Hardware In a Scala Embedded Language，嵌入 Scala 的硬件描述语言）编写，可以用来生成 Rocket 内核和 BOOM 内核，该团队还推出了基于 Rocket Chip 的开源 SoC 项目 freedom，其源代码同样托管在 GitHub 平台上。谷歌公司非官方地推出了一个开源 RISC-V 产品，BottleRocket，BottleRocket 是一个基于 Rocket Chip 的自定义的微处理器结构，实现了 RV32IMC 指令集架构，也就是基础指令集 RV32I 和扩展指令集 RV32M 和 RV32C。BottleRocket 希望用尽量简单的设计实现 RISC-V，只使用了 Rocket Chip 的几个关键部件，包括指令译码器和 CSR（Control Status Register，控制状态寄存器）。谷歌公司还发起了开源项目 OpenTitan，将管理权交由 lowRISC，通过与诸多院校和公司的合作，旨在基于 zero-riscy 内核设计通用的芯片加密模块，苏黎世联邦理工学院、捷德集团、新唐科技和西部数据都是该项目的成员。

lowRISC 是一个由剑桥大学的工程师团队组成的非营利性组织，推出了基于 Rocket 内核的 64 位开源 SoC 平台 lowRISC chip，实现了 RV32IM 指令集架构，正在计划增加对苏黎世联邦理工学院开发的 64 位内核 Ariane 的支持。苏黎世联邦理工学院与波罗尼亚大学一同推出开源 PULP（Parallel Ultra-Low-Power）平台，使用项目团队开发的 32 位 RISC-V 处理器内核 CV32E40P 或者 Ibex，或者 64 位的 RISC-V 处理器内核 Ariane 作为主核。CV32E40P 本是基于 OpenRISC 架构的 ORION 处理器核心的一部分，后来被 PULP 团队以 riscv 为名改成了一个 RISC-V 内核进行使用和维护。CV32E40P 实现了 RV32IMC 指令集架构，是一个有序执

行的 RISC-V 处理器内核，包含 4 级流水线。2020 年 2 月被贡献给 OpenHW，现托管在 OpenHW 的 GitHub 官网上。

2015 年，加州大学伯克利分校的 RISC-V 研发团队成立了商业公司 SiFive，推出了可由客户自定义的 RISC-V IP 核，以及同样支持定制的 SoC 平台，包括针对低功耗需求的 Freedom Aware 平台，面向 IoT 领域的 Freedom Everywhere 平台，针对高性能需求的 Freedom Unleashed 平台，以及面向 AI 领域的 Freedom Revolution 平台。

越来越多的国家也开始对 RISC-V 重视起来，RISC-V 本身就是在美国国防高级研究计划局（Defense Advanced Research Projects Agency，DARPA）资助的项目中出现的，印度的几个自研处理器的政府项目都转向了 RISC-V 架构。在中国，RISC-V 也引起了设计者们的极大兴趣。2018 年 11 月，中国开放指令生态联盟（China RISC-V Alliance，CRAV）成立，旨在基于 RISC-V 建立起中国的开源芯片生态，多个研究机构、企业、院校和个人设计者都是其成员。

蜂鸟 E203 是中国第一个开源 RISC-V 处理器内核，为国内 RISC-V 处理器的研究者和院校提供了极好的实践案例，推动了国内 RISC-V 生态的发展。后来，蜂鸟 E203 的开发者胡振波创立了芯来科技，推出了 N100、N200、N300、N600、N900 等多个系列的 RISC-V 处理器，为多家企业提供了定制 RISC-V 处理器 IP 核。芯来科技与兆易创新合作，为其定制了 RISC-V 处理器内核 Bumblebee，兆易创新基于此推出了 GD32VF103 系列的量产通用微处理器。

阿里平头哥也致力于将 RISC-V 架构应用于自研的玄铁系列处理器上，推出了 64 位 RISC-V 处理器内核 C910 和兼容 RISC-V 的 E902，在 AIoT 领域的生态中占据了重要位置。此外，华米科技推出了针对可穿戴设备的基于 RISC-V 的 AI 处理器黄山 1 号，于 2019 年 8 月宣布量产，并应用于商业产品 Amazfit 米动健康手环上。上海交通大学与瓶钵信息科技合作研发了“蓬莱”安全架构，一个可应用于 RISC-V 平台的开源 TEE（Trusted Execution Environment，可信执行环境）系统，同时也丰富了 RISC-V 的指令集扩展。

众所周知，芯片产业是投入极高、回报极慢的领域，而 RISC-V 提供了免费

开源、开发周期较短的解决方案。面对国外芯片的生态和专利壁垒，RISC-V有望成为我国自主研发处理器芯片的一个极好的选择，或许能为国产芯片市场的繁荣提供机遇。

开源飞控平台的国内外发展现状介绍：

开源 Ardupilot/APM：

APM 是在 2007 年由 DIY 无人机社区（DIY Drones）推出的飞控系统。也是迄今为止最为成熟的开源自动导航系统，可支持多旋翼、固定翼、直升机和无人驾驶车等无人设备。APM 基于 Arduino 的开源平台，对多处硬件做出了改进，包括加速度计、陀螺仪和磁力计组合惯性测量单元(IMU)。由于 APM 良好的可定制性，APM 在全球航模爱好者范围内迅速传播开来。通过开源软件 Mission Planner，开发者可以配置 APM 的设置，接收并显示传感器的数据，使用 Google map 完成自动驾驶等功能，但是 Mission Planner 仅支持 windows 操作系统。目前，APM 飞控已经成为开源飞控成熟的标杆，针对多旋翼 APM 飞控支持各种四、六、八轴产品，并且连接外置 GPS 传感器以后能够增稳，并完成自主起降、自主航线飞行、回家、定高、定点等丰富的飞行模式。APM 能够连接外置的超声波传感器和光流传感器，在室内实现定高和定点飞行。

APM 系列发展至今，APM2.5 和 APM2.6 已经是 ardupilot 飞控最终版本，APM 给我们带来非常强大的功能，非常的成熟可靠，潜能被充分挖掘出来，功能也非常丰富。但源于 APM 系列 8 位 CPU 计算与存储的能力已经远远不能够满足未来的运用需求了，APM 系列产品的终结也是势在必行。PM 系列自动导航板包括几款不同型号，适用于各种飞行控制需求，PX4 是一款基于 32 位 ARM 架构的自动导航仪，支持许多高级功能，采用 NuttX 实时操作系统。APM2 是一款较为流行的 8 位 AVR2560 自动导航仪，适用于多种飞行控制应用。由于 ArduPilot/APM 的源码基于 AP-HAL 硬件抽象层编写，开发人员可以轻松扩展和支持更多不同型号的自动导航板。这一设计让 APM 系列具备较高的兼容性。

ArduPilot/APM 的飞行控制代码主要采用 C++语言编写，而开发工具则使用多种编程语言，最常用的是 Python。

ArduPilot/APM 支持多种地面站，用于飞行计划和实时控制。飞行固件使用 MAVLink 协议，能够与任何 MAVLink 兼容的设备进行通信和控制。Mission Planner 是最常用的地面站软件，使用 C#语言编写，适用于 Windows 系统，也可以通过 Mono 运行在 Linux 和 MacOS 上。源码可以在 GitHub 上找到。APM 系列使用 AvrAtmega1280/2560 作为主控芯片，配备有多种传感器，包括双轴陀螺仪、IMU（单轴陀螺仪、三轴加速度计、三轴磁力计模块）、气压计以及 AD 芯片。系统采用 Arduino IDE 作为编译环境，支持用 Arduino 语言进行开发，语言结构类似于 C 语言。APM 系列采用两级 PID 控制算法，第一层为导航级，第二层为控制级，确保飞行的精度和稳定性。硬件方面，APM2.5 板载电子罗盘，而 APM2.6 则将电子罗盘外置，并且融合了 GPS 模块，提供更精确的导航功能。APM 系列自动导航板以其灵活的硬件支持、多样化的软件工具和强大的开发环境，广泛应用于各种无人载具

开源平台 PX4:

PX4 是一个开源的无人飞行器控制系统（UAV Autopilot），它由苏黎世联邦理工学院的计算机视觉与几何实验室开发，并得到了全球贡献者的共同维护。PX4 以其高度模块化、可扩展性强、支持多种飞行模式等特点，在无人机领域备受欢迎。

PX4 是由 PIXHAWK 项目演变而来，目前由 Dronecode 基金会进行支持和运行，遵循 BSD 协议，该协议允许将开源软件二次开发后用作商业用途，PX4 的设计是高度模块化的，允许开发者根据需要定制和替换不同的组件，极大地提升了系统的灵活性和可扩展性，PX4 可以在不同的硬件平台上运行，包括 Pixhawk 系列飞控板以及一些基于 Linux 的计算平台，这使得 PX4 具有广泛的应用场景，其提供了经过优化的飞行控制算法，支持多旋翼、固定翼、VTOL 等多种飞行模式，确保飞行稳定高效。同时还拥有一个庞大的全球社区，提供技术支持、文档和教程，是 PX4 不断发展壮大的基石。

PX4 的架构由以下几部分组成：1、飞行控制器硬件（Flight Controller Hardware）：PX4 支持多种硬件平台，包括 Pixhawk、Pixhawk 2.1、Cube 系列以

及一些定制硬件。飞行控制器负责处理所有飞行控制任务，如姿态估计、控制算法、传感器数据融合等。

2、NuttX 实时操作系统：PX4 运行在 NuttX RTOS 上，这是一个轻量级、高效能的实时操作系统，专门设计用于嵌入式系统。NuttX 支持多任务处理、内存管理、I/O 操作等功能，确保飞行控制任务的稳定执行。

3、飞行控制算法：PX4 采用了多种控制算法，包括 PID 控制、卡尔曼滤波、航向控制、位置控制等，用于稳定和精确控制飞行器的运动。它还支持多种高级算法，如视觉定位、自动避障、基于地图的导航等。

4、MAVLink 协议：PX4 使用 MAVLink 协议与外部地面站、传感器和其他设备进行通信。MAVLink 是一种轻量级的消息协议，广泛应用于无人机通信中。通过 MAVLink，PX4 可以与 Mission Planner、QGroundControl 等地面站实现数据交换和飞行控制。

5、地面站软件（Ground Control Software）：PX4 支持多种地面站软件的交互，如 QGroundControl（跨平台，支持 Windows、Linux、MacOS）和 Mission Planner（Windows 平台）。这些软件可用于飞行任务规划、飞行数据监控、参数调节等功能。

PX4 支持多种类型的飞行器，包括多旋翼（如三旋翼、四旋翼等）、直升机（单旋翼带尾桨、共轴双旋翼直升机）、飞艇、固定翼无人机（飞翼、标准布局等）、无人车及无人船、水下机器人（如六自由度 BlueRov2）、垂起 Vtol（标准垂起、倾转式垂起、尾座式垂起）以及旋翼机

四、主要研究内容、要解决的问题及本文的初步方案

研究内容：选定若干家国产化 RISC-V 平台（平头哥，芯来，沁恒）对应的商业化芯片作为研究对象。从主频，计算性能，外设资源，工具链，周边传感器配套，电机驱动控制等维度，对比应用现有 STM32F4/H7 的优劣势。研究和评估国产化 RISC-V 平台在小微型行业应用无人机的应用可行性和待解决的问题。研究 RISC-V 国产化芯片实现开源飞控平台全面替代的可行性，研究全自主芯片+全自主飞控的发展设计发展路径

初步方案：关注芯片选型分析时，主要关注以下对比维度：在考虑工具链兼容性

的情况下，该芯片对于内核能力/指令集的支持，通过主频这一关键指标对比计算能力是否满足基本要求，考察内存大小来对比是否满足计算空间的复杂度。外设接口方面，考察维度则包括 UART/I2C/SPI 相关的数字外设接口能力，接口协议的适用性。驱动能力方面，关注 PWM 和 DAC 指标。同时为了开发便利性的考量，还需要考察 IDE 支持、烧录调试的支持，以及样例工程的可用性，代码是否友好等等。

目前的初步选型方案是使用基于 MiniFLy 或者 PCBfly 改良后的软硬件框架，使用沁恒 CH32V208WB 的 riscv 芯片进行开源飞控平台的移植，完成移植后测试系统完整性以及飞控平台性能。

主要参考书目（资料）

- [1] 裴晓芳. RISC-V 架构嵌入式系统原理与应用: CH32V103 单片机编程与项目实践[M]. 北京航空航天大学出版社, 2021.
- [2] 张毅刚. 单片机原理及应用——C51 编程+Proteus 仿真[M]. 3 版. 北京: 高等教育出版社, 2021.
- [3] 陈文智, 王总辉. 嵌入式系统原理及设计[M]. 北京: 清华大学出版社, 2011.
- [4] 荣政. 数据结构与算法分析[M]. 西安: 西安电子科技大学出版社, 2012.
- [5] 楼顺天, 周佳社, 张伟涛. 微机原理与技术接口[M]. 3 版. 北京: 科学出版社, 2020.
- [6] 刘畅, 武延军, 吴敬征, 赵琛. RISC-V 指令集架构研究综述. 软件学报, 2021, 32(12): 3992–4024.
- [7] 赵坤. 基于 RISC-V 架构的 SoC 设计与实现[D]. 海南大学, 2021.
- [8] 邓紫珊. 基于 RISC-V 的 SoC 设计及其 RTOS 移植[D]. 电子科技大学, 2020.
- [9] 雷思磊. RISC-V 架构的开源处理器及 SoC 研究综述[J]. 单片机与嵌入式系统应用, 2017, 17(02): 56-60.
- [10] 胡振波. 手把手教你设计 CPU--RISC-V 处理器[M]. 北京: 人民邮电出版社, 2018. 25-26.

[11] Celio, Christopher, Patterson, et al. The Berkeley Out-of-Order Machine (BOOM): An Industry-Competitive, Synthesizable, Parameterized RISC-V Processor[R]//EECS Department, University of California, Berkeley, 2015.

五、工作的主要阶段、进度和完成时间

2022.12.01—2023.01.14

安排论文具体任务，理解任务的含义及要求，查阅相关资料，完成开题报告。

2023.02.17—2023.02.29

完成英文文献的翻译，熟悉开发工具和设计软件。

2023.03.01—2023.03.15

查阅文献及相关资料，确定飞控平台的移植方案。

2023.03.16—2023.03.31

完成 riscv 飞控平台的 PCB 设计、制版和初步调试。

2023.04.01—2023.04.15

完成 PCB 的优化、修改和最终方案调试。

2023.04.16—2023.04.30

整理资料，撰写毕业论文初稿。

2023.05.01—2023.05.30

修改定稿、论文评审、毕业答辩等相关工作。

六、已进行的前期准备工作

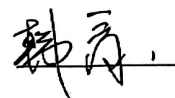
目前已经进行了初步的设计规划，相关文献和文档的阅读以及软件的安装，了解了 MiniFly 平台的基本硬件架构设计和系统运行原理，理解学习了相关的飞行控制算法，使用 MounRiver 管理和编译工程文件并进行仿真

七、指导教师意见

该生通过初期文献检索与指导老师充分讨论，确定并明确且具有一定研究价值的课题。本课题通过初步确定的技术指标、设计思路和方案较为明确合理，并具有一定的前沿创新价值和工作量。

同意该课题开题。

签名



2025 年 1 月 2 日

八、学院审核意见

通过

签名



2025 年 1 月 3 日

西安电子科技大学
本科生毕业论文（设计）中期报告

题 目 RISCV 开源 IP 平台对于移植
开源飞控的资源匹配性研究

学生姓名 何成楷

专 业 空间科学与技术

学 号 21009100863

指导教师 韩高

报告日期 2025 年 3 月 15 日

西安电子科技大学本科生院制

西安电子科技大学

本科生毕业设计（论文）中期报告要求

一、本科生在完成学位论文开题之后三个月内，必须进行学位论文中期考核，考核由各学院自行组织，具体要求参照毕业设计文件执行。

二、中期考核结论分为两种：1. 通过，按专家意见修改后继续学位论文撰写工作；2. 不通过，重新考核，正式答辩前达不到通过标准的，答辩延期进行。

三、中期报告由学生填写，填写完成后，需在限定时间内，在教务系统中上传最终版（如有更新，可重新上传覆盖）。

四、中期考核时需携带此表，本表一式三份，本人、指导教师、学院各一份，用 A3 纸张正反套印；承担毕业设计单位审核盖章后的表格最终胶装入存档论文中。

五、表格填写要求：正文字体宋体，字号小四，行间距固定值 20 磅，可续页，请勿更改表格样式。

1、毕业设计工作是否更换题目及是否按开题报告预定的内容及进度安排进行

按照开题报告预定的内容及进度安排进行

2. 目前已完成的研究工作及结果（内容要详实充分）

本研究旨在深入评估国产 RISC-V 平台在移植开源飞控系统（如 PX4）方面的资源匹配性，以及其在小微型行业应用无人机领域的应用前景和潜在问题。通过选定 2 - 3 家国产化 RISC-V 平台（如平头哥、芯来、沁恒）对应的商业化芯片作为研究对象，从主频、计算性能、外设资源、工具链、周边传感器配套、电机驱动控制等维度，对比应用现有 STM32F4/H7 的优劣势，全面分析国产 RISC-V 平台在无人机飞控系统中的应用性。同时，研究 RISC-V 国产化芯片实现开源飞控平台全面替代的可行性，探索全自主芯片+全自主飞控的发展设计路径，为推动国产芯片在无人机飞控领域的应用提供理论依据和实践指导。

该研究具有重要的理论和实践价值。从理论层面来看，通过对国产 RISC-V 平台与开源飞控系统的适配性研究，能够丰富和完善无人机飞控系统的技术理论体系，为后续相关研究提供参考和借鉴。从实践层面来看，研究结果将有助于推动国产芯片在无人机飞控领域的应用，打破国外芯片在该领域的垄断地位，提高我国无人机产业的自主可控能力。此外，探索全自主芯片+全自主飞控的发展设计路径，对于我国无人机行业的技术创新和产业升级具有重要的引领作用，能够为我国无人机产业在全球市场中赢得更大的竞争优势，促进我国无人机产业的可持续发展。

为了全面评估国产 RISC-V 平台芯片在飞控系统中的应用性，本研究将从以下维度进行对比分析：

1. 主频与计算性能

主频和核心数是衡量芯片计算能力的关键指标。通过对比各平台芯片的主频、核心数和浮点运算能力，评估其在处理复杂飞控算法时的性能表现。例如，高性能的 RISC-V 芯片能够支持更复杂的控制策略和实时数据处理。

2. 外设资源与接口多样性

飞控系统需要连接多种传感器和执行器，因此外设接口的种类和数量是重要的考量因素。对比各平台芯片支持的 UART、I2C、SPI、PWM 等接口的数量和通信速率，评估其在系统集成中的适用性。

3. 功耗与能效比

低功耗设计和高能效比能够显著提升无人机的续航能力。通过对比各平台芯片在不同工作模式下的功耗表现，评估其在实际应用场景中的能效优势。

4. 工具链与开发环境

完善的工具链和友好的开发环境是芯片成功应用的基础。对比各平台提供的编译器、调试器、开发文档和社区支持，评估其对开发效率和系统稳定性的贡献。

5. 安全性与可靠性 飞控系统的安全性直接关系到飞行安全，因此芯片需要具备高可靠性和安全性。对比各平台芯片的安全特性（如加密模块、安全启动）和故障检测与恢复机制，评估其在复杂环境中的稳定性。 6. 兼容性与扩展性 飞控平台需要支持多种硬件组件和软件模块，因此芯片的兼容性和扩展性至关重要。通过对比各平台芯片的模块化设计和 API 接口，评估其在功能扩展和定制化方面的灵活性。																										
<table><tr><th>参数类别</th><th>CH32V203</th><th>CH32V208</th></tr><tr><td>内核与性能</td><td>青稞 32 位 RISC-V 内核（V4B），支持多种指令集组合，主频最高 144MHz，单周期乘法、硬件除法</td><td>青稞 32 位 RISC-V 内核（V4C），支持多种指令集组合，主频最高 144MHz，单周期乘法、硬件除法</td></tr><tr><td>存储资源</td><td>最大 64KB-SRAM，最大 128KB-Flash</td><td>最大 64KB-SRAM，最大 480KB-Flash</td></tr><tr><td>外设接口</td><td>4 个 USART、2 个 I2C、2 个 SPI、1 个 CAN、USB2.0 全速设备接口、USB2.0 全速主机/设备接口</td><td>8 个 USART/UART、2 个 I2C、2 个 SPI、1 个 CAN、USB2.0 全速设备接口、USB2.0 全速主机/设备接口、10M 以太网 PHY、低功耗蓝牙 BLE5.3</td></tr><tr><td>通信能力</td><td>支持 USB2.0 全速设备和主机功能，无以太网和蓝牙功能</td><td>支持 USB2.0 全速设备和主机功能，支持 10M 以太网 PHY 和低功耗蓝牙 BLE5.3</td></tr><tr><td>功耗控制</td><td>工作和睡眠功耗同比大幅下降，支持多种低功耗模式</td><td>支持多种低功耗模式，具体功耗表现未明确提及</td></tr><tr><td>封装形式</td><td>LQFP、QFN、TSSOP 和 QSOP</td><td>LQFP 和 QFN</td></tr><tr><td>适用场景</td><td>工业级低功耗通用微控制器，适用于中小容量通用型应用</td><td>工业级通用微控制器，适用于大容量通用型和无线型应用</td></tr></table>			参数类别	CH32V203	CH32V208	内核与性能	青稞 32 位 RISC-V 内核（V4B），支持多种指令集组合，主频最高 144MHz，单周期乘法、硬件除法	青稞 32 位 RISC-V 内核（V4C），支持多种指令集组合，主频最高 144MHz，单周期乘法、硬件除法	存储资源	最大 64KB-SRAM，最大 128KB-Flash	最大 64KB-SRAM，最大 480KB-Flash	外设接口	4 个 USART、2 个 I2C、2 个 SPI、1 个 CAN、USB2.0 全速设备接口、USB2.0 全速主机/设备接口	8 个 USART/UART、2 个 I2C、2 个 SPI、1 个 CAN、USB2.0 全速设备接口、USB2.0 全速主机/设备接口、10M 以太网 PHY、低功耗蓝牙 BLE5.3	通信能力	支持 USB2.0 全速设备和主机功能，无以太网和蓝牙功能	支持 USB2.0 全速设备和主机功能，支持 10M 以太网 PHY 和低功耗蓝牙 BLE5.3	功耗控制	工作和睡眠功耗同比大幅下降，支持多种低功耗模式	支持多种低功耗模式，具体功耗表现未明确提及	封装形式	LQFP、QFN、TSSOP 和 QSOP	LQFP 和 QFN	适用场景	工业级低功耗通用微控制器，适用于中小容量通用型应用	工业级通用微控制器，适用于大容量通用型和无线型应用
参数类别	CH32V203	CH32V208																								
内核与性能	青稞 32 位 RISC-V 内核（V4B），支持多种指令集组合，主频最高 144MHz，单周期乘法、硬件除法	青稞 32 位 RISC-V 内核（V4C），支持多种指令集组合，主频最高 144MHz，单周期乘法、硬件除法																								
存储资源	最大 64KB-SRAM，最大 128KB-Flash	最大 64KB-SRAM，最大 480KB-Flash																								
外设接口	4 个 USART、2 个 I2C、2 个 SPI、1 个 CAN、USB2.0 全速设备接口、USB2.0 全速主机/设备接口	8 个 USART/UART、2 个 I2C、2 个 SPI、1 个 CAN、USB2.0 全速设备接口、USB2.0 全速主机/设备接口、10M 以太网 PHY、低功耗蓝牙 BLE5.3																								
通信能力	支持 USB2.0 全速设备和主机功能，无以太网和蓝牙功能	支持 USB2.0 全速设备和主机功能，支持 10M 以太网 PHY 和低功耗蓝牙 BLE5.3																								
功耗控制	工作和睡眠功耗同比大幅下降，支持多种低功耗模式	支持多种低功耗模式，具体功耗表现未明确提及																								
封装形式	LQFP、QFN、TSSOP 和 QSOP	LQFP 和 QFN																								
适用场景	工业级低功耗通用微控制器，适用于中小容量通用型应用	工业级通用微控制器，适用于大容量通用型和无线型应用																								
3. 后期拟完成的研究工作及进度安排（要有可行性） 搭建小型无人机国产化平台																										
4. 存在的困难与问题 进展较为顺利，暂时没有太大问题																										

5. 如期完成全部论文工作的可能性

能够如期完成论文工作

6. 指导导师意见

加强对飞控平台的硬件理解，
制作出符合毕业要求的无人机。

导师签名：

韩高

25年3月25日

7. 中期报告检查组意见

(中期考核结论分为两种：1. 通过，按专家意见修改后继续学位论文撰写工作；
2. 不通过，重新考核，正式答辩前达不到通过标准的，答辩延期进行。评语重点指出中期报告存在的问题并提出具体修改意见和建议。)

通过，该生目前进度良好，已完成*初步的硬件
选型调研和性能对比研究，但需要进一步将设计
落地成实物、并尽快进行毕业论文的撰写工作。

组长签名：

王明华

成员签名: 李博 林

25年 3月25日

8、承担毕业设计单位审核（盖章）

（校内毕设学生由学院审核，校外毕设学生由承担毕设企业或单位审核）

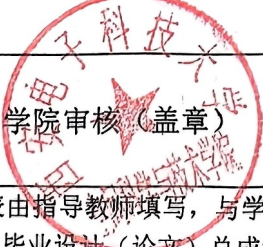
审核意见: 通过

盖章: 空间科学与技术学院

25年 3月25日

西安电子科技大学

毕业设计（论文）中期检查表

学生姓名	何成楷	学 号	21009100863	班 级	2116029
学 院	空间科学与技术学院		专 业	空间科学与技术	
导师姓名	韩高	职 称	讲师	学 院	空间科学与 技术
题目名称	RISCV 开源 IP 平台对于移植开源飞控的资源匹配性研究				
检 查 内 容			检 查 结 果		
题目是否更换及更换原因			否		
学生出勤情况			正常出勤		
进度评价 (完成总工作量的百分比)			45%		
质量评价			较好		
总体评价 (按优、良、中、及格、不及格 五档评价)			优		
存在的问题与建议			需要调查更多方面		
 学院审核（盖章）			通过		

注：此表由指导教师填写，与学生填写的中期报告配套，填写完成交学院办公室，中期检查成绩将作为毕业设计（论文）总成绩的一部分；此表装订入毕业设计（论文）中。

西安电子科技大学

毕业设计（论文）成绩登记表

学 院	空间科学与技术学院		专 业	空间科学与技术	
姓 名	何成楷	学 号	21009100863	成 绩	
题目名称	RISC-V 开源 IP 平台对于移植开源飞控的资源匹配性研究				
指导教师	韩高		职 称	讲师	
指导教师评语及对成绩的评定意见	（指导教师可从以下几方面对毕业设计（论文）进行评审：1.独立查阅文献及调查论证的能力；2.方案设计与实验技能；3.分析与解决问题的能力；4.工作量、工作态度；5.论文质量；6.创新能力。） 论文的选题具有一定工程应用价值，通过文献查阅，对本领域相关工作具有较好的理解。 研究内容明确，工作量较大，体现较好分析与解决问题能力。 论文写作认真，具有较好的文字组织能力。 建议成绩：☒优 ☐良 ☐中 ☐及格 ☐不及格 签名 <u>韩高</u> 2021 年 5 月 15 日				

评阅人评语 及成绩评定 意见	(评阅人主要从以下几方面对毕业设计(论文)进行评价:(1)综合运用理论知识和基本技能的能力;(2)分析和解决实际问题的能力;(3)论文写作能力;(4)所解决的问题与论文质量评价。) 本研究从多个维度对国产RISC-V的相关参数性能进行分析,并结合小四轴无人机的产品特性,构建了基于RISC-V架构的无人机飞控硬件平台,并最终通过实物形式展现。该同学前期调研充分,对无人机的硬件构成以及飞控系统核心模块理解深入,所开发的硬件平台在架构选择上具有创新性,为无人机飞控系统提供了新的技术方案,研究过程清晰,论证严谨,体现了作者扎实的专业基础和良好的工程实践能力。 建议成绩: ☐优 ☒良 ☐中 ☐及格 ☐不及格 签名 <u>王明</u> 2023年5月23日
答辩小组 意见	答辩通过 建议成绩: ☐优 ☒良 ☐中 ☐及格 ☐不及格 签名 <u>李博</u> 2023年5月26日
学院答辩委 员会意见	通过 答辩委员 会主任签名 <u>周峰</u> (学院盖章) 2023年6月1日

注: 1.学院、专业名均写全称;成绩登记表双面打印。

2.各学院可根据本学院专业认证要求及细则对表格进行修改,修改后的表格需向本科招生院备案。

班 级 2116029

学 号 21009100863

西安电子科技大学

本科毕业设计论文



题 目 RISCV 开源 IP 平台对于移植
开源飞控的资源匹配性研究

学 院 空间科学与技术学院

专 业 空间科学与技术

学 生 姓 名 何成楷

导 师 姓 名 韩高

摘要

近年来，无人机技术在多个领域得到了广泛应用，然而随着应用场景的不断拓展和技术要求的日益提高，现有的无人机系统面临着性能提升、功能拓展和成本优化等挑战。传统 STM32 架构芯片在处理复杂飞行任务时逐渐暴露出性能瓶颈，难以满足无人机对高性能、低功耗和功能拓展的要求。RISC-V 作为一种开源的指令集架构，具有简洁、灵活、可扩展性强等特点，展现出在无人机领域的巨大应用潜力。本文聚焦于 RISC-V 开源 IP 平台在小型四轴无人机飞控系统资源匹配性研究，致力于将国产 RISC-V 架构芯片移植到现有的 STM32 芯片为底座的小型四轴无人机平台上，以实现性能优化和功能拓展。

本文首先分析了国内外 RISC-V 架构及小型飞控平台的研究现状，明确了研究背景与意义。接着，从主频、计算性能、外设资源、开发工具链、周边传感器配套、电机驱动控制等维度，对比了国产 RISC-V 平台与 STM32 架构的优劣势，并以 CoreMark 基准测试量化评估 MCU 性能。基于评估结果，设计并实现了基于 RISC-V 架构的无人机飞控硬件平台，完成了原理图设计、PCB 布局、硬件打板与系统验证。实验结果表明，所构建的 RISC-V 架构飞控平台能够实现稳定飞行和精确姿态调整，具备良好的抗干扰能力，且在性能和成本方面具有显著优势。

本文的研究为 RISC-V 架构在无人机领域的应用提供了技术路径，为物联网设备的架构选型提供了新的思路，对未来无人机飞控系统的硬件设计和性能优化具有重要的参考价值。通过高性能芯片集成、传感器融合技术优化、电源管理系统改进以及飞行控制算法的进一步优化，有望进一步提升 RISC-V 架构无人机的性能和功能，推动无人机技术的创新和发展。

关键词：RISC-V 架构，小型四轴无人机，飞控系统，资源匹配性，硬件设计

Abstract

In recent years, drone technology has been widely applied across various fields. However, with the continuous expansion of application scenarios and increasing technical requirements, existing drone systems are facing challenges in performance enhancement, functional expansion, and cost optimization. Traditional STM32 architecture chips are gradually showing performance bottlenecks when handling complex flight tasks, and they are unable to meet the requirements of drones for high performance, low power consumption, and functional expansion. RISC-V, as an open-source instruction set architecture, features simplicity, flexibility, and strong scalability, and has demonstrated significant potential for application in the drone field. This thesis focuses on the resource matching of the RISC-V open-source IP platform in the flight control system of small quadrotor drones, and is committed to transplanting domestic RISC-V architecture chips to the small quadrotor drone platform based on the STM32 chip in order to achieve performance optimization and functional expansion.

This thesis first analyzes the research status of RISC-V architecture and small quadrotor flight control platforms both domestically and internationally, clarifying the background and significance of the study. Subsequently, it compares the advantages and disadvantages of domestic RISC-V platforms and STM32 architecture from dimensions such as clock frequency, computing performance, peripheral resources, development toolchains, supporting sensors, and motor drive control, and quantitatively evaluates MCU performance using the CoreMark benchmark test. Based on the evaluation results, a drone flight control hardware platform based on the RISC-V architecture was designed and implemented, completing the schematic design, PCB layout, hardware prototyping, and system validation. Experimental results indicate that the RISC-V-based flight control platform can achieve stable flight and precise attitude adjustment, with good anti-interference capability and significant advantages in performance and cost.

This study provides a technical pathway for the application of RISC-V architecture in the drone field, offers new ideas for the architecture selection of IoT devices, and has important reference value for the hardware design and performance optimization of

future drone flight control systems. Through the integration of high-performance chips, optimization of sensor fusion technology, improvement of power management systems, and further optimization of flight control algorithms, the performance and functionality of RISC-V-based drones are expected to be further enhanced, promoting innovation and development in drone technology.

Keywords: RISC-V architecturesmall quadrotor drone, flight control system, resource matching, hardware design

目 录

摘要	I
Abstract	II
第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 国内研究现状	2
1.2.2 国外研究现状	2
1.3 研究内容和主要工作	3
1.4 本文的结构安排	4
第二章 小型四轴无人机硬件系统架构原理简述	5
2.1 无人机构成的基础框架	5
2.1.1 四旋翼无人机的运动控制方法	5
2.1.2 无人机总体硬件框架概览	6
2.2 无人机的各模块介绍	7
2.2.1 MCU 部分详解	7
2.2.2 姿态传感器模块详解	9
2.2.3 通讯模块详解	12
2.2.4 电机模块详解	14
2.2.5 串行通信协议-SPI	15
2.2.6 通用输入输出接口 GPIO	17
第三章 MCU 多维度评估与小四轴硬件资源匹配性研究	21
3.1 MCU 的相关参数性能与量化比较	21
3.1.1 小四轴无人机 MCU 选型评价维度	21
3.1.2 芯片的综合参数对比和目标飞控平台的所需资源量化	24
3.1.3 芯片的开发工具链，价格，普及程度比较	26
3.2 CoreMark 基准测试性能评估	26
3.1.1 Coremark 的引入	26
3.2.2 coremark 代码流程拆解	27

3.2.3 coremark 任务操作	31
3.2.4 coremark 结果展示	32
3.3 完成 RISCV 架构平台的硬件原理图设计	33
3.3.1 CH32V203C8T6 的原理图设计	33
3.3.2 MPU6050 的原理图设计	35
3.3.3 电机驱动的原理图设计	36
3.3.4 供电模块的原理图设计	37
第四章 小四轴无人机飞控板的实例化与系统架构	39
4.1 飞控板设计	39
4.2 主控程序的系统架构介绍	40
4.3 飞行结果展示	42
第五章 未来研究展望	43
5.1 硬件优化与性能提升	43
5.2 软件算法改进与功能拓展	44
致谢	45
参考文献	46

第一章 绪论

1.1 研究背景与意义

近年来，无人机技术取得了飞速发展，并在多个领域得到了广泛应用。小四轴无人机因其结构简单、灵活性高、成本较低等优点，被广泛应用于航拍、物流配送、农业植保、环境监测、电力巡检等领域。然而，随着应用场景的不断拓展和技术要求的日益提高，现有的无人机系统面临着性能提升、功能拓展和成本优化等挑战。

目前，许多小四轴无人机采用 STM32 架构的芯片作为核心控制器。STM32 架构的芯片具有良好的稳定性和较低的功耗，在无人机领域得到了广泛应用。然而，随着无人机功能的不断复杂化，如高清图像传输、复杂飞行控制算法、多传感器融合等^[1]，STM32 架构的芯片在处理能力、内存容量和指令集扩展性等方面逐渐暴露出一些局限性。例如，其指令集相对固定，难以满足一些特殊功能的定制化需求；在处理高分辨率图像数据或复杂算法时，可能会出现性能瓶颈，影响无人机的飞行性能和功能实现。

RISC-V 是一种开源的指令集架构，具有简洁、灵活、可扩展性强等特点。近年来，RISC-V 架构得到了广泛关注和快速发展，其在嵌入式系统、物联网、人工智能等领域展现出了巨大的应用潜力。与传统的 STM32 架构相比，RISC-V 架构的芯片可以根据不同的应用场景和需求进行定制化设计，能够更好地满足无人机系统对高性能、低功耗和功能拓展的要求^[2]。此外，RISC-V 架构的开源特性也降低了芯片设计和开发的成本，有利于推动无人机技术的创新和发展。本文的工作致力于将国产 riscv 架构芯片移植到现有的 STM32 芯片为底座的小型四轴无人机平台上，选定若干家国产化 RISC-V 平台对应的商业化芯片作为研究对象。从主频，计算性能，外设资源，工具链，周边传感器配套，电机驱动控制等维度，对比应用现有 STM32F4/H7 的优劣势。因此研究和评估国产化 RISC-V 平台在微型行业应用无人机的适配性、并实例化一个 RISC-V 架构的无人机平台，是本文将要解决的问题。

1.2 国内外研究现状

1.2.1 国内研究现状

国内关于 RISC-V 架构的研究中,中科院软件研究所的刘畅,武延军等对近年来 RISC-V 指令集架构相关的研究成果进行了综述,围绕 RISC-V 系统设计问题,从指令集、功能实现、性能提升、安全策略这 4 个方面,论述了 RISC-V 处理器基本的研究思路,并展望了 RISC-V 架构后续研究的可能切入点和未来发展方向^[4]。在工业界 RISC-V 架构的落地成果更多,蜂鸟 E203 是中国第一个开源 RISC-V 处理器内核,为国内 RISC-V 处理器的研究者和院校提供了极好的实践案例,推动了国内 RISC-V 生态的发展。后来,蜂鸟 E203 的开发者的胡振波创立了芯来科技,推出了 N100、N200、N300、N600、N900 等多个系列的 RISC-V 处理器,为多家企业提供了定制 RISC-V 处理器 IP 核^[5]。近年来,国内多家科技企业在 RISC-V 架构处理器研发领域取得重要突破。芯来科技与兆易创新达成战略合作,联合开发了代号为"Bumblebee"的 RISC-V 处理器核心,该技术成果已成功应用于兆易创新推出的 GD32VF103 系列商用通用微控制器产品线。与此同时,阿里巴巴旗下半导体企业平头哥也在积极推进 RISC-V 技术的产业化应用,其自主研发的玄铁处理器系列包含 64 位 RISC-V 核心 C910 及符合 RISC-V 标准的 E902 等产品,这些处理器在人工智能物联网(AIoT)应用场景中展现出显著的技术优势,并逐步构建起完善的产业生态体系^[6]。

国内在小型飞控平台的项目同样非常丰富。一些企业和科研机构致力于提升飞控系统的性能与可靠性。例如,基于国产开源嵌入式实时操作系统 rt-thread 开发的 openfly 飞控,核心算法源自国外开源飞控 ardupilot,支持 mavlinkv2.0 协议,可利用 missionplanner 进行遥测接收、任务规划等操作^[7]。该飞控采用 STM32F427 主芯片及 STM32F405 协处理器(惯导专用),集成导航、舵机控制、发动机控制和电源管理功能,为用户提供飞行控制模型的软硬件开发平台,其自主研发的导航算法通过扩展卡尔曼滤波器融合多种传感器信息,能在不同条件下提供稳定导航结果^[8]。

1.2.2 国外研究现状

在国际 RISC-V 生态中,美国学术界和产业界做出了重要贡献。加州大学伯克利分校的 RISC-V 研究团队开发了名为 Rocket Chip 的开源处理器生成框架,该项目被视为 RISC-V 指令集架构的参考实现之一。该框架采用 BSD 开源许可协议,

使用 Chisel 硬件描述语言开发,能够自动生成包括 Rocket 核心和 BOOM 核心在内的多种处理器设计。此外,该团队还基于 Rocket Chip 开发了 freedom 开源 SoC 平台,相关代码资源均在 GitHub 平台公开共享。这些开源项目为 RISC-V 生态的发展提供了重要的技术参考和实现基础^[9]。CV32E40P 处理器核心最初源自 OpenRISC 架构的 ORION 处理器设计,后经 PULP 研发团队将其重构为符合 RISC-V 标准的处理器内核。该处理器采用顺序执行架构,支持 RV32IMC 指令集,具有四级流水线结构。在技术演进过程中,研发团队对原有架构进行了深度改造,使其完全适配 RISC-V 指令集规范^[10]。

国外高校和科研机构在小型四轴飞行器控制系统领域积累深厚,技术成熟度较高,且多以开源飞控项目为主。2007 年,DIY 无人机社区推出 APM(ArduPilot Mega)飞控产品,作为成熟的开源硬件项目,凭借良好的可定制性在全球航模爱好者中广泛传播^[11]。PX4 是遵循 BSD 协议的软硬件开源项目,致力于为学术、爱好和工业团体打造低成本、高性能的高端自驾仪,在多旋翼飞行器、固定翼飞机等多种飞行器类型中得到应用^[12]。瑞典的 Crazyflie 是著名的微型四轴飞行器开源项目,由三位嵌入式工程师创建,旨在以最少的零件构建适用于室内环境的小型飞行机器人。该项目硬件设计精巧,软件架构基于 FreeRTOS 实时操作系统,将飞行控制任务分解为多个优先级不同的软件任务,便于开发与维护^[13]。

1.3 研究内容和主要工作

RISC-V 平台的开发研究工作在当今全国产化替代的浪潮下有着重要意义,但是由于新兴平台的开发生态尚未构建完善,关于 RISC-V 架构芯片的飞控系统目前研究工作并不多。本文的研究工作聚焦于构建基于 RISC-V 架构的无人机飞控硬件平台,首先对无人机硬件平台的核心组件展开研究,其次通过芯片性能比较、计算能力评估与开发生态分析完成 RISC-V 平台设计,最终通过硬件实现与飞行实验验证系统可行性。具体研究工作包含以下几个层面:

本文首先针对无人机飞控平台进行硬件架构解构,建立完整的硬件选型评价体系。通过剖析典型飞控系统的五类核心模块(MCU、姿态传感器、通讯模块、驱动电机、供电模块),明确各子系统的功能边界与性能指标要求,重点阐释 MCU 作为计算核心在指令吞吐量、外设接口带宽、实时响应能力等方面的关键参数特征,性能比较方面,引入 CoreMark 基准测试,通过多维度任务(包含矩阵操作、状态机控制、链表遍历等核心负载)量化评估计算单元的性能。随后开展完整的

系统实现与验证实验，在硬件层面绘制了小四轴无人机主控板的原理图并完成了布局和布线，后续进入打板环节。在软件层面分析了项目工程移植的原理和工作流，完成了系统项目移植的解析工作。

本研究通过前期分析论证，后期落地实验验证了 RISC-V 在无人机领域的应用潜力，所提出的量化评估模型与硬件设计方法为物联网设备的架构选型提供了新的技术路径。

1.4 本文的结构安排

本毕业论文的后续主要分为四章，在第二章中，本文首先解读并分析了无人机的基础硬件组成框架，并对于飞控系统的五类核心模块（MCU、姿态传感器、通讯模块、驱动电机、供电模块）进行介绍。在第三章中，本文重点展开对于 MCU 的相关参数性能与量化比较，包括 RISC-V 架构芯片内核与 ARM 架构内核的比较，后续具体细化到多款国产 RISC-V 架构芯片的参数对比和芯片的开发工具链，价格，普及程度比较。接着通过 CoreMark 基准测试，多角度测评 MCU 计算单元的性能。最后综合多方面评价信息，设计并形成了基于 RISC-V 架构的无人机飞控硬件平台，完成了全流程的 pcb 设计、打板、系统验证、功能测试环节，并观察总结无人机的综合性能效果。在第五章中，本文对本研究的优化方向和未来的研究方向提出设想。

第二章 小型四轴无人机硬件系统架构原理简述

2.1 无人机构成的基础框架

2.1.1 四旋翼无人机的运动控制方法

四轴无人机采用四个独立调速的外转子无刷电机驱动螺旋桨作为动力来源。这类飞行器的典型结构采用十字形布局，由两根碳纤维横梁交叉构成对称框架，四个固定桨距的螺旋桨分别安装在横梁末端。值得注意的是，相邻螺旋桨采用反向旋转设计以平衡扭矩。相比传统直升机，四旋翼飞行器具有显著的低转速特性，这使得其能够在狭小空间内稳定飞行，特别适用于室内环境或近地作业场景^[14]。

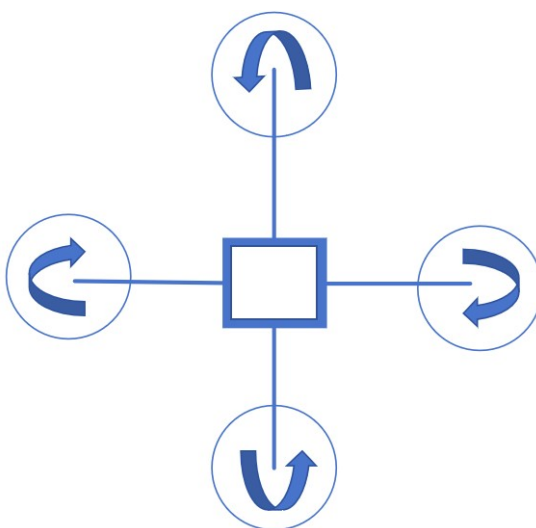


图 2.1 常见小型四旋翼无人机的结构示意图

四旋翼飞行器系共有 4 个输入，分别为一个上升力和三方向的转矩，但飞行器在空间中却有 6 个自由度的输出，可以进行三个坐标轴方向的平动运动和围绕三个坐标轴方向的转动运动。与常规纵列式直升机需要尾桨抵消反扭矩不同，四旋翼飞行器通过独特的四电机布局实现了动力平衡。该飞行系统采用四个独立控制的电机作为动力装置，由微控制器产生 PWM 信号精确调节各电机转速。通过改变四个螺旋桨的旋转速度来调整升力分布，从而实现对飞行姿态和运动轨迹的控制。这种创新设计不仅简化了飞行器的机械结构，还优化了其动力学特性^[15]。四旋翼的前桨 1 和后桨 3 逆时针旋转，左右 2、4 两桨顺时针旋转，这种反向对称结构代替了传统直升机尾旋翼，在飞行过程中，如图 2-2 所示，改变 4 个旋翼螺旋桨的转速，可使四旋翼产生各种飞行姿态，也可使四旋翼飞行器向预定方向运动，完成任务^[16]。

无人机通过转速差产生不同方向的力和力矩，实现六种基本运动：

- 1) 垂直运动（油门控制）：四个螺旋桨转速同步增加或减少，产生向上或向下的推力，实现上升或下降。
- 2) 前后运动（俯仰控制）：增大后螺旋桨转速、减小前螺旋桨转速，机身前倾，产生向前的水平分力；反之则后倾后退。
- 3) 左右运动（横滚控制）：增大右螺旋桨转速、减小左螺旋桨转速，机身左倾，产生向右的水平分力；反之则右倾左移。
- 4) 偏航运动（旋转控制）：顺时针旋转的螺旋桨组（1、3 号）加速，逆时针组（2、4 号）减速，产生逆时针偏航力矩；反之则顺时针旋转。
- 5) 悬停控制：四个螺旋桨推力与重力平衡，且水平方向合力为零，保持位置不变。
- 6) 复合运动：组合上述控制量（如同时调整油门和俯仰），实现斜向飞行或复杂轨迹。

2.1.2 无人机总体硬件框架概览

小型四轴无人机硬件系统以微控制器单元（MCU）为核心控制节点，通过模块化设计实现感知、决策与执行的闭环控制。系统架构可分为以下功能模块。首先是控制决策模块：MCU 作为中央处理器，运行实时控制算法（如 PID 或状态反馈），接收来自姿态传感器（包含陀螺仪、加速度计及磁力计）的原始数据，通过卡尔曼滤波融合多轴惯性信息，解算无人机的实时姿态角（俯仰、横滚、偏航）与角速度。MCU 同时解析通信芯片接收的遥控指令或自主导航目标，通过 PID 控制算法生成电机驱动信号以调整飞行状态。然后是感知与通信模块：姿态传感器通过 I²C、SPI 或者 GPIO 总线与 MCU 交互，构成姿态稳定控制的基础；通信芯片支持无线协议（如 2.4GHz 射频或 Wi-Fi），实现遥控指令传输及飞行状态回传，并通过协议栈解析确保低延迟人机交互^[17]。最后是动力执行模块：无刷电机与螺旋桨作为动力输出单元，其转速由 MCU 输出的 PWM 信号经电子调速器（电调）调制后驱动，通过四电机的差速控制实现无人机的升力调节与姿态调整。系统还包括供电管理模块：锂聚合物电池经 DC-DC 降压电路分配为高压（电机驱动）与低压（MCU、传感器供电）双电源轨，并集成电压监测与过流保护功能，确保系统在动态负载下的稳定运行。

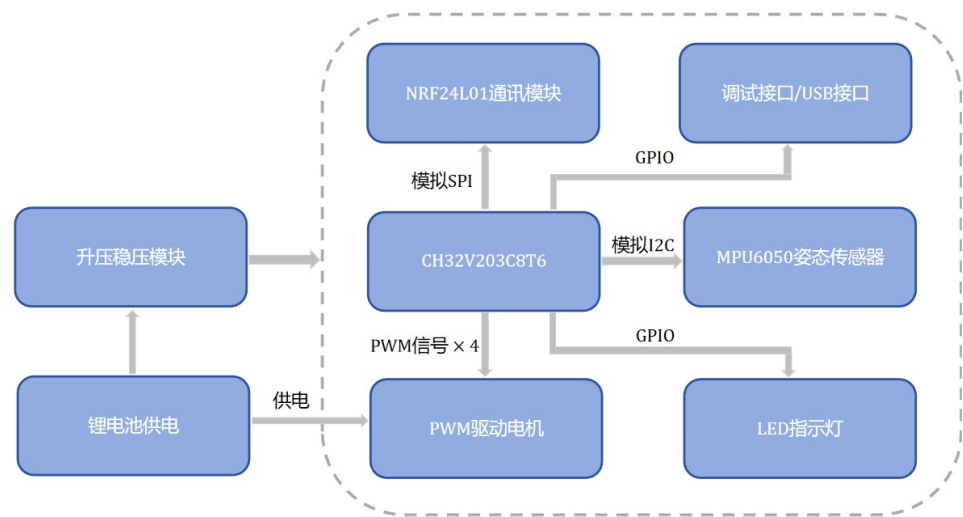


图 2.2 硬件框架图

各模块通过分层总线架构互联：MCU 作为主控节点，通过模拟 I2C 实现传感器数据的实时读取与控制指令的快速下发；通信芯片以模拟 SPI 与 MCU 交互，采用帧校验机制提升数据传输可靠性。LED 指示灯通过 GPIO 接口与 MCU 相连，LED 指示灯用于直观显示系统的工作状态，比如电源是否接通、通讯是否正常、是否有错误发生等^[18]。系统运行时，MCU 以固定频率（取决于 MCU 的主频）执行“数据采集-滤波解算-控制量计算-电机驱动”的闭环流程，其低延迟特性保障了飞行稳定性。此外，模块间通过电源域隔离与信号屏蔽设计，抑制电机高频噪声对敏感电路的干扰，进一步优化系统鲁棒性。

2.2 无人机的各模块介绍

2.2.1 MCU 部分详解

MCU 部分本文选用小型四轴飞控板常用的典型单片机来介绍：STM32F103C8T6。STM32F103C8T6 是一款由意法半导体公司（ST）推出的基于 Cortex-M3 内核的 32 位微控制器，Cortex-M3 是来自于 ARM 的 IP 核，加上意法半导体自由特色的外设和总线优化，使得 STM32 兼顾 ARM32 位处理器的高性能运算和同时还具有高度可扩展性，故其对于嵌入式市场的占有率居高不下。硬件采用 LQFP48 封装，属于 ST 公司微控制器中的 STM32 系列。该芯片的具体参数如下：

表 2.1 STM32F103C8T6 具体参数

内核	Cortex-M3
Flash	64K*8bit
SRAM	20K*8bit
ADC	2 个 12bit 定时器/计数器，分别为 TIM1，TIM2，TIM4，TIM1 带死区插入，常用于产生 PWM 控制电机
	2 个看门狗定时器（独立看门狗 IWDG、窗口看门狗 WWDG
	1 个 24bit 向下计数的滴答定时器 systick
工作电压、温度	2V~3.6V、-40℃~85℃
多达 9 个通信串口	多达 2 个 I2C 接口（支持 SMBus/PMBus）； 多达 3 个 USART 接口(支持 ISO 7816 接口、LIN、IrDA 接口和调制解调控制）； 多达 2 个 SPI 接口（18Mps）； CAN 接口； USB2.0 全速接口
系统时钟	内部 8Mhz 时钟最高可以倍频到 64Mhz，外部 8Mhz 时钟 HSE 最高可倍频到 72Mhz
DMA	7 通道 DMA 控制器

ARM 公司在 ARM11 以后改用 Cortex 命名，并分成 A、R 和 M 三类，M 系列有 M0、M0+、M3、M4、M7。Cortex-M3 是 ARM 公司推出的基于 ARMv7 架构的 MCU 内核，ST 公司在此内核的基础上完成了 USART、DMA、GPIO 等外围电路的设计。在 M 系列不同内核开发下的 STM32，大致可分成低功耗系列、主流系列、高性能系列 3 个部分，其具体信息如下图所示：

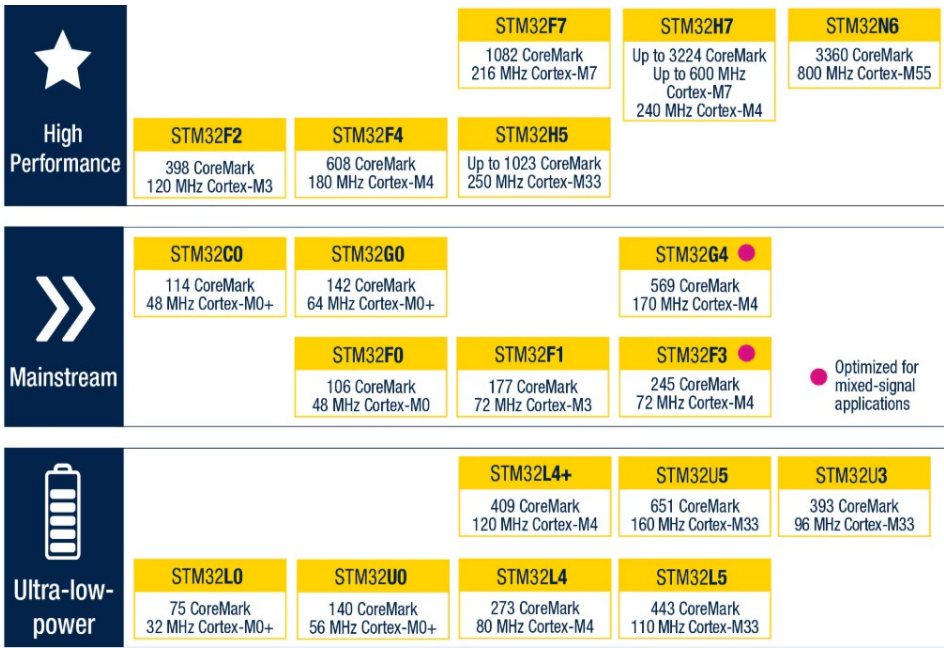


图 2.3 STM32 系列选型分类

STM32F103C8T6 整个系统结构可以分为由 ARM 公司设计的 Cortex-M3 内核和 ST 公司在此基础上优化的总线矩阵、DMA（Direct Memory Access，直接内存读取）、AHB、APB1 以及 APB2 上挂载的外设等两个部分^[19]。在文章的第三部分会展开讨论 STM32F103C8T6 和与之相对应的主流 RISC-V 芯片（比如 CH32V203C8T6）的内部系统结构。具体来说 STM32F103C8T6 作为市面上最流行的微控制器，包括大量的优点：价格低廉、资料多，容易上手，而且 RAM 大，时钟频率高，非常适合用于飞行控制的大量运算，能够高效完成对飞控数据的采集和处理。后期考虑到计算能力的扩展，可以选用基于 M4 内核的 STM32F4 系列产品，相较于之前的 STM32F1 系列，STM32F4 系列最大的优势就是新增了硬件 FPU（Float Point Unit，浮点计算单元）以及 DSP（Digital Signal Processing，数字信号处理）指令，同时它的主频也提升了很多，达到了 168Mhz（可获得 210DMIPS 的处理能力），这使得在需求高性能边缘计算能力（例如接入视觉模块，图像处理）的场景可以采用新的方案^[20]。

2.2.2 姿态传感器模块详解

姿态传感器所测量的数据最多会包括加速度、姿态角、磁场强度、气压（用于计算飞行高度）。3 轴加速度计的原理基于牛顿第二定律 $F=ma$ 。内部通常有一个可在三个正交方向（X、Y、Z 轴）移动的质量块，当传感器所在物体加速运动时，质量块由于惯性会产生相对位移，通过检测质量块在电容极板间的位移变化

（电容式加速度计常用原理，利用电容值与极板间距的关系），或利用压电材料在受力时产生的电荷（压电式加速度计原理）等方式，将加速度引起的物理变化转换为电信号，经放大、滤波等处理后输出与加速度成比例的电压或数字信号，从而测量物体在三个轴向的加速度。3 轴陀螺仪传感器的测量原理基于科里奥利力效应。内部有一个高速旋转的陀螺转子（或等效的微机电结构），当传感器所在物体绕轴转动时，由于科里奥利力的作用，陀螺转子会在垂直于旋转轴和运动方向的平面上产生微小的力或位移，通过检测这种变化（如通过检测电容变化、检测振动结构的振动频率变化等方式），并将其转换为电信号，经过处理后得到与角速度成比例的输出，从而测量物体绕 X、Y、Z 轴的旋转角速度。3 轴磁场传感器的原理是利用磁阻效应或霍尔效应。磁阻效应型传感器中，某些材料的电阻会随外部磁场强度和方向变化而改变，通过检测电阻变化来测量磁场强度和方向；霍尔效应型传感器则是当电流通过半导体薄片，在垂直于电流和磁场方向会产生电压差（霍尔电压），通过测量霍尔电压来确定磁场强度和方向^[21]。传感器测量空间中 X、Y、Z 三个轴向的磁场强度分量，结合地球磁场特性，可确定物体的方位。

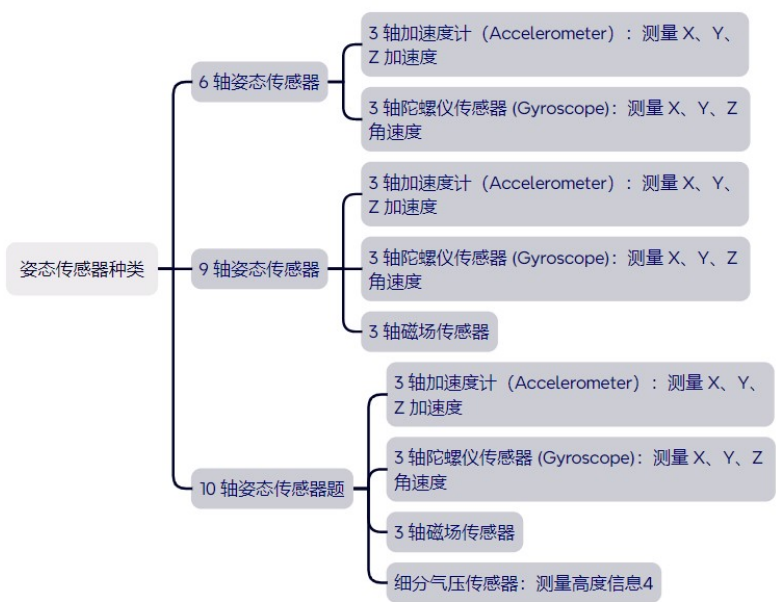


图 2.4 常见姿态传感器的种类

例如市面上典型的姿态传感器的模块为 MPU6050。MPU6050 为全球首例整合 6 轴运动姿态检测的数字传感器，消除了多传感器组合的轴间差问题，并减少了传感器的体积，降低了系统的功耗。MPU6050 内部集成 3 轴陀螺仪和 3 轴加速度计，

输出信号都是 16 位的数字量：可以通过集成电路总线（IIC）接口和单片机进行数据交互，传输速率可达 400krad/s，具有良好的动态响应特性^[22]，MPU6050 的内部结构如图所示：

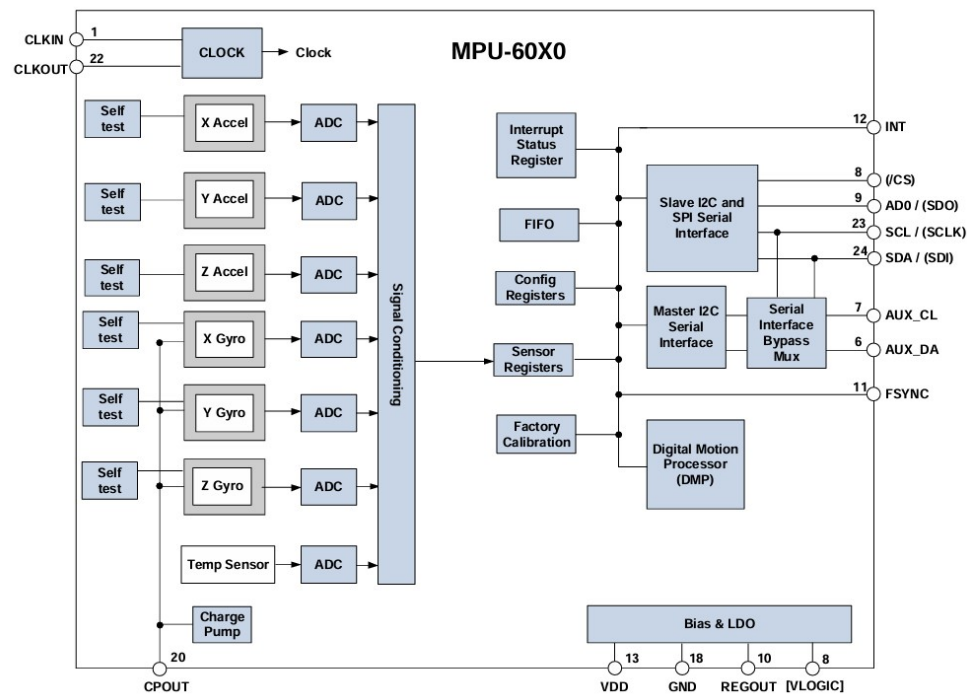


图 2.5 MPU6050 的内部结构示意图

在使用 MPU6050 进行数据传输的时候，使用 I2C 协议，如果要写 MPU-6050 寄存器，主设备除了发出开始标志（S）和地址位，还要加一个 R/W 位，0 为写，1 为读。在第 9 个时钟周期（高电平时），MPU-6050 产生应答信号。然后主设备开始传送寄存器地址（RA），接到应答后，开始传送寄存器数据，然后仍然要有应答信号，依次类推。时序如下表所示：

表 2.2 单字节写入时序

Master	S	AD+W		RA		DATA		P
Slave			ACK		ACK		ACK	

表 2.3 多字节写入时序

Master	S	AD+W		RA		DATA		DATA		P
Slave			ACK		ACK		ACK		ACK	

如果要读取 MPU6050 寄存器的值，首先由主设备产生开始信号（S），然后发送从设备地址位和一个写数据位，然后发送寄存器地址，才能开始读寄存器。

紧接着，收到应答信号后，主设备再发一个开始信号，然后发送从设备地址位和一个读数据位。然后，作为从设备的 MPU-60X0 产生应答信号并开始发送寄存器数据。通信以主设备产生的拒绝应答信号（NACK）和结束标志（P）结束。拒绝应答信号（NACK）产生定义为 SDA 数据在第 9 个时钟周期一直为高。

表 2.4 单字节读出时序

Master	S	AD+W		RA		S	AD+R			NACK	P
Slave			ACK		ACK			ACK	DATA		

表 2.5 多字节读出时序

Master	S	AD+W		RA		S	AD+R			ACK		NACK	P
Slave			ACK		ACK			ACK	DATA		DATA		

2.2.3 通讯模块详解

这里介绍常用的无线收发芯片 NRF24L01，NRF24L01 是一款单芯片 2.4GHz 收发器，内嵌了增强型 ShockBurst™基带协议引擎，适用于超低功耗无线应用。NRF24L01 支持 250kbps、1Mbps 和 2Mbps 的空中数据速率。高空中数据速率与两种省电模式相结合，使得 NRF24L01 非常适合超低功耗设计^[23]。同时 NRF24L01 增加的内部滤波功能提高了满足射频法规标准的裕度。内部稳压器确保了高电源抑制比（PSRR）和宽电源电压范围。下图是 NRF24L01 的内部结构图：

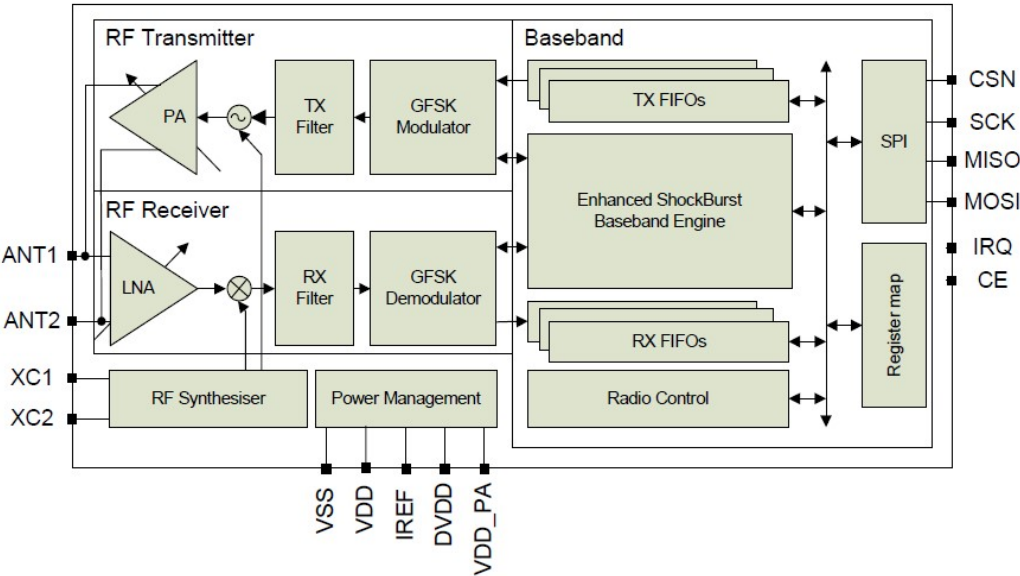


图 2.6 NRF24L01 内部结构示意图

nRF24L01+芯片的架构设计包括射频发射器、射频接收器、射频合成器、基带、电源管理、SPI 接口和其他接口等关键模块。射频发射器模块由功率放大器（PA）、

发射滤波器 (TX Filter) 和高斯频移键控调制器 (GFSK Modulator) 组成。PA 用于放大射频信号, TX Filter 用于滤除不需要的频率成分, 而 GFSK Modulator 则将基带信号调制到射频载波上, 以实现数据的无线传输。射频接收器模块包括低噪声放大器 (LNA)、接收滤波器 (RX Filter) 和高斯频移键控解调器 (GFSK Demodulator)。LNA 用于放大接收到的微弱射频信号, RX Filter 用于滤除不需要的频率成分, GFSK Demodulator 则将接收到的射频信号解调回基带信号, 以便进一步处理。射频合成器模块负责生成所需的射频载波频率, 通常通过锁相环 (PLL) 技术实现。电源管理模块则负责管理芯片的电源供应, 包括电压调节和电源管理功能, 确保芯片在不同工作模式下的电源需求。基带模块是 nRF24L01+ 芯片的核心, 包括发送 FIFO (TX FIFOs)、增强型 ShockBurst 基带引擎 (Enhanced ShockBurst Baseband Engine)、接收 FIFO (RX FIFOs) 和无线控制 (Radio Control)。TX FIFOs 用于存储待发送的数据, Enhanced ShockBurst Baseband Engine 负责处理数据包的发送和接收, 包括地址匹配、数据包处理等。RX FIFOs 用于存储接收到的数据, Radio Control 负责控制无线收发器的操作, 包括频率设置、功率控制等^[24]。

SPI 接口模块包括芯片选择信号 (CSN)、串行时钟信号 (SCK)、主设备输入从设备输出信号 (MISO)、主设备输出从设备输入信号 (MOSI)、中断请求信号 (IRQ) 和芯片使能信号 (CE)。这些信号用于实现芯片与外部微控制器之间的通信。此外, nRF24L01+ 芯片还提供了天线接口 (ANT1/ANT2) 和晶振接口 (XC1/XC2), 用于连接外部天线和晶振, 以支持单天线或双天线配置, 并提供时钟信号。



图 2.7 NRF24L01 模块实物图

NRF2401 有工作模式有五种: 收发模式、配置模式、空闲模式、关机模式。工作模式由 PWR_UP register、PRIM_RX register 和 CE 决定, NRF2401 的所有配置工作都是通过 SPI 完成, 共有 30 字节的配置字。下图是常用的 NRF24L01 模块,

在开发小四轴无人机的时候，通常会采用集成好的 NRF24L01 模块（包括射频天线和相应的功率放大部分）来加速主控的开发。

2.2.4 电机模块详解

电机按电流输入方式分为有刷电机(如图 2.2.3.1 所示)和无刷电机(如图 2.2.3.2 所示)两类，二者的区别在于是否通过电刷传导电流。有刷电机通过电刷与换向器接触，将外部电源电流传导至转子绕组，实现电机转动。无刷电机无需电刷和换向器，通过电子换向电路（如驱动芯片）控制定子绕组电流方向，驱动转子转动。有刷电机主要有空心杯和碳刷型的直流电机，其特点是可以做到体积小型化，价格相对较低^[25]。无刷电机的应用一般为稍大型的飞行器，载重大，可以有更广泛的用途。无刷电机的型号如 2218 KV2000，22 代表电机的外转子直径 22mm。18 代表转子的高度为 18mm，KV2000 代表电压每增加 1V 则电机的实际转速增加 2000r/min。

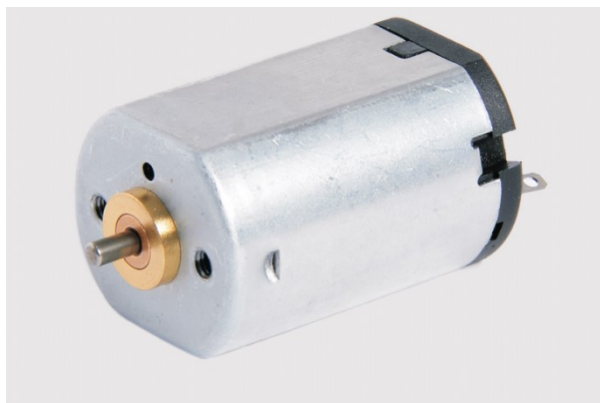


图 2.8 有刷电机



图 2.9 无刷电机

下图整理了电机的多维度分类：

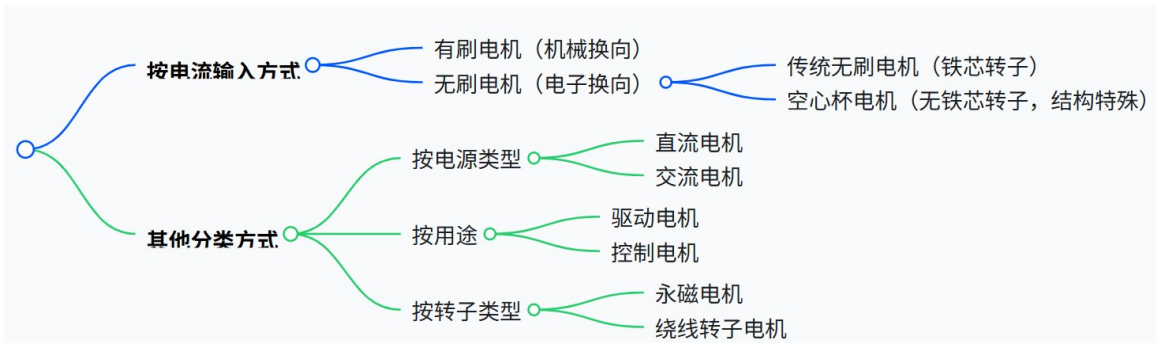


图 2.10 电机的分类

小四轴无人机（四旋翼飞行器）通常使用 无刷直流电机（BLDC）作为动力源，通过 脉冲宽度调制（PWM）技术 控制电机转速，进而实现飞行姿态和动力的调节。飞控系统负责接收遥控器信号、传感器数据（如陀螺仪、加速度计），通过算法（如 PID 控制）计算出各电机的目标转速，并输出 PWM 信号。电子调速器（ESC，Electronic Speed Controller）接收飞控输出的 PWM 信号，将其转换为电机所需的驱动电流（直流变三相交流），控制无刷电机转速。无刷电机通常为 外转子永磁无刷电机，搭配螺旋桨提供升力。PWM(Pulse Width Modulation)控制——脉冲宽度调制技术是控制电机的关键技术，通过对一系列脉冲的宽度进行调制，来等效地获得所需要波形(含形状和幅值)。其具有两个很重要的参数：频率和占空比。占空比=高电平时间/周期时间*100%。通过改变单位时间内脉冲的个数可以实现调频；通过改变占空比可以实现调压。而改变 PWM 信号的占空比，就可以改变直流电机两端的平均电压，从而实现直流电机的调速。

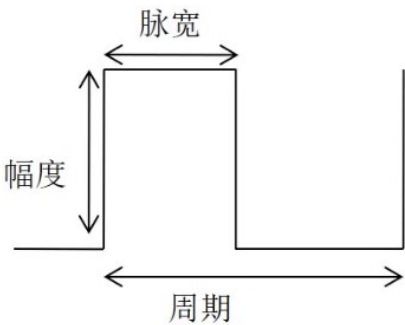


图 2.11 一个脉冲的波形

2.2.5 串行通信协议-SPI

SPI(Serial Peripheral Interface，串行外围设备接口)，是一种高速全双工的通信总线。它由摩托罗拉公司提出，被广泛地使用在 ADC、液晶显示模块等设备与微控制器间通信的场合^[26]。SPI 总线包含 4 条总线，分别为 SS、SCK、MOSI、MISO。它们的作用介绍如下：

SS(Slave Select, 片选信号线)：当有多个 SPI 设备与 MCU 相连时，每个设备的这个片选信号线是与 MCU 单独的引脚相连的，而其他的 SCK、MOSI、MISO 线则为多个设备并联到相同的 SPI 总线上，如图 2.2.4.1 所示。当 SS 信号线为低电平时，片选有效，开始 SPI 通信。

SCK(Serial Clock, 时钟信号线):由主通信设备产生，不同设备支持的时钟频率不一样，如 STM32 的 SPI 时钟频率最大为 $f_{CLK}/2$ 。

MOSI(Master Output Slave Input), 主设备输出/从设备输入引脚): 主机的数据从这条信号线输出，从机由这条信号线读入数据，即这条线上数据的方向为主机到从机。

MISO(Master Input Slave Output), 主设备输入/从设备输出引脚):主机从这条信号线读入数据，从机的数据则由这条信号线输出，即在这条线上数据的方向为从机到主机。

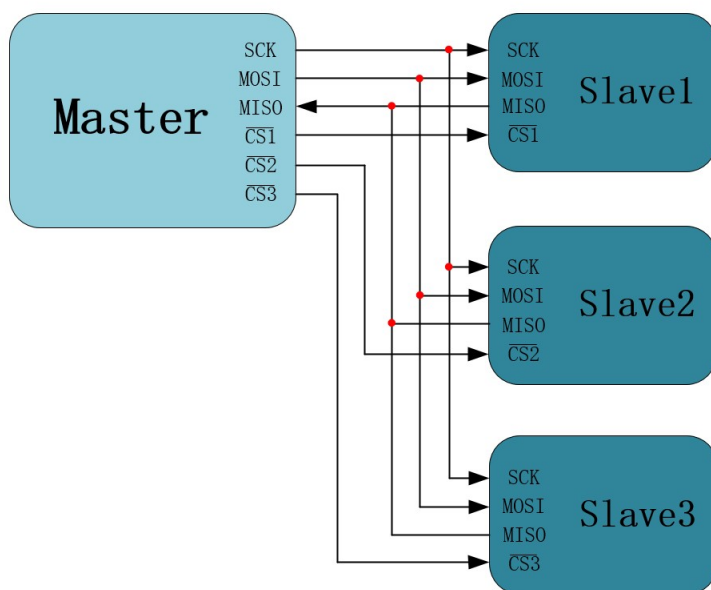


图 2.12 SPI 协议一对多连接方法

SPI 通信协议定义了四种工作模式，由时钟极性(CPOL)和时钟相位(CPHA)两个参数共同决定。时钟极性参数 CPOL 决定了 SCK 信号线在空闲状态（即 SS 信号为低电平的初始阶段）时的电平状态：当 CPOL 设为 0 时，SCK 保持低电平；设为 1 时则保持高电平。时钟相位参数 CPHA 则规定了数据传输时的采样时机：CPHA 为 0 表示数据在 SCK 时钟信号的奇数边沿（如上升沿或下降沿）被采样；而 CPHA 为 1 时，数据将在 SCK 的偶数边沿进行采样。这两种参数的组合形成了 SPI 协议的四种不同工作模式。

当我们在小四轴飞控主板上使用 STM32 系列产品时，则 STM32 不同容量产品配备的 SPI 接口数量不同，小容量产品有 1 个 SPI 接口，中容量产品有 2 个，大容量产品则有 3 个。STM32 的 SPI 接口具备一系列实用特性。在数据传输方面，单次传输位数可在 8 位或 16 位中选择，并且数据的传输顺序能够编程选择，支持 MSB（最高有效位）在前或 LSB（最低有效位）在前的方式。波特率预分频系数最大可 $f_{\text{PCLK}}/2$ ，能满足不同的通信速率需求。其时钟极性（CPOL）和相位（CPHA）都可编程设置，可灵活适配不同的通信设备。此外，SPI 接口还设有可触发中断的专用发送和接收标志，方便系统及时响应数据的收发情况。同时，它还支持使用 DMA 进行数据传输操作，这有助于减轻 CPU 的负担，提高数据传输效率^[27]。

2.2.6 通用输入输出接口 GPIO

GPIO（General Purpose Input Output，通用输入输出接口）是微控制器、单片机或其他嵌入式系统中一种非常重要的接口，它为系统提供了灵活的输入输出功能。GPIO 是一种可编程的接口，它允许用户根据需要将其配置为输入端口或输出端口。每个 GPIO 引脚都可以独立设置为输入模式或输出模式，并且可以进一步配置为不同的功能，例如普通数字输入/输出、中断输入、模拟输入等。端口的每个引脚都可以配置成以下的多种模式之一：

- （1）浮空输入：Floating Input
- （2）上拉输入：Pull-up Input
- （3）下拉输入：Pull-down Input
- （4）模拟输入：Analog Input
- （5）开漏输出：Open-drain Output
- （6）推挽输出：Push-pull Output
- （7）复用功能的输入和输出：Multiplexed Function Input and Output

每个引脚在芯片内部都有两只保护二极管，IO 口内部可分为输入和输出驱动模块。其中输入驱动有弱上下拉电阻可选，可连接到 AD 等模拟输入的外设；如果输入到数字外设，就需要经过一个 TTL 施密特触发器，再连接到 GPIO 输入寄存器或其他复用外设。而输出驱动有一对 MOS 管，可通过配置上下的 MOS 管是否使能来将 IO 口配置成开漏或推挽输出；输出驱动内部也可以配置成由 GPIO 控制输出还是由复用的其他外设控制输出。

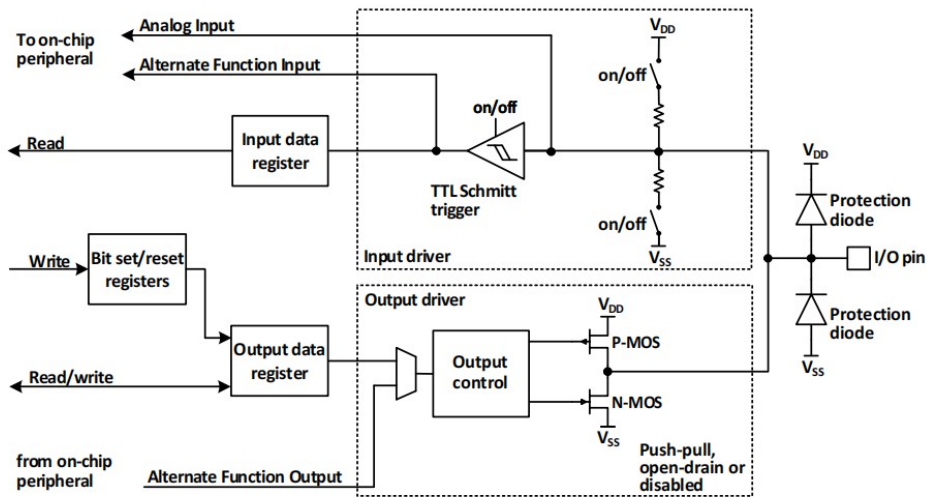


图 2.13 GPIO 接口内部构造

GPIO_Mode_TypeDef 定义了 STM32 系列微控制器中 GPIO（通用输入输出）引脚的不同工作模式，每种模式都有不同的输入输出特性^[28]。下面逐项解释这些模式的含义：

GPIO_Mode_AIN(0x00)：模拟输入。该模式用于模拟信号输入，GPIO 引脚处于模拟状态，不参与数字电路的工作。在这个模式下，引脚没有上拉或下拉电阻，适用于模拟信号采集（如 ADC 输入）。

GPIO_Mode_IN_FLOATING (0x04)：浮空输入。该模式下，GPIO 引脚设置为输入状态，但没有内部的上拉或下拉电阻，意味着该引脚处于浮空状态，可能受到外部干扰。在这种模式下，输入信号可以是任意电平，通常用于接收外部信号，但需要外部电路来确保输入的稳定性。刚复位后，GPIO 口运行在初始状态，这时大多数 IO 口都是运行在浮空输入状态，但也有 HSE 等外设相关的引脚是运行在外设复用的功能上。

GPIO_Mode_IPD (0x28)：输入下拉。该模式下，GPIO 引脚被配置为输入，并且启用了内部下拉电阻。这意味着如果没有外部信号，输入引脚会被拉到低电平，避免输入端漂浮，适用于需要保持低电平的输入状态。

GPIO_Mode_IPU (0x48)：输入上拉。该模式下，GPIO 引脚被配置为输入，并启用了内部上拉电阻。如果输入端没有外部信号，输入会被拉高，维持在高电平状态。适用于需要保持高电平的输入状态。

GPIO_Mode_Out_OD (0x14)：输出开漏。该模式下，GPIO 引脚被配置为输出模式，但为开漏输出。开漏输出不提供内部的高电平电压，当输出低电平时，GPIO

引脚会被拉低；当输出高电平时，GPIO 引脚处于高阻抗状态，外部拉高电阻会将引脚拉高。开漏输出常用于 I2C 总线或与其他外设的互联，适用于多个设备共用同一线路的情况。

GPIO_Mode_Out_PP (0x10): 输出推挽。该模式下，GPIO 引脚被配置为输出模式，并且是推挽输出。在推挽模式下，GPIO 引脚能提供高电平和低电平电压，即它可以主动输出高电平（逻辑 1）或低电平（逻辑 0）。推挽输出通常用于驱动负载，如 LED 或电机等。

GPIO_Mode_AF_OD (0x1C): 复用功能，开漏。该模式下，GPIO 引脚配置为复用功能模式，并且是开漏输出。这意味着引脚不仅可以作为一般的数字输入/输出，还可以用作特定的外设功能（如 SPI、I2C、UART 等）。在开漏模式下，信号输出为开漏形式，需要外部拉高电阻。

GPIO_Mode_AF_PP (0x18): 复用功能，推挽。该模式下，GPIO 引脚配置为复用功能模式，并且是推挽输出。同样，GPIO 引脚不仅可以作为数字输入/输出，还可以用作特定的外设功能。与开漏模式不同，推挽模式能够提供明确的高电平和低电平输出。

在嵌入式开发中，操作寄存器 CRH 和 CRL 来配置 IO 口的模式和速度是通过 GPIO 初始化函数完成。

第三章 MCU 多维度评估与小四轴硬件资源匹配性研究

3.1 MCU 的相关参数性能与量化比较

在该部分文章将开展关于 MCU 多维度评估与小四轴硬件资源匹配性研究。目前相关数据显示，2023 年中国 RISC-V 芯片出货量高达 21 亿颗，到 2025 年预计会突破 80 亿颗，复合增长率达 56%。2025 年全球 RISC-V 芯片市场规模预计将达到 240 亿美元，中国厂商有望占据 60% 的份额。目前国内主要生产 RISC-V 架构 MCU 的厂商包括兆易创新、沁恒微电子、中微半导体等，上流还有做 RISC-V 架构内核授权的阿里平头哥等。这些厂商所出品的 RISC-V 架构芯片的应用场景包括物联网、消费电子、工业控制领域等。接下来文章将讨论在遇到小四轴无人机的芯片应用场景的情况，如何综合考虑芯片的多维度评价指标，并在实际调研比较一系列国产 RISC-V 架构芯片之后（也包括作为比较基准的 ARM 架构 STM32 系列代表性芯片），确定我们的硬件选型^[29]。

3.1.1 小四轴无人机 MCU 选型评价维度

首先在性能指标方面，主频是衡量芯片处理速度的关键指标，它直接决定了芯片在单位时间内能够执行的指令数量。对于小四轴无人机的飞控系统而言，飞控算法的实时性至关重要。飞控系统需要在短时间内处理来自多个传感器（比如 IMU、气压计、GPS 等）的数据，并进行复杂的计算（如 PID 控制、路径规划等），以确保无人机的稳定飞行和精确控制。因此通常来说，芯片的主频越高，其处理飞控算法的能力就越强，能够更好地满足实时性要求。在选择芯片时，需要对比各芯片的主频，确保其能够满足飞控系统的需求。

RISC-V 架构的芯片支持不同的指令集版本和扩展指令集。指令集的丰富程度直接影响芯片的通用性和灵活性。例如，RV32IMAC 指令集支持整数运算、乘法和除法运算，这对于飞控算法中的基本计算是必要的。此外，一些扩展指令集（如浮点运算指令集）能够进一步提升芯片的计算能力，使其能够更高效地处理复杂的数学运算。在选择芯片时，需要检查其是否支持所需的 RISC-V 指令集版本和扩展指令集，以确保其能够满足飞控系统中各种算法的计算需求。比如 M（乘法和除法扩展）提供整数乘法和除法指令，增强处理器的运算能力。其中 mul 指令用于整数乘法，div 指令用于整数除法。而浮点运算在小四轴无人机的飞控系统中扮演着重要角色。传感器数据（如加速度、角速度、气压等）通常以浮点数形式表

示，且飞控算法（如 PID 控制、姿态估计等）需要对这些浮点数进行复杂的数学运算。如果芯片不支持浮点运算，开发人员可能需要通过软件模拟浮点运算，这将大大降低系统的运行效率^[30]。因此，支持浮点运算的芯片能够显著提高飞控系统的性能，确保其在处理浮点数据时的实时性和准确性。

虽然目前大多数用于小四轴无人机的芯片仍以单核为主，但随着无人机功能的复杂化和计算需求的增加，多核架构逐渐成为一种趋势。多核芯片能够通过并行处理提高系统的整体性能，例如，一个核心可以专门处理飞控算法，而另一个核心可以处理传感器数据采集或通信任务。此外，多核芯片还需要具备高效的通信机制和任务调度机制，以确保各核心之间的协同工作。在选择芯片时，如果应用场景对计算能力有较高要求（比如需要目标识别、路径规划、自主导航、避障），可以考虑支持多核架构的芯片，并评估其多核之间的通信和任务调度机制是否满足需求。

片上 SRAM 是芯片用于存储程序运行时的变量和数据的重要资源。在飞控系统中，实时性要求高，因此需要足够的 SRAM 来存储临时数据和变量，以避免频繁访问外部存储器带来的延迟。例如，IMU 数据的采集和处理需要在短时间内完成，足够的 SRAM 能够确保这些数据能够快速存储和读取，从而提高系统的响应速度^[30]。在选择芯片时，需要根据飞控系统的程序大小和数据处理需求，评估芯片的 SRAM 容量是否足够。除了 SRAM 还有 FLASH 资源，片上 Flash 用于存储程序代码和固件。飞控系统的程序代码通常包含多个模块（如传感器驱动、飞控算法、通信协议等），因此需要足够的 Flash 空间来存储这些代码。此外，固件的更新也需要一定的 Flash 空间^[31]。在选择芯片时，需要考虑芯片的 Flash 容量是否能够满足程序代码和固件存储的需求，避免因存储空间不足而导致的程序运行问题。

芯片的外设接口资源是芯片与外部设备进行通信和数据交换的重要部分，它们增强了芯片的功能并使其能够适应各种应用场景。以下是一些需要考虑的外设接口资源：

（1）GPIO：通用输入输出引脚（GPIO）是芯片与外部设备连接的基本接口。在小四轴无人机中，GPIO 用于连接各种传感器（如 IMU、气压计、GPS 等）和驱动器（如电机、舵机等）。足够的 GPIO 数量能够确保所有必要的设备都能够连接到芯片上。例如，一个典型的四轴无人机需要至少 4 个 GPIO 引脚来控制电机的

PWM 信号输出，还需要额外的 GPIO 引脚用于传感器的连接和状态监测。在选择芯片时，需要根据无人机的硬件设计需求，评估芯片提供的 GPIO 数量是否满足要求。

(2) SPI/I2C/UART: 这些串行通信接口用于连接外部传感器和通信模块。SPI 和 I2C 接口常用于连接 IMU、气压计等传感器，UART 接口则用于连接 GPS 模块、Wi-Fi 模块等通信设备。不同的传感器和模块可能需要不同的通信接口，因此芯片需要提供足够的 SPI、I2C 和 UART 接口数量。此外，这些接口的速率也会影响数据传输的效率。例如，高速 SPI 接口能够更快地读取传感器数据，从而提高系统的实时性。在选择芯片时，需要检查芯片提供的 SPI、I2C 和 UART 接口数量和速率，以确保其能够满足无人机系统的通信需求。

(3) ADC/DAC: 模数转换器 (ADC) 和数模转换器 (DAC) 是芯片与模拟信号设备交互的关键接口。ADC 用于将传感器的模拟信号 (如电压信号) 转换为数字信号，以便芯片进行处理。例如，IMU 中的加速度计和陀螺仪输出的模拟信号需要通过 ADC 转换为数字信号后才能被芯片读取。DAC 则用于将数字信号转换为模拟信号，例如在电机 PWM 信号输出中，DAC 可以将数字控制信号转换为模拟 PWM 信号，从而控制电机的转速。在选择芯片时，需要评估其 ADC 和 DAC 的分辨率、精度和数量，以确保其能够满足传感器信号采集和电机控制的需求。

接口的带宽和速率直接影响数据传输的效率。例如，SPI 接口的速率决定了传感器数据的读取速度，I2C 接口的速率影响传感器数据的传输延迟，而 USB 接口的带宽则决定了数据传输的速率。对于小四轴无人机而言，高带宽和高速率的接口能够更快地传输传感器数据和控制指令，从而提高系统的实时性和响应速度。在选择芯片时，需要对比各芯片的接口带宽和速率，确保其能够满足无人机系统的数据传输需求。接口兼容性是指芯片的接口是否能够与市场上主流的传感器和通信模块兼容^[32]。良好的接口兼容性能够减少硬件设计的复杂性和开发成本。例如，如果芯片的 SPI 接口与主流的 IMU 传感器兼容，开发人员可以直接使用这些传感器而无需额外的适配电路。在选择芯片时，需要研究其接口的电气特性、协议规范等，确保其与无人机系统中所使用的传感器和通信模块兼容，从而避免因接口不匹配而导致的系统集成问题

3.1.2 芯片的综合参数对比和目标飞控平台的所需资源量化

根据前期对市面上主流 RISCV 架构芯片制造厂商和其产品线的初步调研，最终选择以下几款芯片比较，CH32V103 和 CH32V203 均是由南京沁恒微电子有限公司推出的 32 位通用微控制器，CH32V103 是一款基于 RISC-V3A 处理器的 32 位通用微控制器，CH32V203 是同一个产品线中基于 RISC-V4B 内核的工业级增强型低功耗通用微控制器。GD32VF103 则是全球首款 RISC-V 内核 32 位通用 MCU。在比较中，STM32F103 作为常见飞控平台代表性 MCU 以比较基准的形式出现。

表 3.1 四款 MCU 的参数列举

参数/芯片型号	CH32V103	CH32V203	GD32VF103	STM32F103
内核	青 稞 RISC-V3A	青稞 RISC-V4B	RISC-V umblebee 处理器内核	ARM Cortex-M3
主频 (MHz)	80	144	108	72
指令集支持	RV32IMAC	RV32IMAC	RV32IMAC	Thumb-2
浮点运算支持	无	无	无	无
SRAM (KB)	20	64	32	20
Flash (KB)	64	224	128	128
GPIO 数量	51	51	80	80
ADC 通道数	16	16	16	16
ADC 分辨率	12 位	12 位	12 位	12 位
定时器数量	7	10	11	10
USART 数量	3	4	5	3
SPI 数量	2	2	3	2
I2C 数量	2	2	2	2
USB 接口	USB2.0	USB2.0	USB2.0	USB2.0
CAN 接口	无	1	2	1

在小四轴无人机飞控系统的设计中,为了确保飞控系统能够高效地处理多个传感器数据并进行复杂计算,主频成为了一个关键指标。飞控算法要求主频达到 72MHz 一般可以完成普通小型飞控系统程序的计算负载,以满足实时性需求。在评估的芯片中,CH32V103 的主频为 80MHz,CH32V203 的主频高达 144MHz,GD32VF103 的主频为 108MHz,而 STM32F103 的主频为 72MHz,刚好满足最低要求。这些芯片均具备足够的处理能力,其中 CH32V203 凭借其最高的主频 144Mhz,在处理复杂任务时会显著的表现更为出色。

尽管所列举的芯片不支持硬件浮点运算,但可以通过软件模拟来实现。飞控系统要求芯片支持 RV32IMAC 指令集,以确保基本运算和浮点运算的效率。三款芯片均支持 RV32IMAC 指令集,能够确保基本运算和浮点运算的效率

在存储资源方面,通常一个小四轴主控板的系统程序要求 SRAM 至少为 20KB,Flash 至少为 64KB。CH32V103 配备了 20KB 的 SRAM 和 64KB 的 Flash,CH32V203 则提供了 64KB 的 SRAM 和 224KB 的 Flash,GD32VF103 拥有 32KB 的 SRAM 和 128KB 的 Flash,STM32F103 也配备了 20KB 的 SRAM 和 128KB 的 Flash。这些芯片均能满足飞控系统对存储资源的基本需求,其中 CH32V203 在 SRAM 和 Flash 容量方面表现更为优秀,为系统提供了更多的存储空间^[33]。

GPIO 数量是飞控系统连接传感器和电机的关键资源。飞控系统至少需要 40 个 GPIO 来满足连接需求。CH32V103 和 CH32V203 均提供了 51 个 GPIO,GD32VF103 提供了 80 个 GPIO,STM32F103 也提供了 80 个 GPIO。这些芯片均能满足飞控系统对 GPIO 数量的需求,其中 GD32VF103 和 STM32F103 提供了更多的 GPIO,为系统扩展提供了更大的灵活性。飞控系统需要高精度的 ADC 来采集传感器数据,至少需要 16 个 ADC 通道,且分辨率应达到 12 位。所有评估的芯片均提供了 16 个 12 位 ADC 通道,完全满足飞控系统对 ADC 通道数和分辨率的要求。所有评估的芯片均提供了足够的 USART、SPI 和 I2C 接口,满足飞控系统对通信接口的需求。

在工作电压和温度范围方面,飞控系统要求工作电压范围为 2.7V~5.5V,工作温度范围为-40℃~85℃。CH32V103、CH32V203 和 GD32VF103 的工作电压和温度范围均能满足飞控系统的需求,而 STM32F103 的工作电压范围较窄(2.0V~3.6V),但其工作温度范围更宽(-40℃~105℃),在某些极端环境下可能更具优势。

综合以上对比和量化分析,CH32V103、CH32V203、GD32VF103 和 STM32F103 均能满足小四轴无人机飞控系统的基本需求。然而, CH32V203 在主频和 SRAM 容量方面表现更为出色, 且提供了更多的外设资源, 适合对实时性和计算能力要求较高的应用场景。STM32F103 已有开源小四轴飞控项目的落地应用 (crazepony 等), 证明其作为比较基准的合理性。

3.1.3 芯片的开发工具链, 价格, 普及程度比较

在选择适合小四轴无人机飞控平台的微控制器时, 开发工具链的完善性、芯片的价格是两个重要的考量因素。这些因素不仅影响开发的便利性和成本, 还决定了芯片在市场上的接受度和应用范围。以下是对 CH32V103、CH32V203、GD32VF103 和 STM32F103 这四款芯片在这两个方面的详细比较。

CH32V103 和 CH32V203 均使用 MounRiver Studio (简称 MRS) 作为开发环境。MRS 是基于 Eclipse 深度定制的开发环境, 支持 Keil 工程一键导入和 RISC-V/ARM 跨内核迁移, 提供了丰富的外设例程和完整的文档支持。此外, CH 系列还可以使用开源的工具链和烧录软件 wchisp 进行开发。GD32VF103 系列则提供了更为广泛的开发工具支持。除了 MounRiver Studio 外, 还支持 Nuclei Studio (基于 Eclipse 架构) 和 LonganIDE (基于 Visual Studio 源码和 PlatformIO 插件)。此外, GD32VF103 系列还兼容多种操作系统和软件栈, 如 FreeRTOS、RT-Thread、Arduino 等, 这使得 GD32VF103 系列可以轻松地实现和部署各种应用。STM32F103 作为一款成熟的 ARM 架构微控制器, 拥有广泛的开发工具支持。其开发环境包括 STM32CubeIDE、Keil MDK、IAR EWARM 等, 这些工具提供了强大的代码编辑、调试和优化功能。

价格同样是在后续生产落地环节十分关键的参数之一。CH32V203 的批量价格在 2~5 元左右, 这一价格在同系列芯片中属于非常低的水平。CH32V103 的价格在 11 元左右。GD32VF103 系列的价格相对较高, 在采购平台上价格位 22 元左右。STM32F103 的价格在市场上较为稳定, 在采购平台的价格为 10 元, CH32V203 具有显著价格优势。

3.2 CoreMark 基准测试性能评估

3.2.1 Coremark 的引入

为了能够量化这几款芯片的性能强弱, 本文将在这一章节采用 CoreMark 测试 CH32V103, CH32V203, GD32VF103 和用于比较的基准 STM32F103 这四款芯片,

CoreMark 是一个广泛使用的基准测试工具，主要用于评估 MCU 的性能，尤其是在嵌入式系统中。它由嵌入式微处理器基准联盟（EEMBC，Embedded Microprocessor Benchmark Consortium）开发和维护，旨在提供一个标准化、可重复的测试方法，以衡量处理器在常见计算任务中的表现。CoreMark 主要用于评估处理器在以下方面的性能：首先是单线程性能：衡量处理器在单线程任务中的执行效率。整数运算能力则用于测试处理器对整数运算的处理速度。内存访问性能能够评估处理器对内存的读写速度。代码密度通过测试代码的紧凑性，反映处理器在嵌入式系统中的适用性。CoreMark 与其他基准测试的比较中，例如 Dhrystone：是一个经典的基准测试工具，主要用于评估处理器的整数运算性能。它通过执行一系列简单的计算任务来测量处理器的性能，但测试内容相对单一。而 CoreMark 在 Dhrystone 的基础上，增加了矩阵运算、状态机和 CRC 校验等模块，测试内容更加丰富，能够更全面地评估处理器的性能^[34]。

资料显示该评测标准可能存在一些局限性，CoreMark 主要测试处理器的整数运算能力，对浮点运算的支持较少。但是在小四轴飞控的项目中浮点数的计算是通过 MATH 库的软件模拟，所以整数计算能力能够反映在该项目中处理器的处理计算任务的性能。除此之外 CoreMark 的测试结果主要反映处理器的单线程性能，对于多核处理器，其多线程性能的评估可能不够全面，但是这在我们的测试中不构成干扰（四款 MCU 都是单核架构）。

3.2.2 coremark 代码流程拆解

文章这部分将拆解 CoreMark 的主要流程，CoreMark 的主要工作代码都打包在一个名为 `core_main.c` 的文件里面，CoreMark 代码基于 Apache License 2.0 授权，是一个相对宽松的开源软件协议，允许使用者自由地使用、复制、分发、传播和修改软件。以下是 `core_main.c` 的代码流程图。

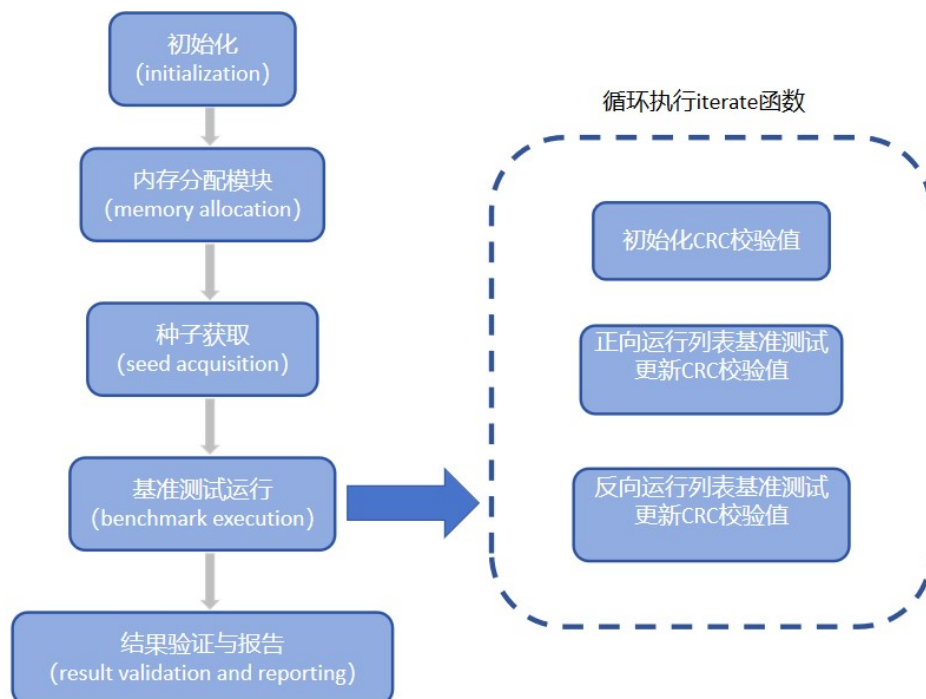


图 3.1 core_main.c 的代码流程图

CoreMark 基准测试框架的设计采用了模块化结构，这种结构不仅提高了代码的可维护性和可扩展性，还使得各个功能模块之间的关系清晰明确。整个框架可以分为六个主要模块：初始化模块、内存分配模块、种子值获取模块、基准测试运行模块、结果验证模块和结果报告模块。这些模块相互协作，共同完成了从测试初始化到结果输出的全过程。

初始化模块是整个测试框架的基础，负责完成测试前的准备工作。它初始化串口通信，确保测试过程中能够正确输出调试信息和结果。此外，该模块还根据配置选择合适的内存分配方式，并通过特定的函数获取种子值。种子值是基准测试的重要输入参数，用于初始化数据结构和控制测试流程。portable_init 函数是初始化模块的核心，它为不同目标平台提供了统一的初始化接口，确保测试框架能够在多种硬件平台上运行。

内存分配模块的功能是为基准测试分配所需的内存资源。根据配置选项，内存分配方式可以是静态分配、堆分配或栈分配。静态分配使用预先定义的内存块，适用于内存需求固定且已知的情况；堆分配通过动态内存分配函数（如 malloc）获取内存，适用于内存需求可能变化的情况；栈分配则利用函数调用栈分配内存，适用于内存需求较小且临时使用的情况。这种灵活的内存分配方式使得 CoreMark 能够适应不同的测试场景和硬件限制。

种子值获取模块负责获取基准测试所需的种子值。种子值的获取方式取决于配置选项 `SEED_METHOD`。如果该选项设置为 `SEED_ARG`，则从命令行参数中获取种子值；否则，通过预定义的函数或全局变量获取种子值。种子值的正确获取对于确保测试结果的可重复性和准确性至关重要。

基准测试运行模块是 CoreMark 的核心部分，负责执行实际的基准测试。该模块根据种子值和配置运行多种类型的算法，包括列表基准测试、矩阵基准测试和状态机基准测试。`iterate` 函数是该模块的核心，它控制了基准测试的迭代过程。在每次迭代中，`core_bench_list` 函数用于运行列表基准测试，`core_init_matrix` 和 `core_init_state` 函数分别用于初始化和运行矩阵基准测试和状态机基准测试。通过多次迭代运行这些算法，程序能够全面评估处理器在不同计算任务下的性能表现。

结果验证和报告模块的作用是验证基准测试的结果是否正确并负责输出基准测试的结果，该模块通过比较计算出的 CRC 校验值与已知的正确值来判断测试结果的有效性，然后输出测试的基本信息，如迭代次数、运行时间、每秒迭代次数等，还根据结果验证模块的输出报告测试结果的正确性。

首先是 `core_bench_list` 的解读，在 CoreMark 基准测试中，`core_bench_list` 函数是链表处理任务的核心实现。它通过一系列复杂操作，全面评估处理器在数据结构操作、内存管理以及计算能力方面的性能表现。该函数接收一个指向 `core_results` 结构体的指针 `res` 和一个整数 `finder_idx` 作为参数，其中 `res` 包含了链表的指针以及其他测试参数，而 `finder_idx` 用于指定查找操作的起始位置。函数的逻辑可以分为以下几个关键部分：首先，函数初始化了一些变量，包括返回值 `retval`、找到和未找到的计数器 `found` 和 `missed`，以及链表的头指针 `list`。接着，通过一个循环，函数执行了多次查找操作。在每次查找中，它使用 `core_list_find` 函数在链表中搜索特定的值。如果找到目标节点，函数会根据节点的数据更新返回值 `retval`，并将该节点的下一个节点移动到链表的头部，模拟缓存操作。如果未找到目标节点，则增加未找到的计数器 `missed`，并根据链表的某些属性更新返回值。每次查找后，链表都会被反转，以增加操作的复杂性。在查找操作完成后，函数会根据 `finder_idx` 的值决定是否对链表进行排序。如果需要排序，它会调用 `core_list_mergesort` 函数，使用归并排序算法对链表进行排序。排序完成后，函数会从链表中移除一个节点，并计算从指定索引位置开始的链表数据的 CRC 校验值。然后，它会撤销之前的移除操作，将链表恢复到原始状态，并再次对链表进

行排序，这次是按照索引顺序排序。最后，函数会计算整个链表数据的 CRC 校验值，并将该值作为最终的返回值。`core_bench_list` 函数的设计不仅涵盖了链表的查找、反转、排序、节点插入和删除等操作，还通过 CRC 校验机制确保了操作的正确性。这些操作的复杂性和多样性使得该函数能够有效地评估处理器在处理复杂数据结构时的性能，特别是在内存操作、动态内存管理和计算能力方面。通过这种方式，CoreMark 能够为处理器的性能提供一个全面且可靠的评估指标。

然后再介绍 `core_bench_matrix` 在 CoreMark 基准测试中，矩阵运算模块是评估处理器性能的关键任务之一。该模块通过一系列复杂的矩阵操作来模拟科学计算和工程应用中的常见场景，从而全面测试处理器在数值计算和数据处理方面的能力。矩阵运算任务主要包括对矩阵进行加法、乘法、向量乘法以及矩阵与矩阵的乘法操作，这些操作不仅涉及大量的浮点运算，还要求高效的内存访问和数据管理能力。为了确保操作的正确性并验证处理器的可靠性，每次矩阵操作后都会计算矩阵的和，并通过 CRC 校验算法生成校验值。这一过程由 `matrix_test` 函数实现，该函数是矩阵运算的核心，负责执行所有矩阵操作并返回最终的 CRC 校验值。在 `matrix_test` 函数中，首先通过 `matrix_add_const` 函数将常量加到矩阵的每个元素上，随后通过 `matrix_mul_const`、`matrix_mul_vect` 和 `matrix_mul_matrix` 等函数依次执行矩阵与常量的乘法、矩阵与向量的乘法以及矩阵与矩阵的乘法操作。每一步操作后，都会调用 `matrix_sum` 函数计算矩阵的和，并通过 `crc16` 函数更新 CRC 校验值。最终，通过 `matrix_add_const` 函数将矩阵恢复到初始状态，确保测试过程的完整性和一致性。

最后是 `core_state.c`，在 CoreMark 基准测试中，状态机处理任务是评估处理器性能的关键部分之一。状态机作为一种常见的计算模型，广泛应用于嵌入式系统中，用于处理各种输入信号并根据预定义的规则进行状态转换。CoreMark 中的状态机任务通过模拟一个简单的状态机，测试处理器在处理复杂逻辑和状态转换时的性能表现。这一任务的核心函数是 `core_bench_state` 和 `core_state_transition`。`core_bench_state` 函数负责初始化输入数据并运行状态机，它通过多次遍历输入数据，模拟正常和引入干扰后的状态转换过程，最终通过 CRC 校验值验证结果的正确性。而 `core_state_transition` 函数则是状态机的核心实现，它根据输入字符进行状态转换，通过一系列预定义的规则判断输入字符串的格式是否有效。这些函数

的设计不仅能够有效评估处理器在逻辑处理和内存访问方面的性能，还通过引入干扰和恢复操作，测试了处理器在处理错误输入时的鲁棒性。

3.2.3 coremark 任务操作

(1) 移植准备，在开始 CoreMark 移植之前，需要准备以下资源：首先，选择适合目标 MCU 的集成开发环境（IDE）。例如，对于 CH32V 系列，MounRiver Studio 是一个合适的选择。其次，准备目标 MCU 的开发板，如 CH32V103R8T6 开发板。最后，从 EEMBC 官方网站下载 CoreMark 的开源软件包。如果无法直接下载，可以参考相关技术支持论坛提供的备用下载链接。

(2) 工程搭建与文件准备，在 IDE 中创建一个新的工程，并配置好目标 MCU 的相关设置，包括时钟配置和外设初始化等。接下来，从 CoreMark 软件包中提取必要的文件，包括 `core_list_join.c`、`core_main.c`、`core_matrix.c`、`core_state.c`、`core_util.c` 和 `coremark.h`，并将它们放置到工程的指定目录中。此外，还需要从 `simple` 文件夹中复制 `core_portme.c` 和 `core_portme.h` 到工程目录下。

(3) 初始化代码迁移，CoreMark 的 `core_main.c` 文件中已经定义了 `main()` 函数，这与目标 MCU 的初始化代码可能存在冲突。因此，需要将原工程中 `main.c` 文件的初始化代码移动到 `core_portme.c` 中的 `portable_init()` 函数中。在 CH32V103 的示例中，初始化代码包括系统时钟配置、GPIO 初始化、定时器初始化和串口初始化。系统时钟配置用于设置时钟源和时钟分频，确保 MCU 运行在适当的频率。GPIO 初始化配置了用于串口通信的引脚，定时器初始化设置了定时器的中断频率，而串口初始化则配置了串口的波特率和工作模式。

(4) 定时器配置，CoreMark 的性能测试依赖于精确的时间测量。在 `core_portme.c` 中，需要定义每秒的时钟滴答数（`EE_TICKS_PER_SEC`），例如设置为 1000，表示每秒 1000 个滴答。定时器的启动、停止和计数值获取功能分别由 `start_time()`、`stop_time()` 和 `get_time()` 函数实现。以 TIM1 定时器为例，`start_time()` 函数通过启用定时器和中断来开始计时，`stop_time()` 函数则停止定时器和中断。`get_time()` 函数返回定时器的计数值，用于计算测试时间。定时器的中断处理函数负责在每个滴答时更新计数值，从而实现时间的累积。

(5) 测试参数调整和编译器优化设置，CoreMark 要求测试时间至少为 10 秒，以确保测试结果的准确性。根据目标 MCU 的主频，需要调整 `core_portme.c` 中的 `ITERATIONS` 值。例如，对于 72MHz 的 CH32V103，`ITERATIONS` 可以设置为 1200。

这一参数的调整是为了确保测试时间满足 CoreMark 的要求，同时避免测试时间过短导致的误差。为了获得最佳性能，需要在 `core_portme.h` 中通过 `COMPILER_FLAGS` 宏定义编译器优化等级和调试等级。例如，可以使用 `-Ofast` 优化和 `-g3` 调试等级。优化等级的选择对测试结果有显著影响，因此需要根据目标 MCU 的特性进行合理设置。

(6) 打印输出重定向，为了将测试结果输出到串口，需要重定向 `printf` 函数。通过实现一个自定义的 `_write` 函数，可以将数据发送到串口。该函数在发送每个字节时，会等待串口发送完成标志 (`USART_FLAG_TC`)，然后发送下一个字节。通过这种方式，测试结果可以通过串口调试助手查看。

(7) 编译与运行，完成上述步骤后，编译工程并确保无误。即可将编译好的程序烧录到目标 MCU 开发板上，并运行程序。通过串口调试助手查看 CoreMark 的测试结果。

3.2.4 coremark 结果展示

CoreMark 的测试结果以“每秒迭代数”表示，分数越高，表明 MCU 的性能越强。通过对比不同 MCU 的测试结果，可以评估它们的性能差异。如下图是一次典型的 coremark 跑分结果展示，其中标注了任务组大小、总运行时间（coremark 要求运行时间大于 10s，以确保基准测试结果的合理性）、总循环次数、编译器版本、优化加速策略等，最后打印出了跑分结果。

```
CoreMark Start!
2K performance run parameters for coremark.
CoreMark Size:666
Total ticks:4589|
Total time (secs):12.188
Iterations/Sec: 1200Iterations
Compiler version :GCC8.2.0
Compiler flags :-Ofast -g3
Memory location :STACK
seedcrc: 0xe9f5
[0]crclist:0xe714
[0]crcmatrix:0x1fd7
[0]crcstate:0x8e3a
[0]crcfinal:0xbd59
Correct operation validated. See README.md for run and reproting rules.
CoreMark 1.0: 120.83872 / GCC8.2.0 - Ofast -g3 / STACK
```

图 3.2 跑分结果格式展示

下面的表格展示了本文所考虑的芯片的跑分结果：

表 3.2 MCU 的 CoreMark 跑分结果展示

处理器型号	CH32V103	CH32V203	GD32VF103	STM32F103
跑分结果	357.739	788.424	426.229	675.348

CoreMark 的具体得分的核心决定要素通常是处理器的时钟频率、核心数量、指令集架构以及优化程度等，而且在实际应用中的表现还受到软件优化和特定应用场景的影响。根据这里的跑分结果可以为我们的处理器选择提供关键参考。

3.3 完成 RISCv 架构平台的硬件原理图设计

在这个部分文章将完成完成 RISCv 架构平台的框架设计和实例化，现已经在 EDA 设计平台绘制了小型四轴无人机主控板的电路原理图，下面将逐一讲解各部分的原理。

3.3.1 CH32V203C8T6 的原理图设计

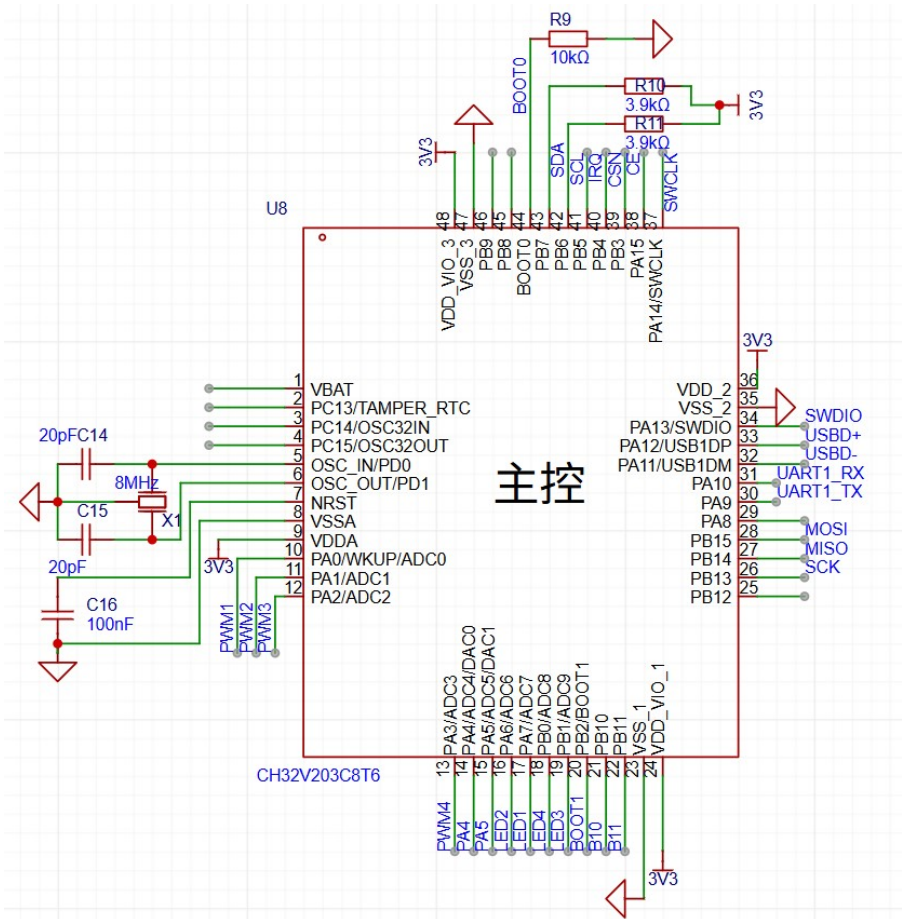


图 3.3 CH32V203C8T6 的原理图

该部分将详细解析 CH32V203C8T6 在小四轴飞控主板中的原理图设计，涵盖电源管理、时钟配置、引脚分配及外围电路设计等方面。

电源管理,电源管理是确保微控制器稳定运行的关键。CH32V203C8T6 的电源引脚（VDD 和 VDDA）连接至 3.3V 电源轨。为了确保电源的稳定性，电路设计中采用了去耦电容（C15 和 C16），分别为 20pF 和 100nF。这些电容能够有效滤除电源噪声，确保微控制器的正常工作。此外，得益于 CH32V203 的低运行功耗，可用 LDO 直接进行 15V 转 3.3V，在 U9 输入端加 RC 电路，可以增强 MCU 电源稳定性，并承担部分耗散功率。

时钟配置,时钟电路是微控制器正常工作的核心。CH32V203C8T6 的时钟输入引脚（OSC_IN 和 OSC_OUT）通过 8MHz 晶振（X1）连接。晶振两端并联 20pF 电容（C15 和 C16），以确保振荡器的稳定启动和运行。此外，RTC（实时时钟）电路通过 PC13 和 PC14 引脚连接，以提供低功耗模式下的时钟源。系统引入 4 组时钟源，包括内部高频 RC 振荡器(HSI)、内部低频 RC 振荡器(LSI)、外接高频振荡器(HSE)、外接低频振荡器(LSE)。低频时钟源为 RTC 和独立看门狗提供时钟基准，高频时钟源通过 PLL 倍频后输出为系统总线时钟(SYSCLK)，再由各预分频器提供外设控制时钟及采样或接口输出时钟。

引脚分配，CH32V203C8T6 的引脚分配如图 3.1 所示。主要功能引脚包括：UART 接口：PA9 和 PA10 分别配置为 UART1_TX 和 UART1_RX，用于串行通信。这些接口可以用于代码下载调试、虚拟示波器波形观测等功能。SPI 接口：PB12、PB13、PB14 和 PB15 分别配置为 SPI 的 SCK、MOSI 和 MISO 引脚，用于高速数据传输。这些接口支持多主模式，全双工或半双工同步传输，支持基本的 SD 卡和 MMC 模式。GPIO 接口：多个引脚（如 PA0、PA1、PA2 等）配置为通用输入输出引脚，用于控制和监测外部设备。这些引脚可以由软件配置成输出（推挽或开漏）、输入（带或不带上拉或下拉）或复用的外设功能端口。

外围电路设计，外围电路设计包括 BOOT 引脚配置和复位电路。BOOT 引脚通过 10kΩ电阻（R9）连接至 3.3V 电源，用于选择启动模式。复位电路通过 3.9kΩ电阻（RT0 和 RT1）连接至 3.3V 电源，确保在上电时微控制器能够正确复位。此外，MCU 的 1&2 脚为多功能复用引脚，内部可通过代码配置为 SWD、USB、I2C、串口或普通 IO。

3.3.2 MPU6050 的原理图设计

本文将详细分析 MPU6050 的原理图设计，包括电源管理、信号接口、时钟配置和中断处理等方面。

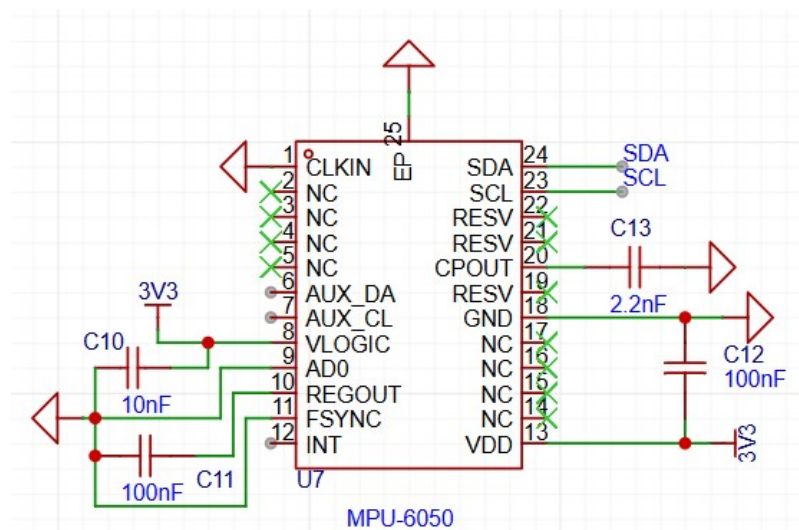


图 3.4 MPU6050 的原理图

电源管理，MPU6050 的电源管理是其正常工作的基础。根据数据手册，MPU6050 的电源电压范围为 2.375V 至 3.46V。在原理图中，MPU6050 的电源引脚(VDD)连接到 3.3V 电源，确保其正常工作。此外，MPU6050 还具有一个 VLOGIC 引脚，用于设置 I2C 接口的逻辑电平。在原理图中，VLOGIC 引脚连接到 3.3V 电源，以匹配微控制器的逻辑电平。

信号接口，MPU6050 支持 I2C 和 SPI 两种通信接口。在原理图中，我们选择了 I2C 接口进行数据通信。I2C 接口包括 SDA（串行数据）和 SCL（串行时钟）两条信号线。在原理图中，SDA 和 SCL 引脚分别连接到微控制器的相应 I2C 引脚，并通过上拉电阻连接到 3.3V 电源，以确保信号线在空闲时保持高电平。

时钟配置，MPU6050 的时钟配置对于其性能至关重要。在原理图中，我们使用了外部晶振作为时钟源。晶振连接到 MPU6050 的 CLKIN 引脚，为 MPU6050 提供稳定的时钟信号。此外，MPU6050 还具有内部时钟源，可以在没有外部晶振的情况下使用。

中断处理和其他配置，MPU6050 具有中断功能，可以在特定事件发生时向微控制器发送中断信号。在原理图中，MPU6050 的 INT 引脚连接到微控制器的中断引脚。当 MPU6050 检测到运动或姿态变化时，它会通过 INT 引脚发送中断信号，通知微控制器进行相应的处理。除了上述主要配置外，MPU6050 还有一些其他引

脚需要配置。例如，AD0 引脚用于设置 I2C 地址的最低位，FSYNC 引脚用于帧同步，REGOUT 引脚用于连接外部滤波电容等。在原理图中，这些引脚根据实际需要进行了相应的配置。

3.3.3 电机驱动的原理图设计

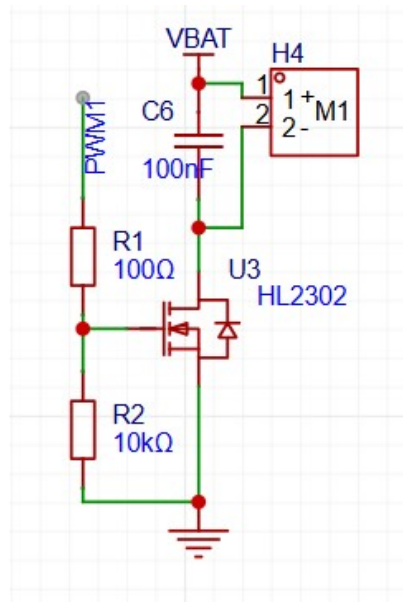


图 3.5 电机驱动的原理图

当 PWM 信号输入端接收到高电平信号时，晶体管 U3 的基极电流增加，使其导通。此时，电源 VBAT 通过晶体管集电极向电机 H4 供电，电机开始运转。PWM 信号的占空比决定了电机的平均电压，从而控制电机的转速。电容 C6 在电源线上起到滤波作用，减少电源噪声对电机运转的影响。电阻 R1 和 R2 则确保晶体管在不同工作状态下的稳定性和安全性。电源 VBAT 为整个电路提供必要的工作电压。在小四轴飞行器中，VBAT 通常由锂电池提供，电压范围一般在 3.7V 至 4.2V 之间。电容 C6 (100nF) 用于滤波，平滑电源电压，减少电源噪声对电路的影响。电容的并联放置有助于提高电源的稳定性，确保电机运行的平稳性。电阻 R1 (100Ω) 和 R2 (10kΩ) 构成一个分压电路，用于限制流过晶体管基极的电流，保护晶体管免受过流损坏。R1 的较小阻值有助于提高驱动信号的响应速度，而 R2 的较大阻值则有助于在电路未激活时将晶体管基极电压拉低，确保晶体管处于截止状态。晶体管 U3 (型号 HL2302) 是一个 NPN 型功率晶体管，用于放大和开关控制。其基极通过 R1 和 R2 连接到 PWM 信号输入端，发射极接地，集电极连接到电机。当 PWM 信号为高电平时，晶体管导通，电机获得电源电压并开始运转；当 PWM 信号为低电平时，晶体管截止，电机停止运转。电机 H4 是电路的负载，电机的运

转由 PWM 信号控制，通过改变 PWM 信号的占空比，可以调节电机的转速，从而控制飞行器的姿态和运动。

3.3.4 供电模块的原理图设计

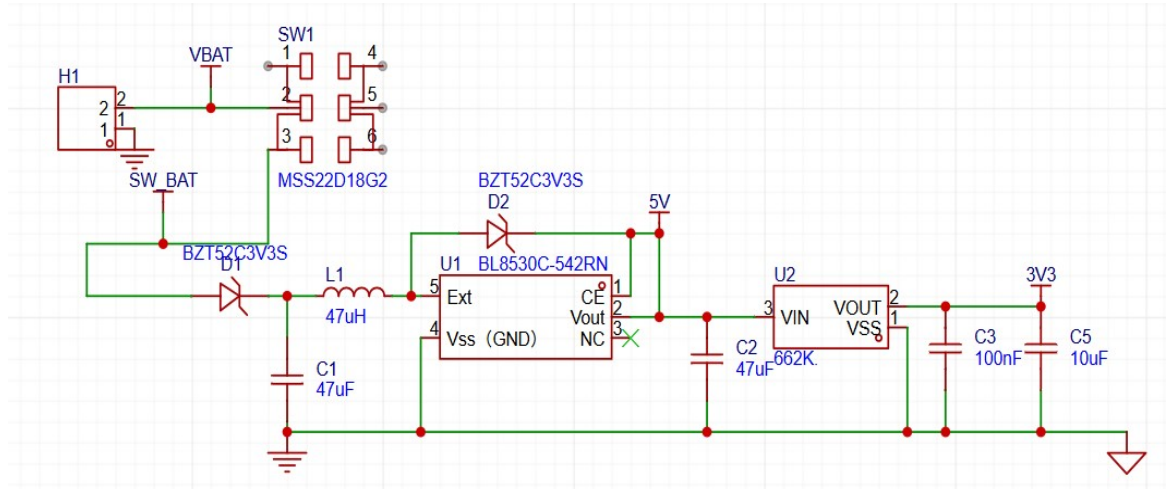


图 3.6 供电模块的原理图

供电电路的主要功能是将电池提供的电压转换为飞控主板所需的稳定电压。图 3.6 展示了小四轴飞控主板供电部分的原理图，该电路主要由电源开关、电感、电容、稳压器和二极管等元件组成。电源开关（SW1）型号为 MSS22D18G2，是一个六引脚的单刀双掷开关，其作用是控制电源的通断。当开关闭合时，电路接通，电源开始工作；当开关断开时，电路断开，电源停止工作。电感（L1）型号为 47 μ H，用于滤波和储能。在电源电路中，电感的主要作用是抑制电流的突变，减少电源噪声，提高电源的稳定性。稳压器（U1）型号为 BL8530C-542RN，是一个低压差线性稳压器（LDO），其作用是将输入电压（5V）转换为飞控主板所需的稳定电压（3.3V）。U2 是一个低压差线性稳压器，用于进一步稳定输出电压。二极管（D2）型号为 BZT52C3V3S，是一个齐纳二极管，用于电压钳位。当电压超过 3.3V 时，二极管导通，将多余的电压钳位到地，保护后续电路元件。

当电源开关（SW1）闭合时，电池提供的电压通过电感（L1）和电容（C1）进行初步滤波，然后进入稳压器（U1）。稳压器（U1）将输入电压（5V）转换为稳定的 3.3V 电压，并通过电容（C2）进行进一步滤波。输出电压通过电容（C3）和（C5）进行最终滤波，确保输出电压的稳定性。二极管（D2）用于电压钳位，保护后续电路元件。这种设计确保了飞控主板能够获得稳定且干净的电源，从而保证了无人机的稳定运行和性能。

第四章 小四轴无人机飞控板的实例化与系统架构

4.1 飞控板设计

在完成了原理图的设计之后，本文在 EDA 软件中完成了小四轴无人机飞控板的 layout，后续完成硬件的实例化和功能测试，下图是飞控板的布局。

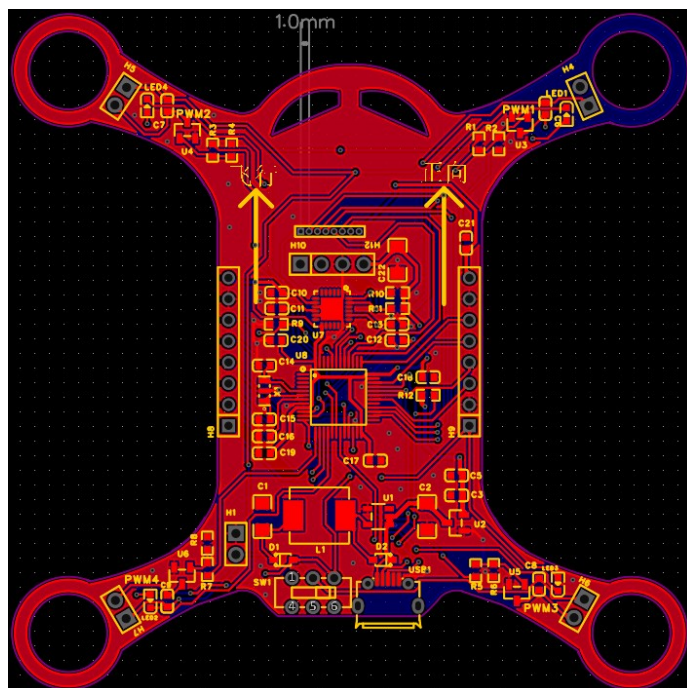


图 4.1 飞控板的 Layout

该电路板布局设计为十字形，四个角设有安装孔，便于固定在飞行器框架上，电路板的中心是主控芯片 CH32V203C8T6，它负责处理飞行控制算法。中下方为供电模组和电源滤波模块等，最下面为开关和 USB 供电接口。上方预留的空白部分和排针用于焊接 NRF23L01 模块。

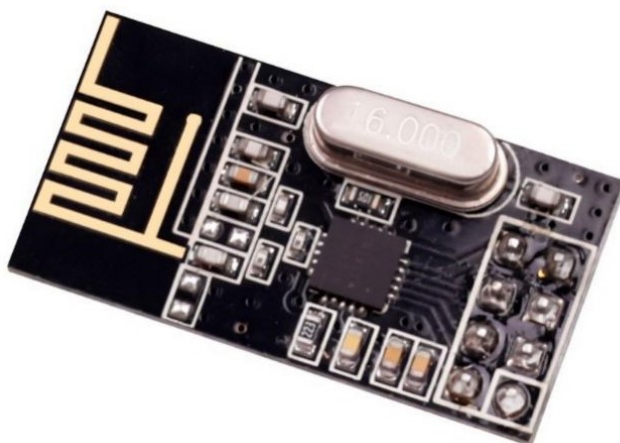


图 4.2 常见的 NRF24L01 集成模块

四周为 PWM 输出（PWM1-PWM4）连接到电机驱动器，控制电机的速度和方向。LED 指示灯用于状态指示，如电源状态、飞行模式等。调试接口（SWD）则用于程序下载和调试。板上标注的“飞向”和“正向”帮助用户正确安装和调试飞行器。

4.2 主控程序的系统架构介绍

主控程序是无人机飞行控制系统的核心，其架构设计直接影响到无人机的飞行性能和稳定性。本文通过对主控程序的详细分析，展示了其系统架构的各个关键环节，包括系统初始化、主循环以及中断服务与控制算法等部分。在初始化阶段，程序完成了硬件模块的配置和校准，为后续的飞行控制提供了基础支持。在主循环中，程序通过定时器中断实现了对飞行数据的实时采集和处理，并通过 PID 控制算法对无人机的姿态进行精确调整^[35]。

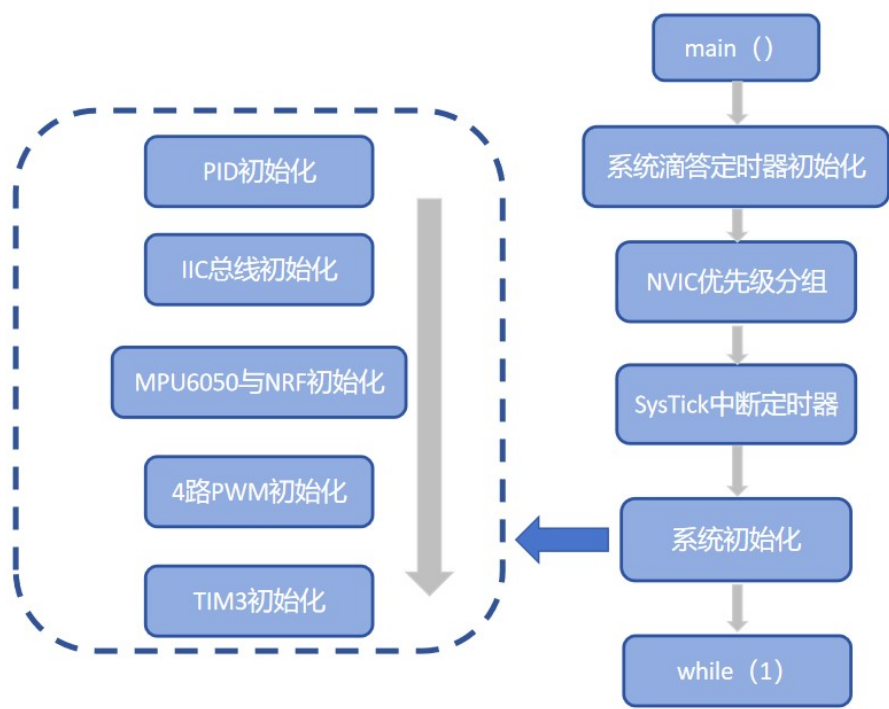


图 4.3 主控程序的系统架构

系统初始化阶段，在主控程序的系统架构中，初始化阶段是确保系统稳定运行的基础。程序启动后，首先对系统滴答定时器进行初始化，配置 NVIC 优先级分组，并设置 SysTick 中断定时器以 1000 微秒的周期产生中断，为系统提供高精度的时间基准。这一时间基准对于后续的定时任务调度和实时控制至关重要。随后，程序依次完成 NRF24L01 无线通信模块、4 路 PWM 信号发生器、定时器 3 中断、PID 参数、LED 指示灯以及 IIC 总线的初始化操作。在 IIC 总线初始化后，进

一步使能 APB2 时钟，并对相关引脚进行配置，确保惯性测量单元能够正常工作。MPU6050 初始化过程中，通过向其配置寄存器写入控制字并读取返回值，判断芯片是否正常工作。若芯片工作正常，则从单片机的特定存储地址读取 MPU6050 的静止校准值，以提高测量精度。这一校准过程对于提高惯性测量单元的测量准确性具有重要意义。NRF24L01 模块的初始化则包括对它的 IO 口进行初始化，使能相关时钟，配置引脚功能，并完成模拟 SPI 总线的初始化。通过一系列的自检操作，确保 NRF24L01 能够正常工作，并将其设置为接收模式，完成通信参数的配置。

主循环阶段，进入主循环阶段后，系统的核心任务是实现对无人机的实时飞行控制。首先，对 4 路 PWM 信号发生器进行再次初始化，使能 GPIOA_APB2 时钟，并对各 PWM 信号通道进行详细配置。具体来说，设置定时器计数周期、预分频系数、时钟分频系数以及计数模式，最终实现 PWM 频率的精确控制。通过精确的 PWM 信号控制，能够实现对无人机电机的精确驱动，从而实现对无人机姿态的精确控制。随后，使能定时器 2，并通过 TIM_OCInitTypeDef 结构体配置 PWM 模式，利用 TIM_OCnInit()和 TIM_OCnPreloadConfig()函数使能各通道，完成 PWM 信号的生成。定时器 3 中断初始化过程中，使能 APB1_TIM3 时钟，配置 NVIC 中断优先级，设置定时器中断的 ARR 和 PSC 参数，确保定时器能够以 3 毫秒的周期产生中断。这一中断周期的设置是根据无人机的飞行控制需求精心设计的，能够满足实时性要求较高的飞行控制任务^[36]。

中断服务与控制算法，在 TIM3 中断服务函数中，每次中断发生时，系统首先检查是否为 TIM3 中断，随后依次执行 MPU6050 数据获取、遥控数据解析以及 PID 控制等操作。每隔 6 毫秒，系统通过 IIC 总线读取 MPU6050 的加速度和角速度数据，将其左移八位并相或连接为 16 位完整数据。对加速度数据进行一维卡尔曼滤波处理，对角速度数据进行一阶低通滤波处理，以提高数据的准确性和可靠性。这一数据处理过程是飞行控制算法的基础，通过滤波处理能够有效降低噪声干扰，提高测量数据的可用性。PID 控制算法是实现无人机飞行控制的核心算法，通过 PID 运算能够实现对无人机姿态的精确调整。最终，根据 PID 运算结果以及油门值，计算出各电机的 PWM 信号值，并通过限幅处理后输出，实现对无人机的飞行控制。

4.3 成品和飞行结果展示

最终成品的展示如下图所示，WCH（沁恒）的 CH32V203C8T6 作为飞控板的 MCU，四周是外围电阻和电容。

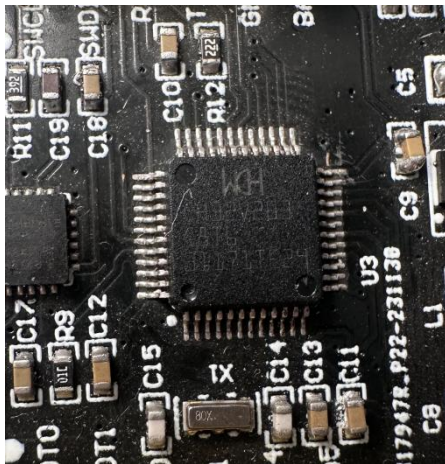


图 4.4 结果展示

在配合小四轴无人机遥控板和上位机的参数设置后，我们进行了无人机的飞行测试环节，无人机在遥控指令下平稳起飞并达到预设高度，悬停时姿态稳定，加速度和角速度数据波动小，滤波处理效果良好，姿态控制精度高。通过遥控板进行俯仰、横滚和偏航操作时，无人机响应迅速且平稳，姿态调整准确。在不同飞行速度下，无人机飞行轨迹符合预期，PWM 信号输出稳定，飞行控制性能良好。飞行数据表明，悬停时姿态角波动小，飞行轨迹偏差低，控制精度高。通过数据分析，进一步优化了 PID 参数，提升了飞行性能。



图 4.5 飞行结果展示

飞行测试结果表明，无人机控制系统能够实现稳定飞行和精确姿态调整，具备一定的抗干扰能力。未来将针对复杂环境进一步改进控制系统，提升飞行性能。

第五章 未来研究展望

5.1 硬件优化与性能提升

随着无人机应用场景的不断拓展和技术要求的日益提高，现有的无人机系统面临着性能提升、功能拓展和成本优化等挑战。未来的研究将聚焦于高性能 RISC-V 架构芯片的集成与优化。一方面，进一步探索多核 RISC-V 架构芯片在无人机飞控系统中的应用，充分发挥多核架构的并行处理能力，以满足复杂飞行任务（如目标识别、路径规划、自主导航、避障等）对计算能力的高要求。例如，通过任务分配算法优化，合理分配多核芯片中的任务负载，提高系统的整体性能和响应速度。另一方面，关注 RISC-V 架构芯片的指令集扩展和定制化设计，针对无人机飞控系统的特定需求，如浮点运算、矩阵运算等，优化指令集，进一步提升芯片的计算效率和性能表现。此外，研究高性能 RISC-V 架构芯片的功耗优化技术，降低芯片在高负载运行时的功耗，延长无人机的续航时间^[37]。

在无人机的飞行控制中，传感器数据的准确性和可靠性至关重要。未来的研究将致力于传感器融合技术的深化与创新。一方面，进一步优化多传感器数据融合算法，如卡尔曼滤波、粒子滤波等，提高传感器数据的融合精度和可靠性。通过引入先进的滤波算法和数据处理技术，减少传感器噪声和误差对飞行控制的影响，提高无人机的姿态估计精度和飞行稳定性。例如，结合深度学习算法，对传感器数据进行特征提取和分析，进一步提高数据融合的准确性和鲁棒性。另一方面，探索新型传感器技术在无人机中的应用，如高精度的惯性测量单元（IMU）、视觉传感器、毫米波雷达等，丰富传感器数据的来源和类型^[38]。通过多传感器的协同工作，实现对无人机周围环境的全方位感知和精确测量，为飞行控制提供更丰富的信息支持。例如，将视觉传感器与 IMU 数据融合，实现无人机的视觉辅助导航和避障功能，提高无人机在复杂环境中的自主飞行能力。

无人机的电源管理系统的性能直接影响其续航能力和飞行稳定性。未来的研究将重点关注电源管理系统的优化与创新。一方面，研究高性能电源管理芯片和电路设计，提高电源转换效率，降低电源系统的功耗和发热。例如，采用先进的电源转换技术，如开关电源、同步整流等，提高电源的转换效率，减少能量损失。另一方面，开发智能电源管理系统，实现对电池状态的实时监测和管理。通过监测电池的电压、电流、温度等参数，实时评估电池的健康状态和剩余电量，优化

电池的充放电过程，延长电池的使用寿命。此外，研究电源系统的冗余设计和故障切换机制，提高电源系统的可靠性和容错能力，确保无人机在电源系统出现故障时能够安全、稳定地飞行。

5.2 软件算法改进与功能拓展

飞行控制算法是无人机飞行控制系统的核心，其性能直接影响无人机的飞行性能和稳定性。未来的研究将致力于飞行控制算法的优化与改进。一方面，进一步优化 PID 控制算法的参数调整方法，通过引入自适应控制、模糊控制等先进的控制理论，提高 PID 控制算法的自适应能力和鲁棒性。例如，采用自适应 PID 控制算法，根据无人机的飞行状态和环境变化自动调整 PID 参数，提高系统的控制精度和响应速度。另一方面，研究先进的飞行控制算法，如模型预测控制、滑模控制等，探索其在无人机飞控系统中的应用。这些先进的控制算法具有更好的控制性能和抗干扰能力，能够提高无人机在复杂环境下的飞行稳定性和控制精度。此外，结合深度学习算法，开发基于数据驱动的飞行控制算法，通过大量的飞行数据训练神经网络模型，实现对无人机飞行状态的智能预测和控制。

随着无人机应用场景的复杂化，自主导航和避障能力成为无人机智能化发展的重要方向。未来的研究将重点开展自主导航与避障算法的研究与开发。一方面，研究基于视觉传感器的自主导航算法，通过图像识别和处理技术，实现无人机对周围环境的实时感知和路径规划。另一方面，探索多传感器融合的避障算法，将视觉传感器、毫米波雷达、激光雷达等传感器数据融合，实现对障碍物的全方位感知和精确测量。通过多传感器数据的协同处理，提高避障算法的可靠性和准确性^[39]。

在无人机的飞行控制和应用中，通信与数据传输技术是实现无人机与地面控制站、其他无人机之间信息交互的关键。未来的研究将重点关注通信与数据传输技术的改进与优化。一方面，研究高性能无线通信模块和协议，提高无人机与地面控制站之间的通信距离和数据传输速率^[40]。例如，采用 5G、Wi-Fi 6 等先进的无线通信技术，实现无人机的高速、稳定通信。另一方面，开发数据加密和安全传输技术，确保无人机通信数据的安全性和保密性。通过加密算法和安全协议，防止数据被窃取和篡改，保障无人机系统的安全运行。此外，研究无人机之间的协同通信技术，实现多架无人机之间的信息共享和协同作业。通过无人机之间的通信与协作，提高无人机群体的作业效率和任务完成能力。

致谢

在西安电子科技大学的本科生活即将结束之际，回顾这段充实而难忘的时光，我心中充满了感激之情。这段旅程中，有许多人对我的成长和学习给予了巨大的帮助和支持，我衷心感谢他们。

首先，我要感谢我的指导老师韩高老师。韩老师在学术上严谨认真，对学生关怀备至。从论文的选题到最终成稿，韩老师给予了我全方位的指导。他不仅耐心解答我在研究过程中遇到的各种问题，还鼓励我大胆尝试新的方法和思路。他的建议总是极具启发性，让我能够在学术探索的道路上不断前行。韩老师的悉心指导不仅提升了我的专业能力，更让我学会了如何以严谨的态度对待学术研究。我由衷地感谢韩老师在这段学习旅程中给予我的悉心帮助和无私支持。

其次，我要感谢我的朋友杜浩南和我的室友们。在他们的陪伴下我度过了快乐的四年大学时光。值得一提的是在考研的那段日子里，杜浩南给了我非常大的帮助，也让我有更多时间和精力去面对各种挑战，我最终也如愿成功上岸。他们的帮助和支持让我在大学中收获了珍贵的友谊和成长。

我还要特别感谢我的父母。他们是我成长道路上最坚实的依靠。从我踏入西安电子科技大学的那一刻起，父母就始终给予我无条件的支持和关爱。他们理解我在学业上的压力，尊重我的每一个选择，并且始终鼓励我追求自己的梦想。无论我遇到什么困难，他们总是耐心倾听，给予我最温暖的安慰和最坚定的支持。他们的爱让我在大学生活中始终充满安全感，也让我有勇气去面对各种挑战。父母的付出和关爱是我不断前行的动力源泉，我将永远铭记他们对我的养育之恩。

在西安电子科技大学的这几年，我收获了知识，收获了友谊，也收获了成长。感谢学校为我们提供了优质的教育资源和良好的学习环境，感谢每一位老师在课堂上的辛勤付出，感谢每一位同学在学习和生活上的陪伴与支持。这段本科生活是我人生中一段珍贵的经历，它将永远留在我的记忆中。

参考文献

- [1] 李若曦,陈杰,刘威.基于芯来蜂鸟 E203 处理器的架构优化[J].电子设计工程,2025,33(08):6-11+16.DOI:10.14022/j.issn1674-6236.2025.08.002.
- [2] Valence, L., Nadalini, A., Veeran, A. H. C., Sinigaglia, M., Sá, B., & Wistoff, N. (2024). A heterogeneous RISC-V based SoC for secure nano-UAV navigation. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 71(5), 2266-2279.
- [3] 邓雅静.AI 时代下兆易创新 MCU 的跃升之路[J].电器,2025,(03):10.
- [4] 李国斌.基于 STM32 和 HAL 库的嵌入式四轴无人机项目设计[J].信息与电脑,2024,36(24):20-23.
- [5] Kühne, J., Magno, M., & Benini, L. (2022). Parallelizing optical flow estimation on an ultra-low power RISC-V cluster for nano-UAV navigation. In 2022 IEEE International Symposium on Circuits and Systems (ISCAS) (pp. 1-5). IEEE. <https://doi.org/10.1109/ISCAS48785.2022.9937215>.
- [6] 刘东,王瑞锦,赵彦钧,等.基于 ARM 架构的边缘计算服务器关键平台研究[J].计算机科学,2024,51(S1):746-753.
- [7] 周志广,许孟孟,石梅林,等.基于 STM32F4 与 FreeRTOS 的无人机飞行控制系统[J].航空工程进展,2024,15(04):93-99.DOI:10.16615/j.cnki.1674-8190.2024.04.11.
- [8] 杨龙飞.基于 RISC-V 的 SoC 设计与实现[D].湖北大学,2024.DOI:10.27130/d.cnki.ghubu.2024.001891.
- [9] Nikiema P R, Palumbo A, Aasma A, et al. Towards Dependable RISC-V Cores for Edge Computing Devices[C]. In: Procee.
- [10] dings of 2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS), Crete, Greece, 2023: 03-05 July.
- [11] 马建业,许致火,汪月霞.基于四轴无人机的快递投递系统仿真[J].电讯技术,2024,64(08):125-1265.DOI:10.20079/j.issn.1001-893x.230509004.
- [12] 周成,孙健,赵天骐,等.基于 STM32 的小型机器人控制系统电路设计[J].机器人产业,2023,(04):72-77.DOI:10.19609/j.cnki.cn10-1324/tp.2023.04.006.
- [13] 刘鑫.面向 RISC-V 架构处理器的集成开发环境设计与实现[D].苏州大学,2023.DOI:10.27351/d.cnki.gszhu.2023.001253.
- [14] 贾思敏.RISC-V 处理器核设计优化与扩展指令集实现研究[D].山东大学,2023.DOI:10.27272/d.cnki.gshdu.2023.002851.
- [15] Bian S, Schulthess L, Rutishauser G, et al. ColibriUAV: An Ultra-Fast, Energy-Efficient Neuromorphic Edge Processing UAV-Platform with Event-Based and Frame-Based Cameras[C]. In: Proceedings of 2023 9th International Workshop on Advances in Sensors and Interfaces (IWASI), Monopoli (Bari), Italy, 2023: 08-09 June.

- [16] 李少童,何明兴,李大彬,等.基于 STM32F4 的四轴无人机设计[J].电子制作,2023,31(07):29-33.
DOI:10.16589/j.cnki.cn11-3571/tn.2023.07.014.
- [17] 李娇,向洲,陈华,等.基于 MPU6050 六轴陀螺仪的高精度动态水平姿态监测系统的开发与应
用[J].河南科技,2025,52(05):38-43.DOI:10.19968/j.cnki.hnkj.1003-5168.2025.05.007.
- [18] 熊赢超.四轴无人机的设计与优化[J].机械工程与自动化,2022,(05):124-125+128.
- [19] 张荣辉,严国峰.基于 STM32 四轴飞行器的设计与实现[J].数字技术与应用,2022,40(06):210-
212.DOI:10.19695/j.cnki.cn12-1369.2022.06.64.
- [20] 梁日杰.小型四轴无人机目标识别与跟踪技术研究[D].长春理工大学,2022.DOI:10.26977/d.
cnki.gccgc.2022.000581.
- [21] 唐陈辉.一种基于 RISC-V 架构的嵌入式 SoC 系统设计[D].西安电子科技大学,2022.DOI:10.
27389/d.cnki.gxadu.2022.000521.
- [22] 余宇宙.基于 PX4 的小型无人机导航技术开发[D].南京航空航天大学,2022.DOI:10.27239/d.
cnki.gnhhu.2022.000105.
- [23] 杨超琼,韦发清.STM32 的四轴飞行器的设计[J].电子世界,2022,(01):187-188.DOI:10.19353/j.
cnki.dzsj.2022.01.087.
- [24] 刘畅,武延军,吴敬征,等.RISC-V 指令集架构研究综述[J].软件学报,2021,32(12):3992-4024.D
OI:10.13328/j.cnki.jos.006490.
- [25] 顾兆军,陈辉,王家亮,等.一种小型四轴飞行器目标跟踪控制算法[J].西安电子科技大学学报,
2021,48(05):117-127+177.DOI:10.19665/j.issn1001-2400.2021.05.015.
- [26] 丁多斌,冯向萍,丁燕,等.基于 STM32 四轴无人机自主巡线设计[J].电子设计工程,2021,29(0
2):142-145.DOI:10.14022/j.issn1674-6236.2021.02.031.
- [27] 陈韩悦,刘晓航,王品智,等.基于 STM32 的小型四轴飞行器编队控制系统设计[J].现代计算
机,2021,(01):115-120.
- [28] 李想,李阳.四轴无人机在林业管理中的应用[J].广西林业科学,2020,49(02):296-299.DOI:10.
19692/j.cnki.gfs.2020.02.028.
- [29] 宋权霖,张伟,刘业钊,等.四轴飞行器系统设计与制作[J].电子制作,2020,(07):20-22.DOI:10.1
6589/j.cnki.cn11-3571/tn.2020.07.007.
- [30] 张治,谢峰,董敏周,等.基于 RaspberryPi 的飞控系统快速原型设计[J].兵器装备工程学报,202
0,41(09):182-186.
- [31] 王筱淇,刘震磊,李颖,等.四轴无人机总体结构模块化设计[J].中国科技信息,2020,(02):21-23.
- [32] 李成诚,乔东海.四轴无人机姿态解算的FPGA实现[C]//中国声学学会微声学分会,四川省声
学学会,上海市声学学会,山东省声学学会,浙江省声学学会.2019 中国西部声学学术交流
论文集.苏州大学电子信息学院;中国科学院声学研究所,;2019:362-364.DOI:10.26914/c.cnki
hy.2019.062463.
- [33] 肖猷坤,梁勇东,曹金梅,等.四轴无人机群协同控制系统的开发[J].机电信息,2019,(06):48-49.
DOI:10.19514/j.cnki.cn32-1628/tm.2019.06.028.

-
- [34] 张宏伟,钱叶册,许卫兵.基于 STM32 的微型四轴救援机的设计与实现[J].现代信息科技,2019,3(01):158-159.
- [35] 李晓英,刘敏.基于 RISCV-32I 的微处理器的设计与实现[J].内江科技,2018,39(11):49-50+35.
- [36] 王亭岭,芦杜洋,马跃涛.PIXHAWK 开源飞控的多旋翼无人机避障技术研究[J].单片机与嵌入式系统应用,2017,17(10):30-32.
- [37] 姜俊俊,李明,张宇.基于视觉的四旋翼无人机目标跟踪系统设计[J].中国科技信息,2016,(14):65-66.
- [38] 贺欢.世界微小型无人机最新发展应用概览[J].中国安防,2015,(15):82-95.
- [39] 冀亮,钱正洪,白茹.基于四元数的四轴无人机姿态的估计和控制[J].现代电子技术,2015,38(11):112-116.DOI:10.16652/j.issn.1004-373x.2015.11.004.
- [40] 王芳良,张伟功,于立新,等.基准集在嵌入式系统性能分析中的应用[J].计算机工程与设计,2015,36(01):115-119+126.DOI:10.16208/j.issn1000-7024.2015.01.021.

西安电子科技大学本科生毕业设计（论文）
空间科学与技术学院盲审意见书

专业名称	空间科学与技术
题 目	RISV 开源 IP 平台对于移植开源飞控的资源匹配性研究
评审专家审阅意见	
参照以下几个方面提出意见： （1）论文选题：理论意义、学术价值、实用价值和经济效益，是否接触学科前沿； （2）文献综述：对本领域文献资料是否有比较深入的了解； （3）研究内容和成果：是否有新见解、新思想，或应用新技术解决实际问题；工作量和工作难度如何； （4）能力水平：基础理论、专业知识、实际技能、设计能力和独立工作能力等方面的水平； （5）论文规范和撰写质量：论文写作是否科学规范，文字通顺，图表齐全，字数符合规定要求，提炼及文字组织表达能力如何； （6）外文翻译：表达正确，语句通顺，字符数达到要求。	
评阅意见(注：评阅意见里无需显示盲审结果，结果在系统里选择)： 本研究从多个维度对国产 RISC-V 的相关参数性能进行详细分析，并结合小四轴无人机的产品特性，构建了基于 RISC-V 架构的无人机飞控硬件平台，并最终通过实物形式展现。该同学前期调研充分，对无人机的硬件构成以及飞控系统核心模块理解深入，所开发的硬件平台在架构选择上具有创新性，为无人机飞控系统提供了新的技术方案。研究过程清晰、论证严谨，体现了作者扎实的专业基础和良好的工程实践能力。	

班级 2116029

学号 21009100863

本科毕业设计（论文）

外文资料翻译

毕业设计题目 RISC-V 开源 IP 平台对于移植开源飞控的资

源 匹 配 性 研 究

外文资料题目 RISC-V 是否准备好进入太空？从安全角度

探 讨

学 院 空间科学与技术学院

专 业 空间科学与技术

学 生 姓 名 何成楷

指导教师姓名 韩高

摘要

用于空间应用的集成电路通常具有低产量和高性能要求。因此，采用商用现货（COTS）组件和第三方知识产权内核（3PIPs）对于使系统设计、实现和部署在性能方面具有成本效益和可行性至关重要。然而，这种设计范式在设计阶段和运行阶段都会使系统面临多种安全威胁。在本文中，我们主要讨论与空间应用相关的安全问题，重点关注由于采用知名的 RISC-V 微处理器而带来的威胁。我们强调，硬件特洛伊木马（HTHs）和微架构侧信道攻击（MSCAs）可能会通过改变系统的正常行为或窃取机密信息来危及整个系统的运行。我们还讨论了 RISC-V 架构提供的安全扩展及其局限性。文章最后对 RISC-V 微处理器在空间领域安全性方面仍待解决的问题进行了概述。

关键词： 微架构侧信道攻击、微处理器、硬件安全、硬件特洛伊木马、RISC-V、空间应用。

第一章 引言

在空间系统中使用商用现货（COTS）组件、第三方知识产权内核（3PIPs）和商业计算机辅助设计（CAD）工具的想法正变得越来越流行。事实上，这种设计选择在集成电路产量极低的应用场景（如空间应用）中，代表了性能与成本之间的一个有趣的权衡。其基本思想是在空间系统中重用遗留的（即非专门为太空设计的）子系统、组件和 3PIPs，以同时受益于降低的开发成本和高性能。

当专注于微处理器时，设计者可以选择定义自己的指令集架构（ISA），或者使用已有的 ISA。尽管使用开源软件可以被视为一种遗留程序，但硬件尚未完全经历开放性带来的颠覆性影响。然而，在过去几年中，RISC-V 架构的流行度迅速上升，吸引了许多之前专注于其他开放和免费 ISA、专有 ISA，甚至是自行设计 ISA（其主要缺点是需要设计和维护一个软件生态系统）的大学和公司的关注。尽管这种设计范式可以降低成本，但它也使系统在设计阶段和运行阶段面临多种安全威胁。商业集成电路（ICs）的生产特点是全球化的供应链。这种全球化的供应链带来的好处是降低了设计成本和时间，但另一方面，这也导致了对最终 ICs 的信任度降低。其后果是很难确保/评估供应链中所有参与方的可信度。因此，产品面临多种威胁：晶圆厂可能会超量生产 ICs 并将其出售到黑市；有缺陷或被废弃的 ICs 可能会被当作合格品交付；IP 内核许可可能会被侵犯，IP 内核可能会被过度使用；设计可能会被恶意修改，以在最终产品中插入隐蔽的不受欢迎的功能，即硬件特洛伊木马（HTHs）。

微处理器系统面临的一种新型威胁是微架构攻击。这些攻击是侧信道攻击（SCA）的一个子类，即通过分析时序信息、功耗、热足迹或电磁辐射等信息来泄露系统中未经授权的信息的攻击。另一方面，微架构攻击不需要攻击者对被攻击系统进行物理访问。事实上，这种攻击仅依赖于对系统运行敏感应用程序时的时序行为的观察。这一类攻击的基本思想是，由于计算机架构针对处理速度进行了优化，因此处理数据、内存访问和执行时间之间存在很强的相关性：如果被攻击进程和攻击者进程共享相同的缓存空间，这种相关性可能成为一个可利用的侧信道。一个著名的微架构攻击案例是熔断（Meltdown），攻击者利用乱序执行破坏地址空间隔离，而无需利用任何软件漏洞。因此，通过利用熔断，攻击者允许自己的程序访问其他程序以及操作系统的内存（从而也可以访问秘密信息）。

在过去，硬件安全并未被航天工业视为问题，因为大多数攻击（例如故障攻击）都需要攻击者对被攻击系统进行物理访问，而航天应用显然不需要。然而，基于 MSCAs 和 HTHs 的攻击可以在没有对系统进行物理访问（在攻击发生时）的情况下成功实施。事实上，一旦植入，HTH 可以通过执行触发程序远程激活，同时 MSCA 可以通过强制执行与被攻击程序并行运行的恶意软件片段来实施。因此，如今在考虑用于航天应用的数字设计时，也应该解决硬件安全问题。

微处理器存在于星载数据处理系统和星载数据处理系统中，既有 ASICs/SoCs 中的硬核，也有 FPGAs 中的软核。微处理器是运行姿态确定与控制系统（ADCS）所需算法以及执行地面控制或基于算法的指令与数据处理（C&DH）的关键组件。方向对于卫星的任务目标至关重要（例如，错误的方向将使与地面站的通信变得不可能）。C&DH 的执行也至关重要，因为地面站与卫星之间存在主/代理关系。

在这篇前瞻性论文中，我们讨论了在空间应用中采用 RISC-V 平台的安全问题。我们首先介绍 RISC-V 架构本身提供的安全特性，并介绍其在防止和减轻硬件安全相关攻击的影响方面的局限性。事实上，专注于硬件安全，我们强调硬件特洛伊木马（HTHs）和软件威胁（特别是微架构侧信道攻击（MSCAs））如何通过改变其正常行为或窃取秘密信息来破坏整个系统的功能。最后，我们概述了来自研究的现有解决方案，旨在进一步确保 RISC-V 处理器的采用，并提出并讨论尚未解决的问题。本文旨在帮助技术人员找到最佳架构配置，以满足性能和安全要求。同时，这项工作也可以作为年轻研究人员和学生的起点，通过提供保护 RISC-V 的最新解决方案的讨论以及对未解决问题的概述。

本文的其余部分安排如下：第二部分描述空间系统中可能的威胁及其对不同航天任务的影响；第三部分讨论 RISC-V 架构提供的主要安全相关特性及其局限性；最后，第四部分总结全文，强调未解决的问题。

第二章 空间数据系统中的安全性

在空间嵌入式系统领域，我们正在见证一种新的趋势。尽管“传统”的处理 ASICs 仍然受到“地球”处理器与空间级处理器之间通常不断扩大的性能差距的困扰（如[19]中所讨论的），然而，最近在基于 SRAM 的 FPGA 方面的进展已经将地球上的“前沿”技术引入空间应用，为空间处理提供了可能的飞跃。特别是在有效载荷数据处理任务中，这些设备很可能会取代任何专用的应用特定标准产品（ASSP），并要求为特定应用提供复杂的处理器 IP 内核。

接下来，我们回顾咨询委员会为空间数据系统（CCSDS）在[20]中强调的主要与空间任务相关的安全问题。

1) 有意数据篡改

攻击者可能会在其源头故意篡改数据（例如，通过干扰传感器），或者由靠近被攻击卫星的攻击卫星进行操作[18]。另一种可能性是，在数据从仪器传输到地面系统的过程中，数据被篡改，例如通过执行恶意软件。除了仪器信息的损坏外，故意的数据篡改可能会导致系统的灾难性损失，例如，如果在接收到命令时，航天器没有采取行动或采取了错误的行动。例如，如果导航机动命令被篡改，航天器最终可能会进入不可用轨道，错过与目标的相遇，甚至被摧毁。

2) 软件威胁

编码错误可能会发生并导致安全问题[20]。此外，系统操作员可能会错误配置系统，从而导致安全漏洞。另一方面，程序员可能会引入逻辑或实现错误，从而导致系统漏洞。最后，外部实体可能会试图利用漏洞注入软件或更改配置。其结果可能是数据丢失、航天器失控、未经授权的航天器控制或任务失败。

软件威胁（STs）也可能来自硬件和架构级别的漏洞利用。例如，高性能处理器可能会采用推测执行和乱序执行。这些特性可能会被恶意利用，以访问受保护的处理器内存位置，从而为所谓的微架构侧信道攻击（MSCAs）[12]打开大门。这种类型的 SCA 特别关键（对于空间应用也是如此），因为它不需要对被攻击系统进行物理访问，而是依赖于对攻击本身感兴趣的特征的监测，例如，被攻击处理器的时序行为、性能计数器等。一些 MSCA 的例子包括熔断（Meltdown）[17]、幽灵（Spectre）[21]和预兆（Foreshadow）[22]，攻击者利用乱序和推测执行破坏地址空间隔离，而无需利用任何软件漏洞。因此，这些攻击允许攻击者访

问未经授权的内存区域并窃取信息。结果是，使用可信组件构建的系统可能会被成功攻击。

3) 恶意硬件

硬件信任问题源于设计系统生命周期中涉及的不可信实体，包括不可信的 IP 提供商、设计团队组件、CAD 工具以及制造、测试和分发设施[23]。当使用 COTS 组件时，恶意硬件可能会进入系统。这些组件通常没有可信的供应链，并被部署到大众市场。事实上，COTS 组件的供应链由许多（可能不可信的）实体组成[23]。

为了进行恶意活动而对现有电路元件进行的不希望的添加或修改被称为硬件特洛伊木马（HT）。HT 可以改变电路的功能、降低其可靠性或泄露有价值的信息[23]。已知软件可利用的 HT 可能会被集成到微处理器中。借助 HT，攻击者可能能够执行自己的恶意软件、修改正在运行的软件或获取 root 权限[24]、[25]、[26]；或者他们可能能够通过软件激活的后门进入特权模式[27]。最后，必须提到的是，最新出现的攻击向量是所使用的 CAD 工具。事实上，已经证明 HTHs 可以通过用于高级和逻辑综合、布局布线以及比特流生成的工具被引入到设计电路中[28]、[29]。

安全相关威胁的影响高度依赖于任务轨道和系统所执行的任务类型。接下来，我们讨论这两点。

1) 任务轨道

一方面，地球静止轨道（GEO）卫星比低地球轨道（LEO）任务更安全，因为它们需要具有更高发射功率和更大天线的地面站。当然，这使得与卫星的通信更加昂贵，从而限制了可能的攻击者数量。另一方面，GEO 卫星比 LEO 卫星更容易受到攻击，因为它们在一定地理区域持续可见，而 LEO 卫星仅在特定时间段内从地面站可见。此外，卫星星座增加了整个空间系统的脆弱性，因为在这种情况下，轨道上有许多卫星，从而增加了从地球上特定点的可见窗口。最后，深空/行星际任务需要比 GEO 卫星更大的天线和更高的功率来进行攻击[20]。

2) 任务类型

空间系统所执行的任务类型以及其安全关键性或任务关键性，高度影响安全相关威胁的影响和可能性。接下来，我们讨论几种类型的空间任务及其相关的安全问题。

地球观测卫星：这些卫星可以是科学用途的，也可以是政府的关键资产（例如，气象预测应用）。在后一种情况下，最可能的威胁是未经授权的访问，通常通过加密命令和任务数据以及认证命令来应对。此外，恶意硬件是可能的威胁，可以通过分析硬件功能或谨慎选择可信供应商来缓解。数据损坏也可能是一个问题，向最终用户提供错误信息。

通信卫星：这些卫星旨在提供通常具有非常高可用性的服务。例如，在 GEO 通信卫星的情况下，整个空间系统预计 99.9% 的时间内提供服务[30]。因此，这种类型的卫星特别容易受到攻击：事实上，即使可用性略有降低，也可能对服务产生巨大影响。对通信卫星的攻击案例在[18]中有所报道：2003 年，通过一个上行站向 Telesat 的 12 颗卫星发送矛盾的频率，覆盖了信号，有效地阻断了电视节目。

导航卫星：这些通常是双用途卫星，通常属于政府的关键基础设施。一个例子是 GPS，它被个人、公司和军事机构使用。与通信卫星一样，导航卫星系统的丢失可能导致潜在的生命损失、个人安全和关键基础设施的损失。

科学任务：这些任务通常不属于关键基础设施的一部分。尽管它们也暴露于与其他任务相同的威胁，但影响要小得多，对任务本身所涉及的实体的财务投资和声誉造成损害[20]。然而，投资可能非常高：例如，最近发射的詹姆斯·韦伯太空望远镜的成本达到了 100 亿美元[31]。

载人任务：从安全角度来看，这些任务当然是最关键的。事实上，在这种情况下，攻击导致航天器丢失的后果可能是灾难性的，可能会对机组人员的生命造成威胁。

第三章 RISC-V 的安全性考量

在本节中，我们介绍 RISC-V 架构，并讨论其现有的与安全相关的特性；最后，我们对现有的局限性以及一些提议的安全扩展进行综述。

A. RISC-V 指令集架构

RISC-V 最初由加州大学伯克利分校开发，旨在支持面向硬件实现的计算机架构研究和教育[32]。这种开放且免费的指令集架构（ISA）的传播，已经为地球应用（例如安全、人工智能等）开辟了广阔的科研领域。事实上，访问专有架构成本高昂，且限制了对某些产品或特定科研活动的使用范围。一个由大型开源社区支持的软件生态系统，激发了前所未有的开发活动，许多开源实现被宣布和

/或发布。因此，采用一种流行的免费和开放的 ISA 可以通过重用实现更快的上市时间和更低的成本。

RISC-V ISA 规范的主要特点是其模块化，这使得它能够覆盖广泛的应用场景，并增加软件的可重用性（通过将新指令作为可选扩展添加，而不是发布整个 ISA 的新版本）。RISC-V 允许使用标准和非标准扩展（在规范之外定义）。RISC-V 的用户级 ISA 定义为一个基础整数（I）ISA，它必须出现在任何实现中，以及对基础 ISA 的可选扩展。当实现针对小型 32 位微控制器时，可以选择实现基础整数的一个子集（E），使用 16 个通用寄存器而不是 32 个。标准定义了一个“通用”子集（G）以及通用计算系统所需的扩展集合。

B. 内存隔离

在软件层面保护系统安全的主要特性是内存隔离（即每个进程都有其自己的地址空间，从而在程序之间隔离内存）[33]。隔离意味着即使一个分区的安全性被破坏，攻击者也无法侵犯其他分区的安全性。Linux 内核提供了内核服务与用户空间应用程序之间的分离。然而，这种方式使得内核在安全性方面成为单点故障，因此它可能负责大量漏洞，鉴于其巨大的攻击面[34]。

内存隔离可以通过使用虚拟机管理程序（hypervisors）来进一步改进，以在多核处理器上运行的不同操作系统实例之间提供隔离。目前，内存隔离正在被研究用于空间应用，以改进安全行为，即避免一个操作系统上运行的应用程序的故障可能影响在同一处理器上另一个操作系统中运行的另一个应用程序的执行[35]。鉴于软件系统的复杂性不断增加，即使在空间应用中，设计者也越来越多地依赖第三方软件组件，通常以软件库的形式提供。当这些库的代码是开源的时，可以进行安全分析和验证。另一方面，专有软件通常以可链接的（且不可检查的）对象模块形式提供。因此，通常需要在系统内强制分离各种软件组件[34]。

C. RISC-V 软件栈和特权级别

RISC-V 的模块化特性允许实现不同安全级别的不同软件栈。例如，图 1 展示了在 RISC-V 特权规范[36]中描述的软件栈。在图的左侧，展示了一个简单的软件栈实现。应用程序通过应用程序二进制接口（ABI）与应用程序执行环境（AEE）进行交互。在这种简单情况下，ABI 由实现的用户级 ISA 子集组成。在图的中央，多个应用程序运行并通过 ABI 与操作系统（OS）通信（后者在此案例中提供 AEE）。反过来，操作系统通过一个监督二进制接口（SBI）与一个

监督执行环境（SEE）通信。SBI 包括用户级和监督级 ISA，以及一组 SBI 函数调用。SEE 可以是一个低端硬件平台上的简单引导程序和 BIOS 风格的输入/输出系统，也可以是一个高端服务器中的虚拟机，或者是一个架构仿真环境中的主机操作系统的薄翻译层。在图的右侧，RISC-V 运行一个虚拟机监视器配置，其中多个操作系统运行在虚拟机管理程序之上。每个操作系统通过 SBI 与虚拟机管理程序通信，后者反过来提供 SEE。虚拟机管理程序通过一个虚拟机管理程序二进制接口（HBI）与虚拟机管理程序执行环境（HEE）通信，以将虚拟机管理程序与硬件平台隔离。

特权级别用于在上述软件栈的不同组件之间提供保护。线程可能具有的可能级别包括：

机器模式（M 模式）：具有最高特权级别，且是唯一强制性的模式。运行在机器模式下的代码必须完全可信，因为它可以访问机器实现。

监督模式：被添加以在操作系统和 SEE 之间提供隔离。

用户模式：专用于用户应用程序。

特权级别被设计和检查，以便任何尝试执行当前模式未允许的操作的尝试都将引发异常。任何特定的 RISC-V 实现都可以选择实现哪些模式（除了强制性的 M 模式）。这允许硬件架构师在提供的安全级别和实现成本之间找到合适的权衡[36]。典型解决方案包括：

M：简单嵌入式系统，安全性不是问题。这种解决方案不会对错误或恶意应用程序代码提供任何保护。

M、U：安全嵌入式系统。

M、S、U：运行类 Unix 操作系统的系统。添加监督模式以在操作系统和 SEE 之间提供隔离。

用户线程通常在用户模式下运行应用程序代码，直到陷阱（例如，监督调用或定时器中断）强制切换到陷阱处理程序，该处理程序通常在更高级别的模式下运行。陷阱处理程序将被执行，最终用户线程将在原始陷阱指令处或之后恢复执行，再次处于用户模式。

除了不同的特权模式外，RISC-V ISA 还提供了额外的扩展以增强安全性。

1) 物理内存保护

M 模式支持一个可选的标准扩展，用于物理内存保护（PMP）。PMP 扩展定义

了控制寄存器，用于指定每个物理内存区域的访问权限（读、写、执行）。尝试从没有执行权限的 PMP 区域获取指令，或者从没有读权限的物理内存位置加载数据，将引发异常。

使用 PMP 提供了许多安全特性。例如，线程被阻止修改甚至读取共享库的数据。因此，可以使用 PMP 将共享库数据所在的内存区域设置为仅执行。即使 PMP 主要用于防止低特权级别（例如 U 和 S 模式）运行的线程访问特权内存内容，“锁定”功能即使仅实现 M 模式时也能提供保护。事实上，如果锁定 PMP 条目，则对该条目的配置寄存器和相关地址寄存器的写操作将被忽略。实际上，PMP 限制对 M 模式也有效：锁定的 PMP 条目只能通过系统复位解锁。

2) 密码学扩展

航天器安全的基本要求是使用 AES 加密标准对遥测和遥控信号进行加密[37]。密码学扩展的实现提高了性能，并将实现细节隐藏在软件之外，从而减少了攻击面[38]。然而，通常更倾向于在单独的硬件组件或板卡上实现遥测和指令的加密和解密，因为要求密钥不能从软件访问。

3) 用户级中断扩展 (N)

此扩展允许具有不同特权级别的中断处理程序之间实现隔离。事实上，中断处理程序可以在用户级别执行，因此无法破坏隔离模型[34]。

大多数针对隔离的攻击利用对处理器的物理访问来执行故障注入 (FI) [39]。在空间应用中，这可能比移动和物联网 (IoT) 等应用更复杂，尽管从原则上仍然可能。例如，在[39]中展示了如何利用每条指令的功耗知识，在上下文切换期间跳过一条指令，从而允许访问受 PMP 保护的内存部分。另一方面，前面提到的攻击（即 MSCAs 和 HTHs）可以在攻击者在攻击时没有任何对被攻击系统的物理访问的情况下成功实施。事实上，HTHs 可以在设计阶段由参与设计的许多不可信方之一植入。一旦植入，HTH 可以通过执行触发程序“轻松”激活。同样，MSCA 是与被攻击软件并行运行的恶意软件。因此，MSCAs 和 HTHs 也必须被视为空间场景中的严重威胁，因为在空间场景中，攻击者几乎不可能对系统进行物理访问。

尽管 RISC-V 架构中集成的原生安全特性无法保护系统免受 MSCAs 和 HTHs 的攻击，但我们认为应该将来自研究的创新安全解决方案集成到 RISC-V 架构中，以使采用这种微处理器的空间任务安全且可信。

最近，针对基于英特尔和 ARM 处理器的系统，提出了一些旨在抵御 MSCAs 和 HTHs 的系统级方法。特别是，新趋势旨在在由不可信组件构建的系统上提供安全且可信的程序执行。这些方法背后的大部分思想是引入硬件安全检查器（HSCs），能够监控取指活动、处理的数据

第四章 讨论与待解决的问题

随着在空间应用中部署商用微处理器的兴趣不断增加，以及新型硬件导向威胁（例如微架构侧信道攻击（MSCAs）和硬件特洛伊木马（HTHs））的出现，安全性（尤其是硬件安全性）成为一个开放且严重的问题。

正如我们在本文中所讨论的，RISC-V 微处理器已经实现了许多安全特性。事实上，物理内存保护（PMP）、密码学扩展和用户级中断等特性已经能够保护基于 RISC-V 的系统免受多种攻击，例如数据篡改和未经授权的访问。然而，RISC-V 处理器对 MSCAs 和 HTHs 的防护仍然是一个亟待解决的问题。

除了对 MSCAs 和 HTHs 的防护之外，我们还看到与在空间任务中可信地采用 RISC-V 处理器相关的两个主要待解决问题。第一个问题是缺乏可信且稳健的工具链：正如前面几节中已经讨论的那样，已经证明所使用的计算机辅助设计（CAD）工具可能会在设计系统中引入恶意修改。因此，可信工具的可用性以及确保所使用工具可信度的方法至关重要。

第二个主要问题是将第三方知识产权内核（3PIPs）集成到基于 RISC-V 的系统中。专用硬件加速器与主处理器之间的可信交互对于确保整个系统的安全性至关重要。因此，需要研究验证可信行为的方法以及硬件级机制（例如基于硬件的防火墙），以隔离不可信组件并限制其潜在危害。

第五章 总结

RISC-V 微处理器在空间应用中的采用面临着来自硬件安全的多重挑战。尽管其已经具备了一些安全特性，但在应对 MSCAs 和 HTHs 等新型威胁时仍显得力不从心。此外，建立可信的工具链以及确保 3PIPs 的安全集成，是实现 RISC-V 在空间领域可信应用的两个关键问题。这些问题的解决将有助于推动 RISC-V 在空间任务中的广泛应用，并确保其安全性。

面向边缘计算设备的可靠 RISC-V 内核

摘要

将计算从云端迁移到边缘设备（即物联网设备）可以减少延迟和流入网络的数据量。随着开源且可定制的 RISC-V 指令集架构（ISA）的兴起，基于此类 ISA 的内核是物联网家族中多个应用领域的有前途的候选方案，例如汽车、无人机（UAV）、工业自动化、医疗保健、农业等领域，在这些领域中，功耗、实时执行、安全性和可靠性至关重要。在这个新兴的连接 RISC-V 物联网设备的新时代，需要机制来实现可靠和安全的执行，同时满足边缘设备的面积、功耗和计算时间约束。我们提出了三种机制以实现这一目标，即：（i）用于后量子安全启动的信任根模块，（ii）用于防范硬件特洛伊木马和微架构侧信道攻击的硬件检查器，以及（iii）用于实时错误检测和纠正的细粒度双核锁步机制。本文阐述了所提出的机制及其相关动机和影响，并讨论了未来研究方向。

关键词： RISC-V, OpenTitan, 信任根（RoT），安全启动，后量子密码学（PQC），安全，安全检查器，硬件特洛伊木马，微架构侧信道攻击。

I. 引言

为了减少延迟以及流入网络的数据量，将计算从云端迁移到边缘设备（即物联网设备）成为了一种必然趋势。随着开源且可定制的 RISC-V 指令集架构（ISA）的兴起，RISC-V 越来越受到行业的关注[1]。RISC-V 是物联网家族中多个应用领域的有前途的候选方案，例如汽车、无人机（UAV）、工业自动化、医疗保健、农业等领域，在这些领域中，功耗、实时执行、安全性和可靠性至关重要[2][3]。然而，目前大多数关于 RISC-V 的研究主要集中在性能和功耗设计上，往往忽视了可靠性和安全性问题[4]。在这个新兴的连接 RISC-V 物联网设备的新时代，需要机制来实现可靠和安全的执行，同时满足边缘设备的面积、功耗和计算时间约束[5]。

为了实现这一目标，这些机制需要覆盖完整的系统堆栈。整个可靠性和安全性堆栈的基础机制是安全启动，其中执行的代码需要经过身份验证并验证其完整性。系统启动后，系统执行需要受到保护，防止恶意或非恶意注入的故障。这些故障可能会影响指令和数据从内存到处理器的获取，甚至可能影响处理器的执行。

在本工作中，我们提出了三种在系统堆栈不同层面上应用的机制，以实现可靠的 RISC-V 内核。首先，需要通过安全启动方案建立强大的可靠性基础，该方案通过信任根（RoT）模块（如 OpenTitan）与硬件绑定，使用基于公钥密码学（PKC）算法（如 RSA 和 ECC）的数字签名。然而，量子计算技术的出现使得基于整数分解和离散对数的密码学（如 RSA 和 ECC）分别变得可破解[6]。因此，需要适合后量子时代的数据、代码完整性和认证解决方案。然而，由于资源限制，OpenTitan 无法在安全启动流程中支持现有的后量子密码学（PQC）算法，除非进行重大修改。为了克服这一限制，我们提出了 OpenTitan RoT 的增强型安全启动流程，通过修改 OpenTitan 的硬件和软件架构，实现后量子安全。

其次，为了保护从内存到处理器的数据传输过程，需要防止执行未经授权的软件和访问非法内存区域。为此，我们提出了硬件机制来监控指令获取活动、处理的数据以及微处理器的状态，以检测潜在的可疑活动，例如硬件特洛伊木马的激活或微架构侧信道攻击的尝试。

最后，为了应对处理器内部发生的各种硬件故障（例如与制造缺陷、老化或

辐射诱导相关的故障），通常使用基于冗余的机制：在不同的处理器上执行相同的操作集和相同的输入。这种方法非常有效，因为所有处理器同时出现相同故障的概率非常小[7]。然而，当以粗粒度级别应用时，其错误检测和纠正成本较高。因此，我们提出了一个细粒度锁步版本的 RISC-V 处理器，具有快速错误检测和纠正功能。我们使用高级综合（HLS）设计了这种方法。与硬件描述语言（HDL）实现相比，HLS 使设计过程更加简单，因为处理器模型可以轻松修改、扩展和验证[8]。

本文的其余部分安排如下：第 II 节介绍后量子安全启动机制；第 III 节介绍用于防范特洛伊木马和侧信道攻击的硬件安全检查器；第 IV 节讨论用于错误检测和纠正的双核锁步实现；最后，第 V 节总结本文。

II. OpenTitan RoT 上的后量子安全启动

OpenTitan 是一个基于 RISC-V 的开源信任根（RoT），其中一个用例是通过安全启动程序建立信任链的基础。如前所述，OpenTitan 当前的安全启动使用的是不具有后量子安全性的公钥密码学（PKC）方案，因此我们的目标是为 OpenTitan 选择最佳的候选算法。

目前的后量子密码学（PQC）数学解决方案基于哈希、编码、多变量和格理论[9]。考虑到 RISC-V 边缘设备的资源受限环境，PQC 安全启动算法的实现需要满足面积、功耗以及计算时间的要求。第一个基于后量子哈希签名方案的 PQC 硬件安全启动实现已经与硬件和软件 ECDSA 安全启动解决方案进行了比较[10]。基于 LMS 和 SPHINCS+ 算法的 PQC 安全启动签名在认证时间和启动时间以及内存占用方面表现出色[11]，这使得这些算法非常适合资源受限的边缘设备。

在[12]中，对在受限嵌入式设备上实现的基于格的密码学进行了调查，并与传统公钥方案进行了比较。结果显示，随着密钥尺寸的增加，基于格的密码学方案的性能会下降。在 ARM Cortex-M4 微处理器上测试不同基于格的方案的性能时，CRYSTALS-Dilithium 相比其他方案具有最高的吞吐量性能。

CRYSTALS-Dilithium 也是美国国家标准与技术研究院（NIST）后量子密码学标准化项目第三轮的两个签名决赛选手之一[13]，这使其成为 OpenTitan 的理想实现候选方案。然而，这种安全方案意味着复杂的软件实现，由于边缘设备的 ROM 尺寸限制，将其引入物联网设备的初始启动阶段并非易事。

为了克服这一限制并提供抗后量子攻击的启动方案，我们将 OpenTitan 的 RTL（寄存器传输级）扩展以支持 CRYSTALS-Dilithium 加速。我们的设计中使用了 CRYSTALS-Dilithium 硬件原语[14]，以验证启动阶段的签名。

提出的 OpenTitan 的 PQC 安全启动确保只有经过验证的有效载荷才会在系统启动时执行。请注意，本文讨论的安全启动流程不包括 BL0 固件的后续启动阶段以及支持的操作系统内核的启动。启动过程的第一阶段位于启动 ROM 中。ROM 代码在制造过程中被编程到芯片的 ROM 中，且不可更改。它包含一组用于验证存储在闪存中的第一启动阶段（即 ROM 扩展）的公钥。

启动 ROM 代码的主要任务是为执行 ROM 扩展做好准备，同时确保加载的 ROM 扩展被允许在此芯片上执行（ROM 扩展存储在闪存中，并由芯片制造商签名）。图 1 展示了启动代码的两个步骤的概览。

一旦系统上电或复位，启动过程就开始了。执行被限制在 OpenTitan 内部的片上 ROM 中。复位逻辑最初会阻止对其他类型/区域内存的执行。执行进入 ROM 代码，并启用并配置 OpenTitan 的 Ibex 核心的增强型物理内存保护（ePMP）。在这个阶段，只有存储可执行启动代码的 ROM 区域对核心是可访问的。除了保持型 SRAM 外，所有 SRAM 都被清零。从闪存启动信息中加载活动启动槽。闪存启动信息位于闪存的默认银行中。从活动 ROM 扩展槽开始，ROM 代码执行以下步骤：

- 通过检测头部魔数值是否存在，判断活动 ROM 扩展槽是否为空。如果存在，则继续验证 ROM 扩展；如果不存在，则执行启动失败逻辑。
- 如果活动 ROM 扩展槽为空，启动过程读取 ROM 扩展代码并计算其 SHA2-256 摘要。
- 使用 CRYSTALS-Dilithium 硬件原语，通过摘要和存储在 ROM 中的芯片制造商公钥，验证 ROM 扩展清单中的签名。如果验证成功，则继续；如果失败，则执行启动失败逻辑。
- 之后，它执行系统状态测量，并推导出将存储在密钥管理器中的 CreatorRootKey 身份，作为中间状态。
- 接下来，它通过配置适当的 PMP 条目来启用 ROM 扩展的执行。
- 最后，它将执行转移到 ROM 扩展清单中指定的活动槽的入口点。
- 执行跳转到闪存，这是执行就地（XIP）的，并进入 ROM 扩展代码部分。

- ROM 扩展读取启动信息页，以确定它应该启动哪个 BL0/内核映像。
- ROM 扩展读取 BL0 并计算其 SHA2-256 摘要。
- 使用 CRYSTALS-Dilithium 硬件原语，通过摘要和从密钥管理器（KM）派生函数派生的芯片所有者公钥，验证 BL0 清单中的签名。如果验证成功，则继续；如果失败，则执行启动失败逻辑。
- 执行被转移到 BL0/内核映像清单中的入口点。
- 执行进入芯片所有者代码（即 BL0、内核应用程序和数据，这些内容不在本文的讨论范围内）。

我们将提出的方法集成到原始 OpenTitan 流水线中，并在 Xilinx Virtex UltraScale+ VCU118 FPGA 板上进行了评估。FPGA 实现结果如表 I 所示。所提出的 Dilithium 硬件原语占整个 OpenTitan 设计的 33%，因为这种实现是针对高性能进行优化的。为了避免同步问题，我们不得不将其时钟频率降低到 OpenTitan 频率的 1/4。为了降低上述成本，作为未来的工作，我们将在下一个硬件原语设计中权衡性能以换取面积占用的减少，并实现硬件加速器解决方案以卸载 Dilithium 算法的格计算。研究仅将算法中最耗时的部分（即随机生成和多项式乘法操作）卸载到硬件中，并与纯硬件实现进行比较。

III. 硬件安全检查器

RISC-V 架构本身具备多种安全扩展功能，例如特权级别、物理内存保护和密码学功能[15]。然而，这些原生机制并不能保护微处理器免受两大日益严重的威胁：硬件特洛伊木马（Hardware Trojan Horses, HTHs）和微架构侧信道攻击（Microarchitectural Side-Channel Attacks, MSCAs）[16]。

硬件特洛伊木马（HTHs）长期以来一直被认为是一个纯学术问题，因为它们通常表现出的复杂性和危险性都很有限。然而，最近出现了一种新的威胁，使得 HTHs 也成为工业界的关注点：可被软件利用的 HTHs[17]。复杂且高度危险的 HTHs 可能会感染实现微处理器的 IP 核：这些 HTHs 可能允许攻击者执行恶意软件、修改正在运行的软件、获取未经授权的特权或窃取秘密信息[18]。最近在现实世界中发现的一个 HTH 是 Rosenbridge 后门，它存在于一款商业化的 Via Technologies C3 处理器中[19]。这种 HTH 可以通过软件激活和利用，以进入系统的特权模式。尽管已经提出了许多电路级设计阶段的 HTH 检测技术[20]–[22]，但越来越多的研究兴趣集中在系统级技术上，这些技术允许使用不可

信组件构建可信系统[23]、[24]，或者在（部分）不可信的系统上实现可信软件执行[25]、[26]。我们认为，现有的解决方案没有考虑到那些通过使 CPU 执行正常（但意外的）指令而不改变特权模式来改变系统功能的 HTHs。换句话说，这些研究都没有检查微处理器是否正在执行未经授权的软件，以及是否正在访问非法内存地址。

近年来出现的另一种威胁是微架构侧信道攻击（MSCAs），例如 Spectre 和 Meltdown[27]。这些攻击允许攻击者在不需要对被攻击系统进行物理访问的情况下窃取未经授权的信息。事实上，这些攻击仅依赖于对合法硬件功能（例如推测执行和分支预测）的利用，以及对系统在运行敏感应用程序时的时序行为的观察。近年来已经提出了许多针对 MSCAs 的对策[28]。常数时间技术依赖于使执行时间保持恒定以保护秘密信息；编译时技术被提出用于识别和修改那些可能为 MSCAs 打开大门的指令序列；基于操作系统级别的解决方案，例如缓存分区和定期缓存刷新，也被提出。所有这些技术都存在适用范围有限和显著性能下降的问题。最后，许多利用机器学习和硬件性能计数器观察的技术也存在。这些技术能够实现非常高的检测精度，但它们都需要多线程支持，或者需要一个额外的（可信且未被攻击的）核心来运行检测引擎。因此，这些技术适用于高性能计算系统（例如高端服务器），但不适合边缘设备和低端系统（例如智能卡或汽车嵌入式系统），在这些系统中，只有一个核心可用，操作系统要么不存在，要么极其简单。

我们的解决方案（其高级表示如图 2 所示）旨在允许系统集成商通过购买可能不可信的组件来构建可信系统。因此，我们将 IP 提供商视为攻击者；而系统集成员工和代工厂则被视为可信方。我们在架构中引入了硬件安全检查器

（Hardware Security Checkers, HSCs），使得 HSCs 能够监控指令获取活动、处理的数据以及微处理器的状态，以检测潜在的可疑活动，即 HTHs 和/或 MSCAs 的激活。这些 HSCs 在两个阶段工作：在第一个设计阶段，它们学习哪些指令/指令序列是合法的；在随后的运行时监控阶段，它们检查微处理器的活动是否与之前学习到的合法配置文件一致。

1、硬件特洛伊木马的检测

在[29]中，我们提出了一种基于布隆过滤器（Bloom filter）的 HSC，用

于保护微处理器免受主内存中的 HTHs 攻击。在软件安装到指令存储器时, HSC 通过存储有关合法指令及其对应内存地址的信息进行配置。在运行时, HSC 监控微处理器访问的内存地址以及对应的获取指令, 并根据这些信息查询布隆过滤器。如果访问的内存地址和/或对应的获取指令不合法, HSC 将发出警报。在[30]中提出了一个更轻量级的 HSC 版本。

我们将提出的抗 HTH 安全检查器集成到 PULPINO 平台的 RI5CY 版本中, 这是一个小型的 4 级 RISC-V 核心。在 Xilinx Artix XC7A35T 上综合时, RI5CY 需要 15097 个查找表 (LUTs) 和 9881 个触发器 (FFs), 工作频率约为 50MHz, 总功耗为 127mW。我们考虑了以下基准测试: 二分查找 (BinS)、矩阵乘法 (MM)、冒泡排序 (BubS)、快速排序 (QS)、数独求解器 (SS) 和运动检测 (MD)。我们模拟了 10,000 个案例, 其中 HTH 强制从意外内存地址获取恶意指令, 以及 10,000 个案例, 其中 HTH 修改从合法地址读取的获取指令。表 II 报告了两种攻击场景下的误报 (虚假警报) 率和漏报 (未检测到的攻击) 率。可以看出, 我们始终保持着 0% 的误报率, 平均只有大约 2% 的攻击未被检测到, 但仅当 HTH 修改获取的指令时。关于开销, 我们引入了大约 10% 的面积增加, 大约 2% 的功耗增加, 且没有工作频率的降低。

2、微架构侧信道攻击的检测

在[31]中提出了一个与之前介绍的方法类似但用于识别 MSCAs 激活的方法。我们利用 Count-Min Sketch 模型在运行时检测预定义且可重现的可疑指令模式集的执行, 这些模式代表特定的 MSCAs。这些指令模式 (每个感兴趣的攻击一个) 由安全工程师在设计阶段仔细识别, 然后用于配置 HSC。因此, 我们的提议能够并行检查多种微架构攻击, 从而代表了一种灵活且可扩展的安全解决方案。

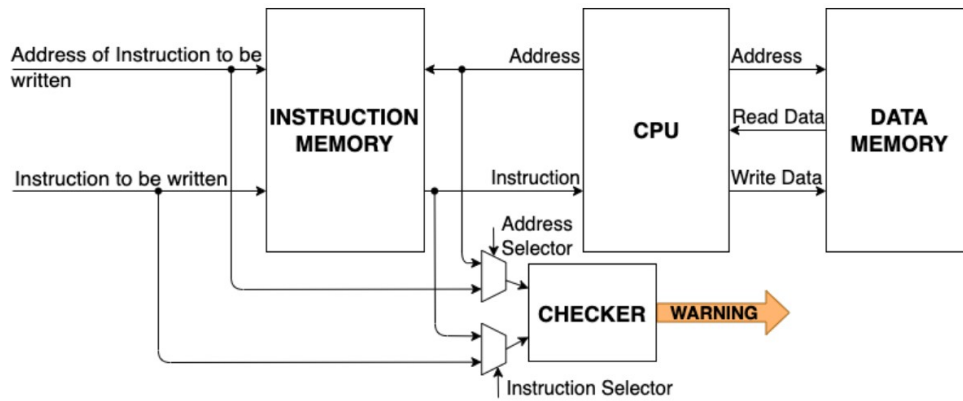


图 1 安全架构

我们将 HSC 集成到 RSD 核心中，这是一个 32 位、推测性乱序、超标量、双获取前端和五发布后端流水线的 RISC-V 核心，具有 16 KB 指令缓存。我们在 Virtex7 FPGA 上实现了目标微处理器，使用了 18334 个查找表（LUTs）、10885 个触发器（FFs）、4512 个 LUTRAM 单元和 17 个 BRAM 单元，工作频率为 57MHz，估计功耗约为 0.926W。我们考虑了 Coremark、Towers、RSort 和 Median 作为受攻击的良性程序，以及 Spectre、Orchestration、Flush+Reload 和 Rowhammer 作为 MSCAs。我们为每个良性程序与攻击程序运行了 10,000 次实验，结果是 100% 的攻击都被检测到。然后，我们测量了引入的 HSC 产生的误报数量：根据 HSC 的配置，误报率从大约 80% 降至 0%。允许我们同时实现 0% 误报率和漏报率的最小 HSC 配置引入了大约 10% 的面积开销，大约 4% 的功耗增加，且没有工作频率的降低。

IV. 锁步双核

处理器冗余可以通过非侵入式或侵入式方式实现，使用同构或异构处理器。非侵入式方法在核心级别引入冗余，而不修改内部处理器微架构，因此通常用于内部微架构细节不可用或难以修改的情况，例如商业现货（Commercial Off-The-Shelf, COTS）处理器。例如，使用两个相同的 MicroBlaze 软核实现同构双核锁步（Dual Core LockStep, DCLS）[32]，其中故障处理器被排除在计算之外，并通过重新配置进行修复，而正确的处理器继续工作。异构非侵入式 DCLS 使用一个 RISC-V 软核和一个 ARM A9 硬核[33]。应用中使用检查点来检测两个核心之间的不匹配，并在检测到不匹配时采用回滚策略。异构方法可能会降低

性能，低速处理器的性能将成为 DCLS 的上限。此外，使用硬核进行锁步需要特定的架构支持，而并非所有处理器都具备这种支持[33]。总体而言，非侵入式方法实现简单，因为无需对微架构进行修改。然而，它们将定义纠正策略（例如通过定期检查点）的责任留给应用层面，可能导致较高的时序开销。

另一方面，侵入式方法修改内部处理器微架构，在实现纠正机制（例如回滚）时更具灵活性。例如，动态自适应冗余架构（Dynamic Adaptive Redundancy Architecture, DARA）是一种应用于 RISC ISA SH-2 处理器的同构侵入式 DCLS 方法，通过回滚实现错误纠正[34]。DARA 添加了额外的硬件来检查锁步核心之间所有流水线阶段寄存器的一致性。然而，DARA 不支持分支指令中发生的故障。另一种同构侵入式方法使用两个虚拟 RISC-V 核心，通过细粒度交错多任务实现 DCLS，以应对共模故障（Common-Mode Failures, CMFs）[35]。其他方法通过扩展流水线寄存器添加错误检测和纠正码，例如 Duckcore 在流水线寄存器中扩展了 RISC-V 核心，使用单错误纠正双错误检测（Single Error Correction Double Error Detection, SECDED）[36]。然而，这种方法可能会因编码和解码时间而增加显著开销，并且不能保证对流水线阶段计算逻辑中故障的保护。其他方法通过在 RISC-V 核心内三重化组件来增强其可靠性。例如，控制和状态寄存器、程序计数器和寄存器文件[37]、触发器（FFs）、查找表（LUTs）、块 RAM（BRAMS）和数字信号处理器（DSPs）[38]，以及算术逻辑单元（ALU）[39]被三重化。总体而言，侵入式方法在实现高效的故障纠正机制方面具有很高的灵活性，但实现起来可能具有挑战性。事实上，现有的侵入式方法基于硬件描述语言（HDL）实现，通常相当复杂。最后，上述方法并未专注于提供有界错误检测和纠正时间。然而，当嵌入式系统被应用于既要求硬实时性又要求可靠执行的领域时，这是关键，因为即使在可能的故障存在的情况下，应用的最坏情况执行时间（Worst-Case Execution Time, WCET）也必须是有界的。

为了解决上述问题，我们提出了一种侵入式的同构 RISC-V 基础的双核锁步实现，提供有界额外时钟周期的瞬态错误检测和纠正。

特别是，在[40]中，我们使用开源的 Comet RV32I RISC-V 架构[8]实现了两种侵入式锁步设计，即部分影子寄存器与回滚（Partial Shadow Register with Rollback, PSRR）和全影子寄存器（Full Shadow Register, FSR），通过高级综合（High-Level Synthesis, HLS）实现。我们使用两个相同的 RISC-V 核心，在每

个时钟周期执行相同的指令。每个流水线阶段将其逻辑计算的结果存储在流水线寄存器中。在每个周期，所提出的机制通过比较两个核心的流水线寄存器来检查执行的一致性，如图 3 所示。如果没有检测到错误，则正常执行。否则，检测到的错误表明故障影响了逻辑，从而产生了错误的结果，或者故障影响了流水线寄存器本身，导致结果中的某一位翻转。在这种情况下，立即进行纠正。在 PSRR 方法中，故障指令被重新获取，并再次通过整个流水线，类似于 DARA[34]。流水线中尚未执行的先前指令保持不变，因为它们的执行未受错误影响；然而，流水线中已经存在的后续指令被丢弃。在 FSR 方法中，如果没有检测到故障，我们创建所有流水线寄存器的备份副本。然后，在检测到故障时，丢弃当前计算的结果，两个核心的流水线寄存器用备份值恢复，最后两个核心重新执行受故障影响的周期。

此外，在[41]中，我们强调了以下关键方面：影响核心的瞬态故障不仅影响应用的功能行为，还会对其时序行为产生显著影响，从而影响 WCET 估计。为了实现这一点，我们利用典型的无故障 WCET 估计方法来实现故障感知，通过考虑核心上发生的瞬态故障的影响。具体来说，我们首先通过广泛的故障注入对目标系统进行脆弱性分析。该分析不仅验证了应用在核心上执行时的功能正确性，还验证了其时序正确性。然后，我们应用一种基于测量的典型 WCET 估计方法来验证故障对 WCET 估计的影响。

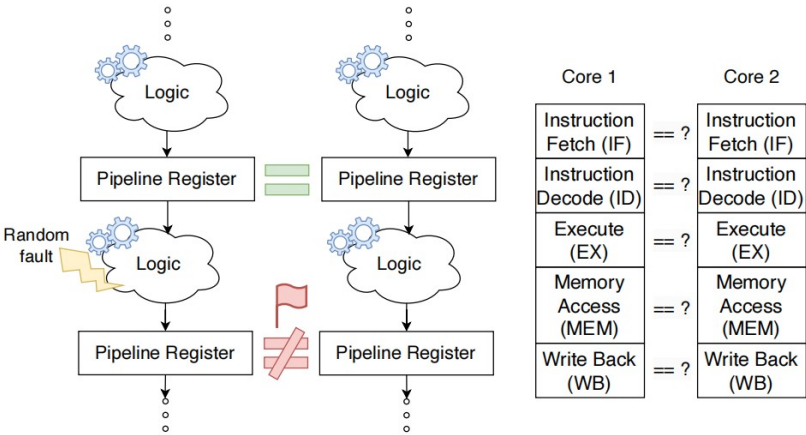


图 2 双核锁步的原理示意图

我们在 TACLeBench（二分查找和素数）、MiBench（快速排序）、AxBench（移动平均）和 Polybench（矩阵乘法）的基准测试中进行了实验。从获得的结果来看，我们观察到在瞬态故障存在的情况下，应用执行时间可能会显著增加，

最高可达 700%，与无故障情况下的应用执行时间相比。此外，与无故障情况下的分布相比，执行时间轨迹的分布也发生了显著变化。

上述观察结果有直接的后果：在故障存在的情况下完成执行所需的时间可能会显著高于无故障的 WCET。因此，现有的方法应该使用看门狗定时器，以限制瞬态故障对应用执行时间的影响，并确保整体调度的安全性。当定时器到期或检测到错误时，应用需要重新执行，根据方法的不同，可能是全部或部分重新执行，从而导致较高的错误检测和纠正时序开销。我们证明了所提出的 DCLS 方法在故障传播并影响执行时间之前立即纠正故障。因此，该方法提供了有界的时序开销（在 FSR 情况下为两个时钟周期），并恢复了 WCET 估计。

V. 讨论与结论

基于 RISC-V 架构的边缘设备正在兴起，这导致了对安全可靠系统的需求。当前的研究提出了三种在 RISC-V 设备上部署的机制，以实现这一目标。随着签名验证算法在后量子时代被证明是不安全的，我们提出了使用 OpenTitan RoT 的物联网设备的后量子安全启动机制。此外，为了检测潜在的可疑活动，我们展示了用于监控指令获取活动、处理的数据以及微处理器状态的硬件机制。最后，我们在双核锁步侵入式 RISC-V 上展示了两种具有有界最坏情况执行时间（WCET）开销的机制，用于检测和从辐射诱导的瞬态故障中恢复。

此外，我们还想强调 RISC-V 开放指令集架构（ISA）能为可靠性研究社区带来的优势。丰富的开源核心实现以及围绕 RISC-V 的庞大社区，使得研究人员能够轻松地实现和测试针对不同可靠性问题的解决方案。然而，我们注意到，目前很少有方法尝试用统一的方法解决不同的挑战，例如，使用相同的硬件同时提供容错和保密性[42]。相反，不同领域的专家（例如可靠性和安全性）通常独立决定他们感兴趣的特定需求，且彼此之间的互动有限。此外，非功能性需求通常被视为可选的附加功能，这增加了相关解决方案的成本。

我们认为，来自不同研究小组且具有不同专业知识的研究人员应该联合起来，将可靠性需求推到未来 RISC-V 核心的主要设计要求之中，这些核心将被用于主流应用。通过这种方式，统一的、高度优化的实现可以蓬勃发展，确保高效且可靠的运行，从而为明天边缘计算设备的 RISC-V 核心提供动力。

参考文献

- [1] [1] C. Palmiero et al., “Design and implementation of a dynamic information flow tracking architecture to secure a risc-v core for iot applications,” in 2018 IEEE High Performance extreme Computing Conference (HPEC). IEEE, 2018, pp. 1 – 7.
- [2] [2] J. Abella et al., “Security, reliability and test aspects of the risc-v ecosystem,” in 2021 IEEE European Test Symposium (ETS), 2021, pp. 1 – 10.
- [3] [3] J. Anders et al., “A survey of recent developments in testability, safety and security of risc-v processors,” in 2023 28th IEEE European Test Symposium (ETS), 2023.
- [4] [4] A. Dorflinger et al., “A comparative survey of open-source application-class risc-v processor implementations,” in Proceedings of the 18th ACM International Conference on Computing Frontiers, ser. CF ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 12 – 20. [Online]. Available: <https://doi.org/10.1145/3457388.3458657>
- [5] [5] Y. H. Hwang, “Iot security & privacy: threats and challenges,” in Proceedings of the 1st ACM workshop on IoT privacy, trust, and security, 2015, pp. 1 – 1.
- [6] [6] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” SIAM review, vol. 41, no. 2, pp. 303 – 332, 1999.
- [7] [7] M. Cui et al., “Fault-tolerant mapping of real-time parallel applications under multiple dvfs schemes,” in IEEE RTAS, 2021, pp. 387 – 399.
- [8] [8] S. Rokicki et al., “What you simulate is what you synthesize: Designing a processor core from c++ specifications,” in 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2019, pp. 1 – 8.

- [9] [9] V. Mavroeidis et al., "The impact of quantum computing on present cryptography," arXiv preprint arXiv:1804.00200, 2018.
- [10] [10] V. B. Kumar et al., "Post-quantum secure boot," in 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2020, pp. 1582 – 1585.
- [11] [11] P. Kampanakis et al., "Post-quantum lms and sphincs+ hash-based signatures for uefi secure boot," Cryptology ePrint Archive, 2021.
- [12] [12] A. Khalid et al., "Lattice-based cryptography for iot in a quantum world: Are we ready?" in 2019 IEEE 8th international workshop on advances in sensors and interfaces (IWASI). IEEE, 2019, pp. 194 – 199.
- [13] [13] J. Zheng et al., "Parallel small polynomial multiplication for dilithium: A faster design and implementation," in Proceedings of the 38th Annual Computer Security Applications Conference, ser. ACSAC ' 22. New York, NY, USA: Association for Computing Machinery, 2022, p. 304 – 317. [Online]. Available: <https://doi.org/10.1145/3564625.3564629>
- [14] [14] L. Beckwith et al., "High-performance hardware implementation of crystals-dilithium," in 2021 International Conference on Field-Programmable Technology (ICFPT). IEEE, 2021, pp. 1 – 10.
- [15] [15] A. Waterman et al., "The risc-v instruction set manual volume ii: Privileged architecture document version 20190608-priv-msu-ratified," RISC-V Foundation, Tech. Rep., 2019.
- [16] [16] L. Cassano et al., "Is risc-v ready for space? a security perspective," in 2022 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), 2022, pp. 1 – 6.
- [17] [17] X. Wang et al., "Software exploitable hardware trojans in embedded processor," in 2012 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), 2012, pp. 55 – 58.
- [18] [18] Y. Jin et al., "Exposing vulnerabilities of untrusted computing platforms," in Proc. Int. Conf. Computer Design, 2012, pp. 131 – 134.
- [19] [19] C. Domas, "Hardware backdoors in x86 cpus," <https://i.blackhat.com/us-18/Thu-August-9/us-18-Domas-God-Mode-Unlocked-Hardware-Backdoors-In-x86-CPU-s-wp.pdf>, 2018.
- [20] [20] X. Chuan et al., "An efficient triggering method of hardware Trojan in AES cryptographic circuit," in Proc. Int. Conf. Integrated Circuits and Microsystems, 2017, pp. 91 – 95.

- [21] [21] Y. Liu et al., "Scca: Side-channel correlation analysis for detecting hardware trojan," in Proc. Int. Conf. Anti-counterfeiting, Security, and Identification, 2017, pp. 196 – 200.
- [22] [22] A. Palumbo et al., "Is your fpga bitstream hardware trojan-free? machine learning can provide an answer," Journal of Systems Architecture, vol. 128, p. 102543, 2022.
- [23] [23] J. Dubeuf et al., "Run-time detection of hardware trojans: The processor protection unit," in 2013 18th IEEE European Test Symposium (ETS), 2013, pp. 1 – 6.
- [24] [24] G. Bloom et al., "Os support for detecting trojan circuit attacks," in 2009 IEEE International Workshop on Hardware-Oriented Security and Trust, 2009, pp. 100 – 103.
- [25] [25] L. Cassano et al., "Deton: Defeating hardware trojan horses in microprocessors through software obfuscation," Journal of Systems Architecture, vol. 129, p. 102592, 2022.
- [26] [26] —, "On the optimization of software obfuscation against hardware trojans in microprocessors," in 2022 25th International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS), 2022, pp. 172 – 177.
- [27] [27] A. P. Fournaris et al., "Exploiting hardware vulnerabilities to attack embedded system devices: a survey of potent microarchitectural attacks," Electronics, vol. 6, no. 3, p. 52, 2017.
- [28] [28] Y. Lyu et al., "A survey of side-channel attacks on caches and countermeasures," Journal of Hardware and Systems Security, vol. 2, no. 1, pp. 33 – 50, 2018.
- [29] [29] A. Bolat et al., "A microprocessor protection architecture against hardware trojans in memories," in 2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS). IEEE, 2020, pp. 1 – 6.
- [30] [30] A. Palumbo et al., "A lightweight security checking module to protect microprocessors against hardware trojan horses," in 2021 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT). IEEE, 2021, pp. 1 – 6.
- [31] [31] K. Arıkan et al., "Processor security: Detect

Is RISC-V ready for Space? A Security Perspective

Luca Cassano¹, Stefano Di Mascio², Alessandro Palumbo³, Alessandra Menicucci²,
Gianluca Furano⁴, Giuseppe Bianchi³, Marco Ottavi^{3,5}

¹*Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano, Italy*

²*Department of Space Engineering, Delft University of Technology, The Netherlands*

³*Dipartimento di Elettronica, Ingegneria dell'Informazione, Università degli Studi di Roma Tor Vergata, Italy*

⁴*European Space Agency, The Netherlands,*

⁵*Faculty of Electrical Engineering, Mathematics and Computer Science, University of Twente, The Netherlands*

¹luca.cassano@polimi.it, ²s.dimascio@tudelft.nl, ²a.menicucci@tudelft.nl,

³name.surname@uniroma2.it, ⁴gianluca.furano@esa.int, ⁵m.ottavi@utwente.nl

Abstract—Integrated circuits employed in space applications generally have very low-volume production and high performance requirements. Therefore, the adoption of Commercial-Off-The-Shelf (COTS) components and Third Party Intellectual Property cores (3PIPs) is of extreme interest to make system design, implementation and deployment cost-effective and viable w.r.t. performance. On the other hand, this design paradigm exposes the system to a number of security threats both at design-time and at runtime. In this paper, we discuss the security issues related to space applications mainly focusing on threats that come from the adoption of the well-known RISC-V microprocessor. We highlight how Hardware Trojan horses (HTHs) and Microarchitectural Side-Channel Attacks (MSCAs) may compromise the overall system operation by either altering its nominal behavior or by stealing secret information. We discuss the security extensions provided by the RISC-V architecture as well as their limitations. The paper is concluded by an overview of the issues that are still open regarding the security of such microprocessor in the space domain.

Index Terms—Microarchitectural Side-Channel Attacks, Microprocessors, Hardware Security, Hardware Trojan Horses, RISC-V, Space Applications.

I. INTRODUCTION

The idea of employing Commercial-Off-The-Shelf (COTS) components, Third Party Intellectual Property cores (3PIPs) and commercial Computer Aided Design (CAD) tools in space systems is becoming increasingly popular [1], [2]. Indeed, this design choice represents an interesting trade-off between performance and cost for application scenarios where the integrated circuits production volume is extremely low, as the case of space applications. The basic idea is to reuse legacy (i.e., not specifically designed for space) subsystems, components and 3PIPs in space systems to benefit from both reduced development cost and high performance [3].

When specifically looking at microprocessors, designers can either choose to define their own instruction set architecture (ISA) or use an already available ISA. While the employment of open-source software can be considered as a legacy procedure, hardware has not yet fully experienced the disruptive effects of openness. Nevertheless, over the last years the RISC-V architecture has risen in popularity, drawing the attention of several universities and companies previously

focusing on other open and free ISAs, proprietary ISAs or even on ISAs designed in-house (with the big drawback of having to design and maintain a software ecosystem) [4].

While reducing costs, this design paradigm exposes the obtained system to a number of security threats both at design-time and at runtime. The production of commercial integrated circuits (ICs) is characterized by a globalized supply chain [5]. The benefit of such a globalized supply chain is a reduction of design cost and time that, on the other hand, comes at the cost of a loss of trust in the final ICs [6]. The consequence of is that it is very hard to ensure/assess the trustworthiness of all the parties involved in the supply chain. Therefore, the product is exposed to a number of threats: ICs may be overproduced by the foundry and sold in the black market [7]; defective or dismissed ICs may be delivered as good ones [8]; IP core licenses may be violated and IP cores may be overused [9], [10]; designs may be maliciously modified to insert stealthy unwanted functionalities in the final product, also known as Hardware Trojan Horses (HTHs) [11].

A novel menace for microprocessor-based systems are *microarchitectural attacks* [12] [13]. These attacks are a sub-class of *Side-Channel Attacks (SCA)*, i.e., attacks that aim at leaking unauthorized information from the system by analysing timing information, power consumption [14], thermal footprint [15] or electromagnetic emanation [16]. On the other hand, microarchitectural attacks do not require the attacker to have physical access to the system under attack. Indeed, such attacks only rely on the observation of the timing behavior of the system while running sensitive applications. The basic idea behind this family of attacks is that since computer architectures are optimized w.r.t. processing speed there is a strong correlation between processed data, memory accesses and execution times: such correlation may represent an exploitable side channel in case the attacked and the attacker processes share the same cache space [12]. A well-known example of microarchitectural attack is *Meltdown* [17], where the attacker exploits out-of-order execution to break address space isolation without exploiting any software bug. Therefore, by exploiting Meltdown an attacker allows his/her own program to access the memory (and thus also secrets) of other programs and of the Operating System (OS).

In the past, hardware security was not considered an issue by the space industry because most attacks, e.g., fault attacks, required the attacker to have physical access to the attacked system, which indeed is not the case of space applications. Nevertheless, attacks based on MSCAs and HTHs may be successfully carried out by without any physical access (at the time of the attack) to the system. Indeed, once implanted, a HTH can be activated remotely through the execution of a trigger program as well as a MSCA can be carried out by forcing the execution of a piece of malicious software running in parallel with the attacked program. Therefore, hardware security issues should nowadays be tackled also when considering a digital design meant for a space application.

Microprocessors are present in On-Board Data Handling systems and On-Board Data Processing systems, both has hard cores in ASICs/SoCs and as soft cores in FPGAs. Microprocessors are key components to run the algorithms required for the Attitude Determination and Control System (ADCS) and execute ground-control or algorithm-based Command & Data Handling (C&DH) [18]. The orientation is critical for the satellite's mission objective (e.g. a wrong orientation would make communication with the ground station not possible). The execution of C&DH is critical too, given the master/agent relationship of ground stations to satellites [18].

In this perspective paper, we discuss the security issues related to the adoption of the RISC-V platform in space applications. We first present the security features that are natively offered by the RISC-V architecture and we present their limitations in preventing and alleviating the effect of hardware security-related attacks. Indeed, specifically looking at hardware security, we highlight how Hardware Trojan horses (HTHs) and Software Threats, with particular emphasis to Microarchitectural Side-Channel Attacks (MSCAs), may damage the overall system functioning by either altering its nominal behavior or stealing secret information. Finally, we overview the existing solutions coming from research that aim at further securing the adoption of RISC-V processors also presenting and discussing issues that have not yet been addressed. This paper is meant to help technicians in finding the best architecture configuration to meet both performance and security requirements. At the same time this work can represent a starting point for young researchers and students by providing a discussion of the state-of-the-art solutions to secure RISC-V as well as an overview of the open problems.

The remainder of this paper is organized as follows: Section II describes possible threats in space systems and their impact on different space missions; Section III discusses the main security-related features provided by the RISC-V architecture and their limitation; Finally, Section IV draws conclusions, highlighting open issues.

II. SECURITY IN SPACE DATA SYSTEMS

In the space embedded systems domain we are now witnessing a novel trend. While “classical” processing ASICs keep suffering of the usual, widening, performance gap between ‘terrestrial’ processors and space grade ones (as discussed

in [19]), on the counter recent developments especially in SRAM-based FPGAs have brought terrestrial ‘edge’ technologies for space use, providing a possible quantum leap for processing in space. Especially for payload data processing tasks, these devices will likely replace any dedicated application-specific standard product (ASSP) and call for availability of sophisticated processor IP cores for specific applications.

In the following we review the main security issues related to space missions highlighted by the Consultative Committee for Space Data Systems (CCSDS) in [20].

A. Threats

1) *Intentional Data Corruption*: Data may be intentionally altered by an attacker at its source (e.g. sensor jamming) by an attacking satellite operating nearby the attacked one [18]. Another possibility is that data is altered during its transmission from the instrument to the ground system, e.g., by the execution of malicious software. Along with the corruption of information coming from the instruments, intentional data corruption could potentially lead to a catastrophic loss of the system, e.g. if, at the reception of a command, no action or a wrong action is taken by the spacecraft. For example, if a navigational maneuvering command is altered, the spacecraft may eventually enter an unusable orbit and miss the encounter with a target, or even be destroyed.

2) *Software Threats*: Coding errors may happen and may cause security problems [20]. Furthermore, the system operator may configure the system incorrectly, leading to security weakness. On the other hand, the programmer may introduce logic or implementation errors that may lead to system vulnerabilities. Finally, external agents may try to use vulnerabilities to inject software or change configurations. The effect can be data loss, loss of spacecraft control, unauthorized spacecraft control, or mission failure.

Software Threats (STs) may also come from the exploitation of hardware- and architecture-level vulnerabilities. For instance, high-performance processors may employ speculation and out-of-order execution. These features can be maliciously used to access protected processor memory locations, opening the way to so-called Microarchitectural Side-Channel Attacks (MSCAs) [12]. This type of SCAs is particularly critical (also for space applications) because it does not require physical access to the system under attack, relying on the monitoring of the features of interest for the attack itself, e.g., timing behavior, performance counters, of the attacked processor. Some examples of MSCA, are the *Meltdown* [17], *Spectre* [21] and *Foreshadow* [22], where the attacker exploits out-of-order and speculative execution to break address space isolation without exploiting any software bug. Thus, these attacks allow the attacker to access unauthorized memory regions and steal information. As a result, a system built using trusted components may be successfully attacked.

3) *Malicious hardware*: Issues related to hardware trust arise from involvement of untrusted entities in the life cycle of the designed system, including untrusted IP providers, design team components, CAD tools and fabrication, test,

and distribution facilities [23]. It is possible that malicious HW finds its way in the system when COTS components are employed. These components usually do not have a trusted supply chain and are deployed to mass markets. As a matter of fact, the supply chain for COTS components consist of many (potentially untrusted) entities [23].

An undesired addition or modification to existing circuit elements, done in order to apply malicious activity is called Hardware Trojan (HT). HTs can change the functionality of a circuit, reduce its reliability or leak valuable information [23]. It has been known that *software-exploitable HTs* may be integrated in Microprocessor. Thanks to an HT, attackers may be able to execute their own malicious software, to modify the running software or to acquire root privileges [24], [25], [26]; or they may be able to enter in supervisor mode thanks to a backdoor activated via software [27]. Finally, it has to be mentioned that the most recent attack vector is now represented the employed CAD tools. Indeed, it has been demonstrated that HTHs may be introduced in the designed circuit by the tools employed for high-level and logic synthesis, for place-and-route and for bitstream generation [28], [29].

B. Impact of threats on different space missions

The impact of security-related threats highly depends on the mission orbit and on the type of mission the system is meant for. In the following we discuss these two points.

1) *Mission orbits*: On the one hand, Geostationary Orbit (GEO) satellites are more secure than Low-Earth Orbit (LEO) missions because they require ground stations with a higher transmission power and larger antennas. This, of course, make communication with the satellite more expensive, thus limiting the number of possible attackers. On the other hand, GEO satellites are more vulnerable than LEO ones because they are continuously visible in specific geographical areas, while LEO satellites are visible only for specific time intervals from the ground station. Moreover, constellations of satellites increase the vulnerability of the whole space system, because in this case there are several satellites in orbit, increasing the visibility window from a specific point on Earth. Finally, deep-space/interplanetary missions require even larger antennas and higher power compared to a GEO satellite to attack [20].

2) *Type of missions*: The type of mission carried out by the space system, as well as its safety-/mission-criticality, highly influences the impact and likelihood of security-related threats. In the following we discuss several types of space missions and associated security issues.

Earth Observation Satellites can be either scientific or a critical asset of governments (e.g. meteorological forecasting applications). In the latter case the most likely threat is unauthorized access, which is typically countered with encryption of commands and mission data and authenticated commands. Moreover, malicious hardware is a possible threat that can be mitigated with analysis of the hardware functionalities, or careful selection of a trusted supplier. Data corruption may also be an issue, providing wrong information to the final user.

Communication Satellites are meant to provide a service with a usually very high availability. For example, in the case of a GEO telecommunication satellite the whole space system is expected to provide the service 99.9% of the time [30]. Therefore, these type of satellites are particularly vulnerable: indeed, even a slight reduction of the availability may have a huge impact on the service. An example of attack to a communication satellite is reported in [18]: in 2003, the 12 satellites of Telestar have been attacked using an uplink station that sent contradictory frequency to the satellites, which overrode the signal, effectively blocking television programming.

Navigation Satellites are often dual-use satellites, and often they belong to critical government infrastructures. An example is the GPS, that is employed by individuals, companies and the military. Like communications satellites, the loss of navigation satellite systems would result in potential loss of life, safety of individuals, and critical infrastructure.

Science missions are usually not part of a critical infrastructure. Even if they are exposed to the same threats as other missions, the impact is much less critical, with a damage to the financial investment and to the reputation of the entities involved in the mission itself [20]. However, investments might be very high: for instance, the cost of the recently launched James Webb Space Telescope reached 10 billion USD [31].

Manned missions are of course the most critical ones under the safety point of view. Indeed, in this cases the effect of an attack causing the loss of a spacecraft may be catastrophic, with the possibility of issues for the lives of the crew.

III. SECURITY CONSIDERATIONS ON RISC-V

We here present the RISC-V architecture and discuss the available security-related features; finally, we survey the existing limitations and some proposed security extension.

A. The RISC-V Instruction Set Architecture

RISC-V was originally developed by UC Berkeley to support computer architecture research and education oriented at hardware implementations [32]. The spread of an open and free ISA already enabled a vast field of research activities for terrestrial application (e.g. security, AI, etc.). Indeed, having access to proprietary architectures is expensive and it limits what can be done with a certain product or within a certain research activity. The availability of a software ecosystem supported by a large open community ignited an unprecedented amount of developments, with several announcements and/or releases of open-source implementations. The adoption of a popular free and open ISA can thus lead to shorter time to market and lower costs thanks to reuse.

The main feature of the RISC-V ISA specifications is its modularity, which allows to cover a large spectrum of applications and to increase software reuse (adding new instructions as optional extensions instead of releasing new versions of the whole ISA). RISC-V allows for both standard and non-standard extensions (defined outside the specifications). The user-level RISC-V ISA is defined as a base integer (I) ISA,

which must be present in any implementation, plus optional extensions to the base ISA. A subset of the integer base (E) can optionally be implemented when an implementation targets small 32-bit microcontrollers, with 16 general purpose registers instead of 32. The standard defines a "general" subset (G) and the set of extension required for general purpose computing systems.

B. Memory isolation

The main feature available at software level to preserve the security of a system is memory isolation (i.e. each process has its own address space, which isolates memory between programs) [33]. Isolation implies that even if the security of a partition is compromised, the attacker cannot breach the security of other partitions. The Linux kernel provides separation between kernel services and user-space applications. However, in this way the kernel represents a single-point of failure in terms of security. Therefore, it may be responsible of a high number of vulnerabilities, given its huge attack surface [34].

Memory isolation can be further improved by employing hypervisors to provide isolation between different instances of OSs running on multicore processors. Memory isolation is currently being investigated in space applications to improve safety behavior, i.e. to avoid that the failure of an application running on an OS may impact the execution of another application running in a different OS on the same processor [35]. Given the increasing complexity of software systems also in space applications, designers increasingly rely on 3rd-party software components, typically provided as software libraries. When the code of these libraries is open source, a security analysis and validation is possible. On the other hand, proprietary software instead generally comes as linkable (and non inspectable) object modules. Therefore, it is often required to enforce the separation of the various software components within the system [34].

C. RISC-V software stacks and privilege levels

The modular nature of RISC-V allows for different software stack implementations with different levels of security [3]. For instance, Figure 1 shows the stacks described in the RISC-V Privileged Specification [36]. On the left side of the figure, a simple stack implementation is shown. The application interacts via an Application Binary Interface (ABI) with the Application Execution Environment (AEE). In this simple case the ABI is composed of the implemented user-level ISA subset. At the center of the figure, multiple applications run and communicate via the ABI with the OS (the latter in this case providing the AEE). In turn, the OS communicates with a supervisor execution environment (SEE) via a Supervisor Binary Interface (SBI). An SBI comprises the user-level and supervisor-level ISA together with a set of SBI function calls. The SEE can be a simple bootloader and BIOS-style IO system on a low-end hardware platform, up to a virtual machine in a high-end server, or a thin translation layer over a host operating system in an architecture simulation environment. On the right side of the figure, RISC-V runs a virtual machine monitor

configuration where multiple OSs run on top of a hypervisor. Each OS communicates via an SBI with the hypervisor, which in turn provides the SEE. The hypervisor communicates with the hypervisor execution environment (HEE) using a hypervisor binary interface (HBI) to isolate the hypervisor from the hardware platform.

Privilege levels are used to provide protection between different components of the described software stacks. The possible levels a thread may have are:

- The *machine mode (M-mode)* has the highest privileges and it is the only mandatory one. Code running in machine-mode has to be totally trusted, as it has access to the machine implementation.
- The *supervisor mode* has been added to provide isolation between the Operating System and the SEE.
- The *user-mode* is dedicated to the user applications.

The privilege levels are designed and checked such that each attempt to perform operations not permitted by the current mode will cause an exception to be raised.

Any specific RISC-V implementation can choose which modes to implement (except for the mandatory M-mode). This allows hardware architects to identify a suitable trade off between offered security level and implementation cost [36]. Typical solutions are:

- M: simple embedded systems where security is not a concern. This solution will provide no protection against incorrect or malicious application code.
- M, U: secured embedded systems
- M, S, U: for systems running a Unix-like OS. The supervisor mode is added to provide isolation between the OS and the SEE.

A user thread normally runs the application code in U-mode until a trap, e.g., a supervisor call or a timer interrupt, forces a switch to a trap handler, that usually runs in a more privileged mode. The trap handler will be executed and eventually the user thread will resume its execution at or after the original trapped instruction, again in U-mode.

D. RISC-V extensions for security

Beside different privilege modes, the RISC-V ISA provides additional extensions to increase security.

1) *Physical Memory Protection*: The M-mode supports an optional standard extension for Physical Memory Protection (PMP). The PMP extension defines control registers to specify access privileges (read, write, execute) for each physical memory region. Attempting to fetch an instruction from a PMP region that does not have execute permissions or to load from a physical memory location within a PMP region without read permissions raises an exception.

The use of the PMP provides several security features. For example, threads are prevented from modifying or even reading the data of shared libraries. Therefore, the memory region in which the shared library data reside can be set as execute only using PMP. Even if the PMP is mainly used to prevent threads running in lower privilege levels (e.g. U

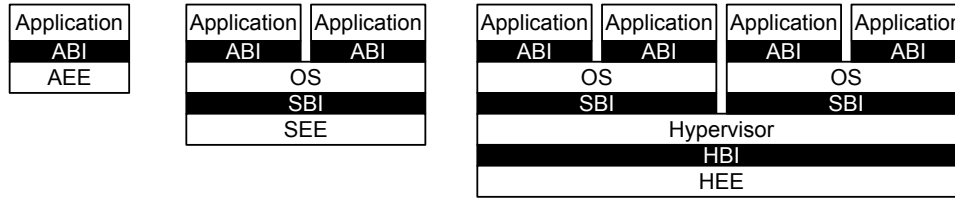


Figure 1: Examples of software stack implementations with different levels of security [36].

and S modes) from accessing privileged memory contents, the "lock" feature provides protection even when only the M-mode implemented. As a matter of fact, if a PMP entry is locked, write operations to the configuration register and the associated address registers of the entry are ignored. Indeed, PMP restrictions are valid also for the M-mode: locked PMP entries may only be unlocked with a system reset.

2) *Cryptographic extension*: The basic requirement for security in spacecrafts is the use the AES encryption standard to encrypt telemetry and telecommand signals [37]. The implementation of a cryptographic extension increases performance and hides implementation details from software, reducing the attack surface [38]. However it is often preferable to implement encryption and decryption of telemetry and commands in a separated hardware component or board, as it is required that the keys are not accessible from software.

3) *User-level interrupt extension (N)*: This extension allows isolation between interrupt handler having different privilege levels. Indeed, interrupt handlers can be executed at user-level, thus being unable to compromise the isolation model [34].

E. Limits of isolation and need for hardware solutions

Most attacks to isolation exploit physical access to the processor to perform Fault Injection (FI) [39]. In space this may be more complex than in applications like mobile and Internet of Things (IoT), although in principle still possible. For instance, in [39] it is shown how knowledge of power consumption for each instruction can be employed to skip an instruction during context switch, allowing access to PMP protected portions of memory. On the other hand, the previously presented attacks, i.e., MSCAs and HTHs, can be successfully carried out by the attacker without any physical access (at the time of the attack) to the system under attack. Indeed, HTHs can be implanted at design time by one of the many untrusted parties involved in the design. Once implanted, a HTH can "easily" be activated through the execution of a trigger program. Similarly, a MSCA is a piece of malicious software running in parallel with the attacked software. Therefore, MSCAs and HTHs have to be considered serious threats also in the space scenario, where it is extremely difficult for the attacker to have physical access to the system. While the security features natively integrated in the RISC-V architecture do not protect the system from MSCAs and HTHs, we argue that innovative security solutions (coming from research) should be integrated in the RISC-V architecture to make space missions adopting this microprocessor secure and trustable.

F. Security Architectures for RISC-V

Several system-level methodologies meant to protect systems based on Intel and ARM processors against both MSCAs and HTHs have been recently proposed. In particular, the new trend aims at providing a secure and trustable program execution over a system built with untrusted components. The main idea behind most of these approaches consists in the introduction Hardware Security Checkers (HSCs) able to monitor the fetching activity and, the processed data and the status of the Microprocessor for detecting potentially suspicious activities, e.g., HTHs and/or MSCAs.

The same effort has not yet been devoted to secure systems based on RISC-V processors. In [40] the authors proposed a HSC based on Bloom filters to protect the system against HTHs infesting the memories. At runtime, the HSC monitors the memory locations accessed by the microprocessor and the corresponding fetched instructions and based on this information it decides whether a HTH has been activated or not. The same authors proposed in [41] a similar solution where the target are HTHs infesting the microprocessor instead of the memory. Finally, probabilistic data structures are employed in [42] to detect the activation of MSCAs.

IV. DISCUSSION AND OPEN ISSUES

The growing interest in deploying commercial microprocessors in space applications as well as the rise of new hardware-oriented menaces, e.g., MSCAs and HTHs, make security (and hardware security, in particular) an open and severe issue. As we have discussed in the paper, RISC-V microprocessor already implements several security features. Indeed, Physical Memory Protection, Cryptography extension and User-level interrupts already protect a RISC-V based system against several attacks, e.g., data corruption and unauthorized access. On the other hand, the protection of RISC-V processors against MSCAs and HTHs is still an issue.

Apart from the protection against MSCAs and HTHs, we still see two main open issues related to the trusted adoption of RISC-V processors in space missions. The first issue is the lack of a trusted and robust tool chain: indeed, as it has already been discussed in the previous sections, it has been demonstrated that malicious modifications can be introduced in the designed system by the employed CAD tools. Therefore, the availability of trusted tools and of methodology to ensure the trustworthiness of the employed tools it is vital. The second main issue is related to the integration of third party intellectual property cores (3PIPs) within a RISC-V based

system. A trusted interaction between dedicated hardware accelerators and the main processor is fundamental to ensure security to the entire system. Therefore, methodologies to verify trusted behaviors and hardware-level mechanisms, like hardware-based firewalls, to isolate untrusted components and limit their dangerousness should be investigated.

REFERENCES

- [1] S. Esposito, C. Albanese, M. Alderighi, F. Casini, L. Giganti, M. L. Esposti, C. Monteleone, and M. Violante, "Cots-based high-performance computing for space applications," *IEEE Transactions on Nuclear Science*, vol. 62, no. 6, pp. 2687–2694, 2015.
- [2] M. Pignol, "Cots-based applications in space avionics," in *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*, IEEE, 2010, pp. 1213–1219.
- [3] S. Di Mascio, A. Menicucci, E. Gill, G. Furano, and C. Monteleone, "Leveraging the openness and modularity of risc-v in space," *Journal of Aerospace Information Systems*, vol. 16, no. 11, pp. 454–472, 2019. [Online]. Available: <https://doi.org/10.2514/1.1010735>
- [4] S. Di Mascio, A. Menicucci, G. Furano, C. Monteleone, and M. Ottavi, "The case for risc-v in space," in *International Conference on Applications in Electronics Pervading Industry, Environment and Society*. Springer, 2018, pp. 319–325.
- [5] DIGITIMES, "Trends in the global IC design service market," <http://www.digitimes.com/news/a20120313RS400.html?chid=2>.
- [6] Mohammad Tehranipoor and Cliff Wang, *Introduction to Hardware Security and Trust*. Springer-Verlag New York, 2012.
- [7] U. Guin, Ziqi Zhou, and A. Singh, "A novel design-for-security (dfs) architecture to prevent unauthorized ic overproduction," in *2017 IEEE 35th VLSI Test Symposium (VTS)*, 2017, pp. 1–6.
- [8] U. Guin, K. Huang, D. DiMase, J. M. Carulli, M. Tehranipoor, and Y. Makris, "Counterfeit integrated circuits: A rising threat in the global semiconductor supply chain," *Proc. IEEE*, vol. 102, no. 8, pp. 1207–1228, 2014.
- [9] A. P. Donlin, P. Sundararajan, and B. J. New, "Method and system for secure exchange of ip cores," Aug. 2010, uS Patent 7,788,502.
- [10] A. Carelli, C. A. Cristofanini, A. Vallerio, C. Basile, P. Prinetto, and S. Di Carlo, "Securing bitstream integrity, confidentiality and authenticity in reconfigurable mobile heterogeneous systems," in *2018 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR)*, 2018, pp. 1–6.
- [11] M. Tehranipoor and F. Koushanfar, "A survey of hardware trojan taxonomy and detection," *IEEE Design & Test of Computers*, vol. 27, no. 1, 2010.
- [12] A. P. Fourmaris, L. Pocero Fraile, and O. Koufopavlou, "Exploiting hardware vulnerabilities to attack embedded system devices: a survey of potent microarchitectural attacks," *Electronics*, vol. 6, no. 3, p. 52, 2017.
- [13] R. Spreitzer, V. Moonsamy, T. Korak, and S. Mangard, "Systematic classification of side-channel attacks: A case study for mobile devices," *IEEE Communications Surveys Tutorials*, vol. 20, no. 1, pp. 465–488, 2018.
- [14] C. Luo, Y. Fei, P. Luo, S. Mukherjee, and D. Kaeli, "Side-channel power analysis of a gpu aes implementation," in *2015 33rd IEEE International Conference on Computer Design (ICCD)*. IEEE, 2015, pp. 281–288.
- [15] R. J. Masti, D. Rai, A. Ranganathan, C. Müller, L. Thiele, and S. Capkun, "Thermal covert channels on multi-core platforms," in *24th {USENIX} Security Symposium ({USENIX} Security 15)*, 2015, pp. 865–880.
- [16] J. Longo, E. De Mulder, D. Page, and M. Tunstall, "Soc it to em: electromagnetic side-channel attacks on a complex system-on-chip," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2015, pp. 620–640.
- [17] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018.
- [18] G. Falco, "When satellites attack: Satellite-to-satellite cyber attack, defense and resilience," in *ASCEND 2020*, 2020, p. 4014.
- [19] G. Furano and A. Menicucci, *Roadmap for on-board processing and data handling systems in space*. Springer International Publishing, 8 2017, pp. 253–281.
- [20] CCSDS, "Security threats against space missions," *Informational Report (CCSDS 350.1-G-2)*, 2015.
- [21] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1–19.
- [22] O. Weiss, J. Van Bulck, M. Minkin, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, R. Strackx, T. F. Wenisch, and Y. Yarom, "Foreshadow-NG: Breaking the virtual memory abstraction with transient out-of-order execution," *Technical report*, 2018.
- [23] S. Bhunia and M. Tehranipoor, *Hardware security: a hands-on learning approach*. Morgan Kaufmann, 2018.
- [24] Y. Jin, M. Maniatakos, and Y. Makris, "Exposing vulnerabilities of untrusted computing platforms," in *Proc. Int. Conf. Computer Design*, 2012, pp. 131–134.
- [25] N. G. Tsoutsos and M. Maniatakos, "Fabrication attacks: Zero-overhead malicious modifications enabling modern microprocessor privilege escalation," *IEEE Trans. Emerging Topics in Computing*, vol. 2, no. 1, pp. 81–93, 2014.
- [26] X. Wang, T. Mal-Sarkar, A. Krishna, S. Narasimhan, and S. Bhunia, "Software exploitable hardware trojans in embedded processor," in *2012 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 2012, pp. 55–58.
- [27] <https://github.com/xoreaxeaxe/roosenbridge>.
- [28] J. A. Roy, F. Koushanfar, and I. L. Markov, "Extended abstract: Circuit cad tools as a security threat," in *2008 IEEE International Workshop on Hardware-Oriented Security and Trust*, 2008.
- [29] M. Potkonjak, "Synthesis of trustable ics using untrusted cad tools," in *Proceedings of the 47th Design Automation Conference*, 2010, pp. 633–634.
- [30] B. Kosinski and K. Dodson, "Key attributes to achieving >99.99 satellite availability," in *2018 IEEE International Reliability Physics Symposium (IRPS)*, March 2018, pp. 6A.3–1–6A.3–10.
- [31] J. B. Pessoa, "Space age: Past, present and possible futures," *Journal of Aerospace Technology and Management*, vol. 13, 2021.
- [32] A. Waterman, Y. Lee, D. Patterson, K. Asanovic, and V. I. U. level Isa, "The risc-v instruction set manual," *Volume I: User-Level ISA*, version, vol. 2, 2014.
- [33] R. Shu, P. Wang, S. A. Gorski III, B. Andow, A. Nadkarni, L. Deshotels, J. Gionta, W. Enck, and X. Gu, "A study of security isolation techniques," *ACM Comput. Surv.*, vol. 49, no. 3, Oct. 2016. [Online]. Available: <https://doi.org/10.1145/2988545>
- [34] C. Garlati and S. Pinto, "A clean slate approach to linux security risc-v enclaves," in *Proceedings of the Embedded World Conference, Nuremberg, Germany*, 2020.
- [35] F. Gómez, M. Masmano, V. Nicolau, J. Andersson, J. Le Rhun, D. Trilla, F. Gallego, G. Cabo, and J. Abella Ferrer, "De-risc-dependable real-time infrastructure for safety-critical computer systems," *Ada User Journal*, vol. 41, no. 2, pp. 107–112, 2020.
- [36] A. Waterman and K. Asanovic, "The risc-v instruction set manual volume ii: Privileged architecture document version 20190608-priv-msu-ratified," RISC-V Foundation, Tech. Rep., 2019.
- [37] D. Lopez and E. Fraga, "Tm/tc encryption system," in *14th International Conference on Space Operations*, 2016, p. 2330.
- [38] T. Lu, "A survey on risc-v security: Hardware and architecture," *ArXiv*, vol. abs/2107.04175, 2021.
- [39] S. Nashimoto, D. Suzuki, R. Ueno, and N. Homma, "Bypassing isolated execution on risc-v with fault injection," *Cryptology ePrint Archive*, Report 2020/1193, 2020, <https://ia.cr/2020/1193>.
- [40] A. Bolat, L. Cassano, P. Reviriego, O. Ergin, and M. Ottavi, "A micro-processor protection architecture against hardware trojans in memories," in *2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*. IEEE, 2020, pp. 1–6.
- [41] A. Palumbo, L. Cassano, P. Reviriego, G. Bianchi, and M. Ottavi, "A lightweight security checking module to protect microprocessors against hardware trojan horses," in *2021 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 2021, pp. 1–6.
- [42] K. Arian, A. Palumbo, L. Cassano, P. Reviriego, S. Pontarelli, G. Bianchi, O. Ergin, and M. Ottavi, "Processor security: Detecting microarchitectural attacks via count-min sketches," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, pp. 1–14, 2022.

Towards dependable RISC-V cores for edge computing devices

Pegdwende Romaric Nikiema¹, Alessandro Palumbo², Allan Aasma³, Luca Cassano², Angeliki Kritikakou^{1,5}, Ari Kulmala³, Jari Lukkarila³, Marco Ottavi⁴, Rafail Psiakis³, Marcello Traiola¹

¹Univ Rennes, Inria, IRISA, CNRS, France, ²Politecnico di Milano, Italy,

³Technology Innovation Institute, United Arab Emirates,

⁴Twente University, the Netherlands and University of Rome Tor Vergata, Italy

⁵Institut universitaire de France (IUF)

¹{pegdwende.nikiema; angeliki.kritikakou; marcello.traiola}@inria.fr, ²first_name.last_name@polimi.it,

³first_name.last_name@tii.ae, ⁴m.ottavi@utwente.nl

Abstract—The migration of the computation from the cloud into edge devices, i.e., Internet-of-Things (IoT) devices, reduces the latency and the quantity of data flowing into the network. With the emerging open-source and customizable RISC-V Instruction Set Architecture (ISA), cores based on such ISA are promising candidates for several application domains within the IoT family, such as automotive, Unmanned Aerial Vehicles (UAVs), industrial automation, healthcare, agriculture etc., where power consumption, real-time execution, security and reliability are of highest importance. In this emerging new era of connected RISC-V IoT devices, mechanisms are needed for a reliable and secure execution, still meeting area, energy consumption and computation time constraints of edge devices. We propose three mechanisms towards this goal, i.e., (i) a Root of Trust module for post-quantum secure boot, (ii) hardware checkers against hardware trojan horses and microarchitectural side-channel attacks, and (iii) a fine-grained dual core lockstep mechanism for real-time error detection and correction. The paper illustrates the proposed mechanisms with related motivations and implications, as well as a discussion on future research directions.

Index Terms—RISC-V, OpenTitan, RoT, Secure Boot, Post-Quantum Cryptography, PQC, Security, Safety, CRYSTALS-Dilithium, Lockstep Dual Core, Recovery, Security Checker, Hardware Trojan Horses, Microarchitectural Side-Channel Attacks.

I. INTRODUCTION

In order to reduce not only the latency, but also the quantity of data flowing into the network, there is an incentive to migrate computation from the cloud into the edge devices, i.e., Internet-of-Things (IoT) devices. With the emerging open-source and customizable RISC-V Instruction Set Architecture (ISA), RISC-V came into the foreground gaining more and more attention of the industry [1]. RISC-V is a promising candidate for several application domains within the IoT family, such as automotive, Unmanned Aerial Vehicles (UAVs), industrial automation, healthcare, agriculture etc., where power consumption, real-time execution, security and reliability are of utmost importance [2], [3]. However, the majority of RISC-V works mainly focus on design for performance and power

consumption, often neglecting dependability and security issues [4]. In this emerging new era of connected RISC-V IoT devices, mechanisms are needed for reliable and secure execution [5], meeting area, energy consumption and computation time constraints of edge devices.

To achieve this goal, such mechanisms should address the complete system stack. The fundamental mechanism, upon which the whole reliable and safety stack is built, is the secure boot, where the executed code is authenticated and its integrity is verified. After the system boot, the system execution has to be protected from faults injected either maliciously or unintended. Such faults can impact the fetching of the instructions and data from the memory to the processors or even the processor execution.

In this work, we propose three mechanisms applied at different layers of the system stack in order to achieve dependable and secure RISC-V cores. As a first step, a strong dependability foundation should be established through a secure boot scheme, bound to the hardware using a Root of Trust (RoT) module, such as OpenTitan, that uses digital signatures based on Public Key Cryptography (PKC) algorithms, such as RSA and ECC. However, the emergence of quantum computing technology makes integer factorization and discrete logarithms-based cryptography, such as RSA and ECC, respectively breakable [6]. Hence, there is a need for data, code integrity and authentication solutions that fit the post-quantum era. However, OpenTitan is unable to support existing public-key Post-Quantum Cryptographic (PQC) algorithms in its secure boot flow without major modifications, due to its resource limitations. To tackle this limitation, we propose an enhanced secure boot flow for OpenTitan RoT, modifying the OpenTitan hardware and software architecture, enabling post-quantum security.

As a second step, in order to protect from attacks during the transfer of the data from memory to the processors, mechanisms are proposed to protect from executing unwanted software and accessing illegal memory locations. To achieve that, we propose hardware mechanisms to monitor the fetching activity, the processed data and the status of the micropro-

cessor in order to detect potentially suspicious activities, i.e., the activation of a Hardware Trojan Horse or the attempt of running a Microarchitectural Side-Channel Attack.

Last, in order to deal with hardware faults of different nature (i.e., related to manufacturing imperfections, to aging, or radiation-induced) occurring inside the processors, mechanisms typically based on redundancy are used: the same set of operations with same inputs is executed on different processors. Such approach is very effective, since the probability of having the same fault concurrently occurring on all processors is very small [7]. However, it comes in high error detection and correction cost, when it is applied at a coarse-grained level. Therefore, we propose a fine-grained lockstep version a RISC-V processor with fast error detection and correction features. We designed the approach by using High-Level Synthesis (HLS). HLS makes the design process less complex, as the processor model can easily be modified, expanded and verified, compared to HDL implementations [8].

The reminder of this paper is organized as follows: Section II presents the post-quantum secure boot mechanism; Section III introduces the hardware security checkers for anti-Trojan and anti-Side-Channel Attacks; Section IV discusses the dual-core lock step implementation for error detection and correction; finally, Section V concludes the paper.

II. POST-QUANTUM SECURE BOOT ON OPENTITAN ROT

A. Motivation

OpenTitan is an open source RISC-V based Root of Trust (RoT), one use case of which is to establish the foundation of a secure chain of trust, through a secure boot procedure. As mentioned, for its secure boot, OpenTitan currently uses PKC schemes that are not post quantum safe, therefore our goal is to identify the best candidate algorithm for OpenTitan.

Current PQC mathematical solutions are based on hashes, codes, multi-variants and lattices [9]. Taking into consideration the resource constraint environments of the RISC-V edge devices, PQC secure boot algorithms should be implemented meeting area and energy consumption, as well as computation time. The first PQC hardware-based secure boot implementation using the post-quantum hash-based signature scheme is presented and compared to a hardware and software ECDSA secure boot solution [10]. PQC secure boot signatures for efficient PQC UEFI Secure Boot based on LMS and SPHINCS+ algorithms have minimal authentication time and boot time, and memory footprint [11], making these algorithms very good candidates for edge devices with restricted resources.

A survey on lattice-based cryptography implementations on constrained embedded devices, compared to traditional public-key schemes, is presented in [12]. It is shown that, as the key size increases, the performance of lattice-based cryptography schemes deteriorates. When measuring performance of different lattice-based schemes on ARM cortex-M4 micro-processor, CRYSTALS-Dilithium has the highest throughput performance in comparison to other schemes. CRYSTALS-Dilithium is also one of the two signature finalists in the third round of the NIST PQC standardization project [13],

making it a good candidate to be implemented in OpenTitan. However, such a security scheme implies a complex software implementation and its introduction in the first boot stages of an IoT device is not straightforward, due to the ROM size limitations of edge devices.

To overcome this limitation and provide PQC resistant boot scheme, we extend the RTL of OpenTitan to support CRYSTALS-Dilithium acceleration. The CRYSTALS-Dilithium hardware primitive [14] is used in our design in order to verify the signature of the next stages of the boot.

B. The proposed PQC Secure Boot For OpenTitan

The proposed PQC Secure Boot on OpenTitan ensures that only payloads, which have been verified, are executed, as the system boots-up. Note that, the Secure Boot flow discussed in this manuscript excludes the later boot stages of BL0 firmware and the boot of the kernel of the supported OS. The first stage of the Boot process resides in the Boot ROM. The ROM code is programmed into the chip's ROM during manufacturing, and it is immutable. It contains a set of public keys used to verify the first boot stage (i.e. ROM_EXT) stored in flash.

The main task of the ROM code is to prepare OpenTitan for executing ROM_EXT, ensuring also that the loaded ROM_EXT is allowed to be executed on this chip (ROM Extension - Stored in flash and signed by the Silicon Creator). Figure 1 gives an overview of the two steps of the boot code. As soon as the system is powered-on/reset, the boot process

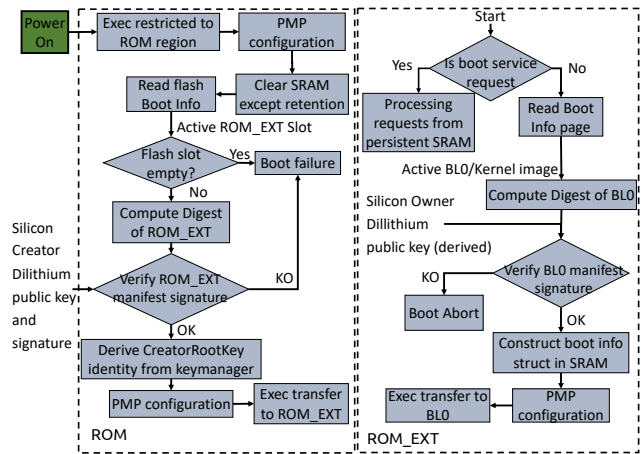


Fig. 1. OpenTitan PQC secure boot code.

starts. The execution is restricted to the OnChip ROM, which resides inside OpenTitan. Execution of other types/areas of memory is initially prevented by the reset logic. The execution enters the ROM code and the enhanced Physical Memory Protection (ePMP of the Ibex core of OpenTitan) is enabled and configured. At this stage, only the ROM region, where the executable boot code resides, is accessible to the core. All SRAM, except for retention SRAM, is cleared. The active boot slot is loaded from the flash Boot Info. Flash boot info resides in the flash default bank. Starting with the Active ROM_EXT Slot, the ROM code performs the following steps:

- It determines if the Active ROM_EXT slot is empty, by testing for presence of the header magic value. If it is present, it continues with the validation of ROM_EXT. If it is not present, it executes the boot failure logic.
- If the Active ROM_EXT slot is empty, the boot process reads the ROM_EXT code and computes its SHA2-256 digest.
- Using CRYSTALS-Dillithium hardware primitive we perform a verification of the signature inside the ROM_EXT manifest, using the digest and the Silicon Creator Public Key, which is stored into the ROM. If the validation is successful, the process continues. If it fails, the process executes the boot failure logic.
- Afterwards, it performs system state measurements and it derives the CreatorRootKey Identity, which will reside in the key manager, as an intermediate state.
- Next, it enables the execution of the ROM_EXT by configuring the appropriate PMP entry.
- Finally, it transfers the execution to the entry point specified in the ROM_EXT manifest for the active slot.
- The execution jumps into the flash, which is eXecute In Place (XIP), and it enters in ROM_EXT code part.
- The ROM_EXT reads the Boot Info page to determine which BL0/kernel image it should be booting.
- The ROM_EXT reads the BL0 and computes its SHA2-256 digest.
- Using CRYSTALS-Dillithium hardware primitive we perform a verification of the signature inside the BL0 manifest, using the digest and the Silicon Owner Public Key derived from the Key Manager (KM) derive function. If the validation is successful, the process continues. If it fails, the process executes the boot failure logic.

Execution gets transferred to the entry point (from the BL0/Kernel image manifest) of the Silicon Owner Code. Execution enters Silicon Owner Code (i.e. BL0, Kernel Apps and Data which are all out of scope of this manuscript).

TABLE I
CRYSTALS-DILLITHIUM FPGA IMPLEMENTATION IN OPENTITAN

Design	LUTs	Relative Footprint	Clk(MHz)
OpenTitan	166.180	-	10
Dilithium	55.123	33%	2.5

We integrated the proposed approach on the original OpenTitan pipeline and evaluated on a Xilinx Virtex UltraScale+ VCU118 FPGA board. The FPGA implementation results are shown in the Table I. The proposed Dillithium hardware primitive is 33% of the whole OpenTitan design, since this implementation is tuned for high performance. In order to avoid synchronization issues, we had to clock it down to 1/4 of OpenTitan's frequency. To reduce the above cost, as a future work, we will trade-off performance for area footprint gains in our next hardware primitive design, and implement hardware accelerator solutions to offload the lattice computations of Dilithium algorithm. Offloading only the most demanding

parts of the algorithm in the hardware, i.e., the random generation and the polynomial multiplication operations, could be studied and compared against pure hardware implementation.

III. HARDWARE SECURITY CHECKERS

A. Motivation

The RISC-V architecture natively features a number of security extensions like privilege levels, physical memory protection and cryptography [15]. Nevertheless, these native mechanisms do not protect the microprocessor from two rising menaces: *Hardware Trojan Horses* and *Microarchitectural Side-Channel Attacks* [16].

Hardware Trojan Horses (HTHs) have been considered as a purely academic issue for a long time, since they generally exposed limited complexity and dangerousness. Nevertheless, a new menace recently raised, that made HTH a concern also for industries: the *software exploitable HTHs* [17]. Complex and highly dangerous HTHs may infect IP cores implementing microprocessors: such HTHs may allow the attacker to execute malicious software, to modify the running software, to acquire unauthorized privileges or to steal secret information [18]. A recent real-world HTH, the *Rosenbridge* backdoor, has been found in a commercial Via Technologies C3 processor [19]. This HTH can be activated and exploited via software to enter the supervisor mode of the system¹. Although several circuit-level design-time HTH detection techniques have been proposed [20]–[22], there is a growing interest in system-level techniques that allow to obtain a trusted system built with untrusted components [23], [24] or a trusted software execution over a (partially) untrusted system [25], [26]. We argue that existing solutions do not take into account those HTHs that change the functionality of the system by making the CPU run normal (but unexpected) instructions without changing privilege mode. In other words, none of these works checks whether the microprocessor is executing an unwanted software and whether it is accessing illegal memory locations.

Another menace that raised in the very last years are *Microarchitectural Side-Channel Attacks (MSCAs)* [27], e.g., Spectre and Meltdown. These attacks allow to steal unauthorized information without requiring the attacker to have physical access to the system under attack. Indeed, such attacks only rely on the exploitation of *legal* HW features, e.g., speculative execution and branch prediction, and on the observation of the timing behavior of the system while running sensitive applications. Several countermeasures against MSCAs have been proposed in the last few years [28]. *Constant-time* techniques, rely on making execution time constant to protect secret information; compile-time techniques have been proposed to identify and modify those instruction sequences that may open the door to MSCAs; and OS-level solutions based on cache partitioning, and periodic cache flushing have been proposed. All these techniques suffer from a limited applicability and a significant slowdown. Finally, a large

¹After the publication of [19] Via Technologies officially commented that this behavior was due to an undocumented feature meant for debug.

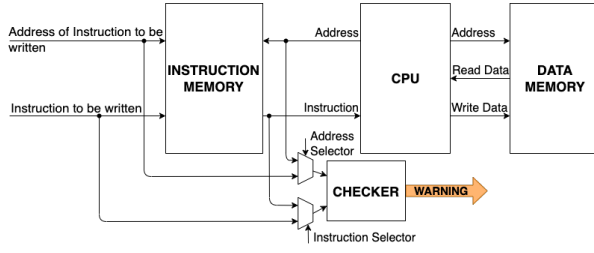


Fig. 2. The security architecture.

number of techniques that exploit machine learning and the observation of hardware performance counters exist. These techniques achieve very high detection accuracy, but they all require either multithreading or an additional (trusted and not attacked core) dedicated to the execution of the detection engine. Therefore, these techniques can be applied to high performance computing systems, e.g., high-end servers, but they are not suitable for edge devices and low-end systems, e.g., smart cards or automotive embedded systems, where a single core is available and the operating system is either not available or extremely simple.

B. The Proposed Hardware Security Solutions

Our solutions (whose high-level representation is depicted in Figure 2) aim at allowing the system integrator to build a trusted system by purchasing possibly untrusted components. Therefore, we consider the IP provider as the attacker; on the other hand the system integration employees and the foundry are considered trusted. We introduce Hardware Security Checkers (HSCs) in the architecture, such that the HSCs are able to monitor the fetching activity, the processed data and the status of the microprocessor in order to detect potentially suspicious activities, i.e., HTHs and/or MSCAs activations. These HSCs work in two phases: in a first design-time *configuration* step they *learn*² which instructions/sequences of instructions are legitimate. In subsequent runtime *monitoring* step, they check whether the activity of the microprocessor corresponds to previously learned legitimate profiles or not.

1) *Detection of Hardware Trojan Horses*: In [29] we proposed a HSC based on Bloom filters to protect the microprocessor against HTHs in the main memory. The HSC is configured while the software is installed in the instruction memory by storing information about legal instructions and corresponding memory addresses. At runtime, the HSC monitors the memory locations accessed by the microprocessor and the corresponding fetched instructions and based on this information it queries the Bloom filter. In case the accessed memory address and/or the corresponding fetched instruction are not legitimate the HSC will raise an alarm. A lighter version of the HSC has been proposed in [30].

We integrated the proposed anti-HTH security checker in the RISCY version of the PULPINO platform, which is a small

²The term “learn” is here used although the proposed solutions do not rely on machine learning/artificial intelligence.

4-stage RISC-V core. When synthesized on a Xilinx Artix XC7A35T, RISCY requires 15097 LUTs and 9881 FFs and it works at about 50MHz with a total power consumption of 127mW. We considered the following benchmarks: Binary Search (BinS), Matrix Multiplication (MM), Bubble Sort (BubS), Quick Sort (QS), Sudoku Solver (SS) and Motion Detection (MD). We simulated 10,000 cases, where the HTH forces a malicious fetch from an unexpected memory address and 10,000 cases where the HTH modifies the fetched instruction read from a legitimate address. Table II reports false positive (false alarms) and false negative (undetected attacks) rates for the two attack scenarios. It can be observed that we always have 0% false alarm rate and on average only about 2% attacks are not detected, but only when the HTH modifies the fetched instruction. Regarding the overhead, we introduce about 10% area increase, about 2% power consumption increase and no working frequency reduction.

TABLE II
HTH DETECTION ACCURACY

Bench.	Address modification		Instruction modification	
	FP	FN	FP	FN
BinS	0%	0%	0%	2.25%
MM	0%	0%	0%	0.40%
BubS	0%	0%	0%	3.01%
QS	0%	0%	0%	3.91%
SD	0%	0%	0%	0.72%
MD	0%	0%	0%	2.83%
AVG	0%	0%	0%	2.18%

2) Detection of Microarchitectural Side-Channel Attacks:

An approach similar to the previously presented ones, but meant for the identification of the activation of MSCAs has been proposed in [31]. We exploited the Count-Min Sketch model to detect at runtime the execution of pre-defined and reproducible sets of suspicious instruction patterns that are representative of specific MSCAs. These instruction patterns, one for each attack of interest, are carefully identified by the security engineer at design and then used to configure the HSC. Therefore, our proposal is able to check several microarchitectural attacks in parallel thus representing a flexible and scalable security solution.

We integrated our HSC into the RSD core, which is a 32-bit, speculative out-of-order, super-scalar, two-fetch front-end and five-issue back-end pipelines RISC-V core with 16 KByte Instruction cache. We implemented the target microprocessor on a Virtex7 FPGA employing 18334 LUTs, 10885 FFs, 4512 LUTRAM cells and 17 BRAM cells and worked at 57 MHz with an estimated power consumption of about 0.926W. We considered Coremark, Towers, RSort and Median as benign programs under attack and Spectre, Orchestration, Flush+Reload and Rowhammer as MSCAs. We ran 10000 experiments for each benign program – attack program and the result has been that 100% of the attacks has been detected. Then, we measured the number of false alarms raised by the introduced HSC: it ranged from

about 80% down to 0% depending on the configuration of the HSC itself. The smallest HSC configuration that allowed us to get both 0% false positive and false negative rates introduced about 10% area overhead, about 4% power consumption increase and no working frequency reduction.

IV. LOCK-STEP DUAL CORE

A. Motivation

Processor redundancy can be implemented in a *non-intrusive* or in an *intrusive* way, using *homogeneous* or *heterogeneous* processors. *Non-intrusive* approaches introduce redundancy at core level without modifying the internal processor micro-architecture, thus they are typically used when internal micro-architecture details are not available or difficult to modify, e.g., Commercial Off-The-Shelf (COTS) processors. For instance, two identical MicroBlaze soft cores are used to implement a homogeneous Dual Core LockStep (DCLS) [32], where the faulty processor is excluded from computation and repaired through reconfiguration, while the correct processor keeps working. A heterogeneous non-intrusive DCLS uses a RISC-V soft core along with an ARM A9 hard core [33]. Checkpoints are used in the application to check for mismatch between the cores and a roll-back strategy is used upon detection. Heterogeneous approaches may reduce performance, with the low-speed processor being the upper bound to the DCLS performance. Moreover, performing lockstep with hard cores requires specific architecture support, which is not present on all processors [33]. Overall, non-intrusive approaches are simple to implement, as no micro-architectural modifications are needed. However, they leave to the application level the responsibility to define a correction strategy (e.g., through regular checkpoints), leading to possible high timing overhead.

On the other hand, *intrusive approaches* modify the internal processor micro-architecture, improving flexibility when implementing correction mechanisms (e.g. rollback). For instance, the Dynamic Adaptive Redundancy Architecture (DARA) is a homogeneous intrusive DCLS approach applied to RISC ISA SH-2 processors and achieves error correction through rollback [34]. DARA adds additional hardware to check the consistency of all pipeline stage registers, between the lockstep cores. However, DARA does not support faults occurring in branch instructions. Another homogeneous intrusive approach uses two virtual RISC-V cores to implement a DCLS through fine-grained interleaved multitasking to tackle common-mode failures (CMFs) [35]. Other approaches extend the pipeline registers with error detection and correction codes, e.g., Duckcore extends a RISC-V core with Single Error Correction Double Error Detection (SECDED) in the pipeline registers [36]. However, such an approach can add significant overhead due to the encode and decode time and do not guarantee protection against faults in the computation logic of pipeline stages. Other approaches triplicate components within the RISC-V core to enhance its reliability. For instance, Control and Status Registers, Program Counter and the register file [37], FFs, LUTs, BRAMS, and DSPs [38], and the

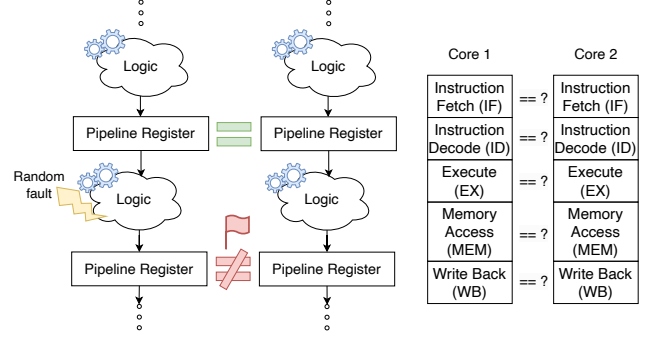


Fig. 3. Dual Core Lock-Step principle illustration

arithmetic and logic unit (ALU) are triplicated [39]. Overall, intrusive approaches offer high flexibility to implement efficient fault correction mechanisms but can be challenging to realize. Indeed, existing intrusive approaches are based on HDL implementations, which are usually quite complex.

Finally, the above described approaches do not focus on providing bounded error detection and correction time. However, this is key when embedded systems are employed in domains requiring both hard real-time and reliable execution, where the Worst-Case Execution Time (WCET) of the application has to be bounded also in presence of possible faults.

B. Proposed approach

To deal with the aforementioned problem, we proposed an intrusive homogeneous RISC-V-based dual-core lock step implementation providing transient error detection and correction with a bounded number of extra clock cycles.

In particular, in [40] we resorted to the open-source Comet RV32I RISC-V architecture³ [8] to implement two intrusive lockstep designs, i.e., *Partial Shadow Register with Rollback (PSRR)* and *Full Shadow Register (FSR)*, through High-Level Synthesis (HLS). We used two identical RISC-V cores executing the same instructions at each clock cycle. Each pipeline stage stores the result of its logic computation in a pipeline register. At each cycle, the proposed mechanism checks for execution consistency by comparing the pipeline registers of the two cores, as sketched in Figure 3. If no error is detected, the execution runs normally. Otherwise, the detected error indicates that a fault impacted the logic, which in turn generated a wrong result, or that a fault impacted the pipeline register itself, flipping a bit in the result. In this case, immediate correction is applied. In PSRR approach, the faulty instruction is re-fetched and goes through the whole pipeline again, similar to DARA [34]. Previous instructions still in the pipeline are left untouched as their execution is not impacted by the error; however, subsequent instructions already in the pipeline are discarded. In FSR approach, when no faults are detected, we create a backup copy of all the pipeline registers. Then, upon fault detection, results of the current computation are discarded, the pipeline registers of both cores are restored

³<https://gitlab.inria.fr/srokicki/Comet/-/tree/master>

with the backup values, and finally both cores re-execute the cycle that was impacted by a fault.

Moreover, in [41] we expose the following key aspect: transient faults affecting the cores impacts not only the functional behavior of an application, but it also has a significant impact on its timing behavior, affecting WCET estimations. To achieve that, we leverage typical fault-free WCET estimations to be fault-aware, by taking into account the impact of transient faults occurring on cores. More precisely, we firstly perform a vulnerability analysis on a target system through extensive fault injection. The analysis verifies not only functional correctness, but also timing correctness of applications, when executed on a core. Then, we apply a typical measurement-based WCET estimation method to verify the impact of faults on WCET estimation.

We experimented on benchmarks from TACLeBench (Binary Search and Prime), MiBench (Qsort), AxBench (Moving Average), and Polybench (Matrix Multiplication). From the obtained results, we observe that the application execution time can be significantly increased under the presence of transient faults, up to 700%, compared to the application execution time without faults. Furthermore, the distribution of execution time traces is significantly modified, compared to the fault-free distribution. The above observations have direct consequences; the time required to finish execution under faults can be significantly higher than the fault-free WCET. Thus, existing approaches should use watchdog timers, in order to bound the impact of transient faults on the application execution time, and keep safe the overall schedule. When the timer expires or an error is detected, the application requires to be re-executed, fully or partially, depending on the approach, leading to high error detection and correction timing overhead. We demonstrated that the proposed DCLS approach corrects faults as soon as they occur – before being propagated and affecting the execution time. Thus, the approach provides bounded timing overhead (i.e., two clock cycles in the FSR case) and restores WCET estimations.

V. DISCUSSION AND CONCLUSION

Edge devices based on RISC-V architectures are emerging, leading to the need for secure and reliable systems. The current work presents three mechanisms deployed on RISC-V devices towards this goal. As signature verification algorithms are proven to be unsafe in the post-quantum era, we proposed a post-quantum safe secure boot mechanism for IoT devices using OpenTitan RoT. Moreover, in order to detect potentially suspicious activities, we show hardware mechanisms to monitor the fetching activity, the processed data and the status of the microprocessor. Finally, two mechanisms with bounded WCET overhead have been shown on a dual-core lockstep intrusive RISC-V to detect and recover from radiation-induced transient faults.

Moreover, we would like to emphasize the advantages that the RISC-V open ISA can bring to the dependability research community. The large availability of open-source core

implementations and the large community around RISC-V enable researchers to easily implement and test their solutions addressing different dependability issues. However, we notice that there are not many approaches trying to address different challenges with a unified approach, e.g., the same hardware used to provide both fault tolerance and confidentiality [42]. Rather, experts of different domains (e.g., reliability and security) usually decide independently about their requirements of interest, often with limited interactions. Moreover, non-functional requirements are often considered as a optional commodity, which increases the cost of the related solutions. We argue that researchers from different groups with different expertise should federate and push dependability requirements to be among the main design requirements for future RISC-V cores to be used in mainstream applications. In this way, unified highly-optimized implementations ensuring efficient and dependable operations can thrive and power the RISC-V cores for edge computing devices of tomorrow.

ACKNOWLEDGMENT

Part of this work has been funded by the French National Research Agency (ANR) through the FASY research project (ANR-21-CE25-0008). Part of this work has been funded by the Technology Innovation Institute in Abu Dhabi, UAE.

REFERENCES

- [1] C. Palmiero *et al.*, “Design and implementation of a dynamic information flow tracking architecture to secure a risc-v core for iot applications,” in *2018 IEEE High Performance extreme Computing Conference (HPEC)*. IEEE, 2018, pp. 1–7.
- [2] J. Abella *et al.*, “Security, reliability and test aspects of the risc-v ecosystem,” in *2021 IEEE European Test Symposium (ETS)*, 2021, pp. 1–10.
- [3] J. Anders *et al.*, “A survey of recent developments in testability, safety and security of risc-v processors,” in *2023 28th IEEE European Test Symposium (ETS)*, 2023.
- [4] A. Dörflinger *et al.*, “A comparative survey of open-source application-class risc-v processor implementations,” in *Proceedings of the 18th ACM International Conference on Computing Frontiers*, ser. CF ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 12–20. [Online]. Available: <https://doi.org/10.1145/3457388.3458657>
- [5] Y. H. Hwang, “Iot security & privacy: threats and challenges,” in *Proceedings of the 1st ACM workshop on IoT privacy, trust, and security*, 2015, pp. 1–1.
- [6] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM review*, vol. 41, no. 2, pp. 303–322, 1999.
- [7] M. Cui *et al.*, “Fault-tolerant mapping of real-time parallel applications under multiple dvfs schemes,” in *IEEE RTAS*, 2021, pp. 387–399.
- [8] S. Rokicki *et al.*, “What you simulate is what you synthesize: Designing a processor core from c++ specifications,” in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.
- [9] V. Mavroeidis *et al.*, “The impact of quantum computing on present cryptography,” *arXiv preprint arXiv:1804.00200*, 2018.
- [10] V. B. Kumar *et al.*, “Post-quantum secure boot,” in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2020, pp. 1582–1585.
- [11] P. Kampnakis *et al.*, “Post-quantum lms and sphincs+ hash-based signatures for uefi secure boot,” *Cryptology ePrint Archive*, 2021.
- [12] A. Khalid *et al.*, “Lattice-based cryptography for iot in a quantum world: Are we ready?” in *2019 IEEE 8th international workshop on advances in sensors and interfaces (IWASI)*. IEEE, 2019, pp. 194–199.

- [13] J. Zheng *et al.*, "Parallel small polynomial multiplication for dilithium: A faster design and implementation," in *Proceedings of the 38th Annual Computer Security Applications Conference*, ser. ACSAC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 304–317. [Online]. Available: <https://doi.org/10.1145/3564625.3564629>
- [14] L. Beckwith *et al.*, "High-performance hardware implementation of crystals-dilithium," in *2021 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 2021, pp. 1–10.
- [15] A. Waterman *et al.*, "The risc-v instruction set manual volume ii: Privileged architecture document version 20190608-priv-msu-ratified," RISC-V Foundation, Tech. Rep., 2019.
- [16] L. Cassano *et al.*, "Is risc-v ready for space? a security perspective," in *2022 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2022, pp. 1–6.
- [17] X. Wang *et al.*, "Software exploitable hardware trojans in embedded processor," in *2012 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2012, pp. 55–58.
- [18] Y. Jin *et al.*, "Exposing vulnerabilities of untrusted computing platforms," in *Proc. Int. Conf. Computer Design*, 2012, pp. 131–134.
- [19] C. Domas, "Hardware backdoors in x86 cpus," <https://i.blackhat.com/us-18/Thu-August-9/us-18-Domas-God-Mode-Unlocked-Hardware-Backdoors-In-x86-CPU-wp.pdf>, 2018.
- [20] X. Chuan *et al.*, "An efficient triggering method of hardware Trojan in AES cryptographic circuit," in *Proc. Int. Conf. Integrated Circuits and Microsystems*, 2017, pp. 91–95.
- [21] Y. Liu *et al.*, "Scca: Side-channel correlation analysis for detecting hardware trojan," in *Proc. Int. Conf. Anti-counterfeiting, Security, and Identification*, 2017, pp. 196–200.
- [22] A. Palumbo *et al.*, "Is your fpga bitstream hardware trojan-free? machine learning can provide an answer," *Journal of Systems Architecture*, vol. 128, p. 102543, 2022.
- [23] J. Dubeuf *et al.*, "Run-time detection of hardware trojans: The processor protection unit," in *2013 18th IEEE European Test Symposium (ETS)*, 2013, pp. 1–6.
- [24] G. Bloom *et al.*, "Os support for detecting trojan circuit attacks," in *2009 IEEE International Workshop on Hardware-Oriented Security and Trust*, 2009, pp. 100–103.
- [25] L. Cassano *et al.*, "Deton: Defeating hardware trojan horses in microprocessors through software obfuscation," *Journal of Systems Architecture*, vol. 129, p. 102592, 2022.
- [26] —, "On the optimization of software obfuscation against hardware trojans in microprocessors," in *2022 25th International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS)*, 2022, pp. 172–177.
- [27] A. P. Fournaris *et al.*, "Exploiting hardware vulnerabilities to attack embedded system devices: a survey of potent microarchitectural attacks," *Electronics*, vol. 6, no. 3, p. 52, 2017.
- [28] Y. Lyu *et al.*, "A survey of side-channel attacks on caches and countermeasures," *Journal of Hardware and Systems Security*, vol. 2, no. 1, pp. 33–50, 2018.
- [29] A. Bolat *et al.*, "A microprocessor protection architecture against hardware trojans in memories," in *2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*. IEEE, 2020, pp. 1–6.
- [30] A. Palumbo *et al.*, "A lightweight security checking module to protect microprocessors against hardware trojan horses," in *2021 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 2021, pp. 1–6.
- [31] K. Arıkan *et al.*, "Processor security: Detecting microarchitectural attacks via count-min sketches," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 7, pp. 938–951, 2022.
- [32] A. Hanafi *et al.*, "Dual-lockstep microblaze-based embedded system for error detection and recovery with reconfiguration technique," in *2015 Third World Conference on Complex Systems (WCCS)*, 2015, pp. 1–6.
- [33] A. B. De Oliveira *et al.*, "Lockstep Dual-Core ARM A9: Implementation and Resilience Analysis Under Heavy Ion-Induced Soft Errors," *IEEE Transactions on Nuclear Science*, vol. 65, no. 8, pp. 1783–1790, Aug. 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8401918/>
- [34] J. Yao *et al.*, "Dara: A low-cost reliable architecture based on unhardened devices and its case study of radiation stress test," *IEEE Transactions on Nuclear Science*, vol. 59, no. 6, pp. 2852–2858, 2012.
- [35] M. T. Sim *et al.*, "A dual lockstep processor system-on-a-chip for fast error recovery in safety-critical applications," in *IEEE IECON*, 2020, pp. 2231–2238.
- [36] J. Li *et al.*, "Duckcore: A fault-tolerant processor core architecture based on the risc-v isa," *Electronics*, vol. 11, no. 1, 2022.
- [37] L. Blasi *et al.*, "A RISC-V fault-tolerant microcontroller core architecture based on a hardware thread full/partial protection and a thread-controlled watch-dog timer," in *APPLEPIES*, 2019, pp. 505–511.
- [38] A. E. Wilson *et al.*, "Neutron radiation testing of fault tolerant risc-v soft processor on xilinx sram-based fpgas," in *IEEE SCC*, 2019, pp. 25–32.
- [39] D. A. Santos *et al.*, "A low-cost fault-tolerant risc-v processor for space systems," in *DTIS*, 2020, pp. 1–5.
- [40] P. R. Nikiema *et al.*, "Design with low complexity fine-grained dual core lock-step (dcls) risc-v processors," in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, July 2023.
- [41] —, "Impact of transient faults on timing behavior and mitigation with near-zero wcet overhead," in *ECRTS 2023 - 35th Euromicro Conference on Real-Time Systems*, Vienna, Austria, July 2023.
- [42] N. I. Deligiannis *et al.*, "Towards the integration of reliability and security mechanisms to enhance the fault resilience of neural networks," *IEEE Access*, vol. 9, pp. 155 998–156 012, 2021.

西安电子科技大学
毕业设计（论文）指导情况登记表

专业	空间科学与技术	题目	RISCV 开源 IP 平台对于移植开源飞控的资源匹配性研究		
学生姓名	何成楷	学号	21009100863	教师姓名	韩高
指导次数	指导内容	指导效果		指导时间	指导教师签名
1	明确研究任务，包括选定国产化 RISC-V 平台、对比开源飞控 PX4 等。 3. 确定研究计划和时间安排。	明确了研究方向和任务分工。制定了初步的研究计划，为后续研究奠定了基础。		2024.12.25	韩高
2	1. 讨论国产化 RISC-V 平台的选取标准，包括平头哥、芯来、沁恒等平台的特点和优势。	1. 对国产化 RISC-V 平台有了更深入的了解，明确了选取芯片的标准和方向		2025.1.3	韩高
3	1. 指导学生如何收集和整理相关芯片的外设资源、工具链等信息。 2. 讨论周边传感器配套和电机驱动控制的重要性。	1. 掌握了收集和整理信息的方法，能够更系统地获取相关数据。 2. 明白了周边传感器配套和电机驱动控制在飞控中的关键作用。		2025.1.10	韩高
4	对比分析 RISC-V 平台与现有 STM32F4/H7 芯片	1. 能够初步完成对比分析，找出两者在外设资源等方面的差异。 2. 提高了数据分析能力，能够更准确地表达对比结果。		2025.2.26	韩高
5	1. 研究国产化 RISC-V 平台在小型行业应用无人机的应用可行性。 2. 讨论可能遇到的问题及解决方案	1. 对 RISC-V 平台在无人机应用中的可行性有了初步判断。 2. 能够提出一些可能的问题，并思考相应的解决方案。		2025.3.2	韩高
6	1. 指导学生如何查阅相关文献和技术	1. 掌握了查阅文献的方法，能够找到更多支持研究的资料。		2025.3.8	韩高
7	1. 讨论 RISC-V 国产化芯片实现开源飞控平台全面替代的可行性。 2. 分析全自主芯片+全自主飞控的发展设计路径。	1. 对全面替代的可行性有了更深入思考。 2. 初步明确了全自主芯片+全自主飞控的发展方向。		2025.3.15	韩高

指导次数	指导内容	指导效果	指导时间	指导教师签名
8	1. 指导学生如何撰写研究报告，包括结构安排和内容撰写要点。2. 讨论报告中需要重点突出的内容。	1. 掌握了研究报告的撰写方法，能够合理安排报告结构。2. 明确了报告中需要重点关注和详细阐述的内容。	2025.3.22	韩高
9	1. 审阅学生撰写的初步研究报告，提出修改意见。2. 指导学生如何进一步完善报告内容。	1. 根据导师的修改意见，对报告进行了初步修改和完善。2. 提高了报告撰写的质量，使内容更加清晰、准确。	2025.4.1	韩高
10	1. 继续讨论报告内容的完善，重点关注数据的准确性和分析的深度。2. 指导学生如何进行逻辑推理和论证。	1. 进一步完善了报告内容，确保数据准确无误。2. 提高了逻辑推理和论证能力，使报告更具说服力。	2025.4.8	韩高
11	1. 指导学生如何进行实验验证，以支持研究结论。2. 讨论实验方案的设计和实施方案。	1. 明确了实验验证的必要性和方法。2. 制定了实验方案，并准备开始实验。	2025.4.15	韩高
12	1. 指导学生进行实验操作，确保实验过程规范、准确。2. 讨论实验数据的记录和分析方法。	1. 按照实验方案顺利完成了实验操作。2. 掌握了实验数据的记录和分析方法，能够从实验数据中得出结论。	2025.4.22	韩高
13	1. 审阅实验报告，提出修改意见。2. 指导学生如何将实验结果与研究报告相结合。	1. 根据导师的修改意见，对实验报告进行了完善。2. 将实验结果成功融入研究报告，使结论更具可信度。	2025.4.30	韩高
14	1. 讨论研究报告的最终完善，重点关注语言表达和格式规范。2. 指导学生如何进行论文查重和修改。	1. 对研究报告进行了最终完善，确保语言表达流畅、格式规范。2. 掌握了论文查重的方法，并根据查重结果进行了修改。	2025.5.8	韩高
15	1. 模拟论文答辩，指导学生如何进行答辩陈述。	1. 熟悉了答辩流程，能够清晰、准确地进行答辩陈述。	2025.5.15	韩高
16	1. 最后一次指导，总结整个研究过程中的收获和不足。2. 对学生未来的研究方向提出建议。	1. 对整个研究过程进行了全面总结，明确了收获和不足之处。2. 获得了导师对未来研究方向的建议，为今后的研究提供了指导。	2025.5.29	韩高

说明：

1. 本表由学生填写；
2. 学生提出的问题或导师指导的问题均填入指导内容栏内，经教师指导将解决结果记在效果栏内，并请指导教师签名作为记载。
3. 各学院可根据本学院专业认证要求及细则对表格进行修改，修改后的表格需向本科生院备案。